

Java Design Patterns



Course Development Team

**Chád
Darby**



**Ravi
Singh**

Java Design Patterns

- Design patterns are a simple way to solve common recurring problems
- They help you write clean code that is easy to understand
- Patterns are used in the software industry for enterprise applications
- Provides a common vocabulary for software architectures

Java Design Patterns

Creational

Factory
Singleton
Prototype

...

Structural

Adapter
Bridge
Composite

...

Behavioral

Iterator
Observer
Strategy

...

Prerequisites

Java Programming Introduction

- You are already experienced with Java
 - OOP, classes, interfaces, inheritance, exception handling, collections
- You should have the following items already installed
 - Java Development Kit (JDK)
 - Java IDE (we'll use IntelliJ in the videos, but any Java IDE will work)

About IntelliJ

Super Amazing IDE!!!

- In this course, we will use the free version of IntelliJ
 - Known as **IntelliJ Community Edition**
 - Download from: **<https://www.jetbrains.com/idea/download>**
 - Select **Community Edition**
- You can also use the Ultimate Edition (\$) ... free trial version is available

Source Code and PDFs

All source code is available for download

All PDFs of slides are available for download

Questions / Help

If you have questions or need help...

Post the question in the classroom discussion forum

Java Design Patterns

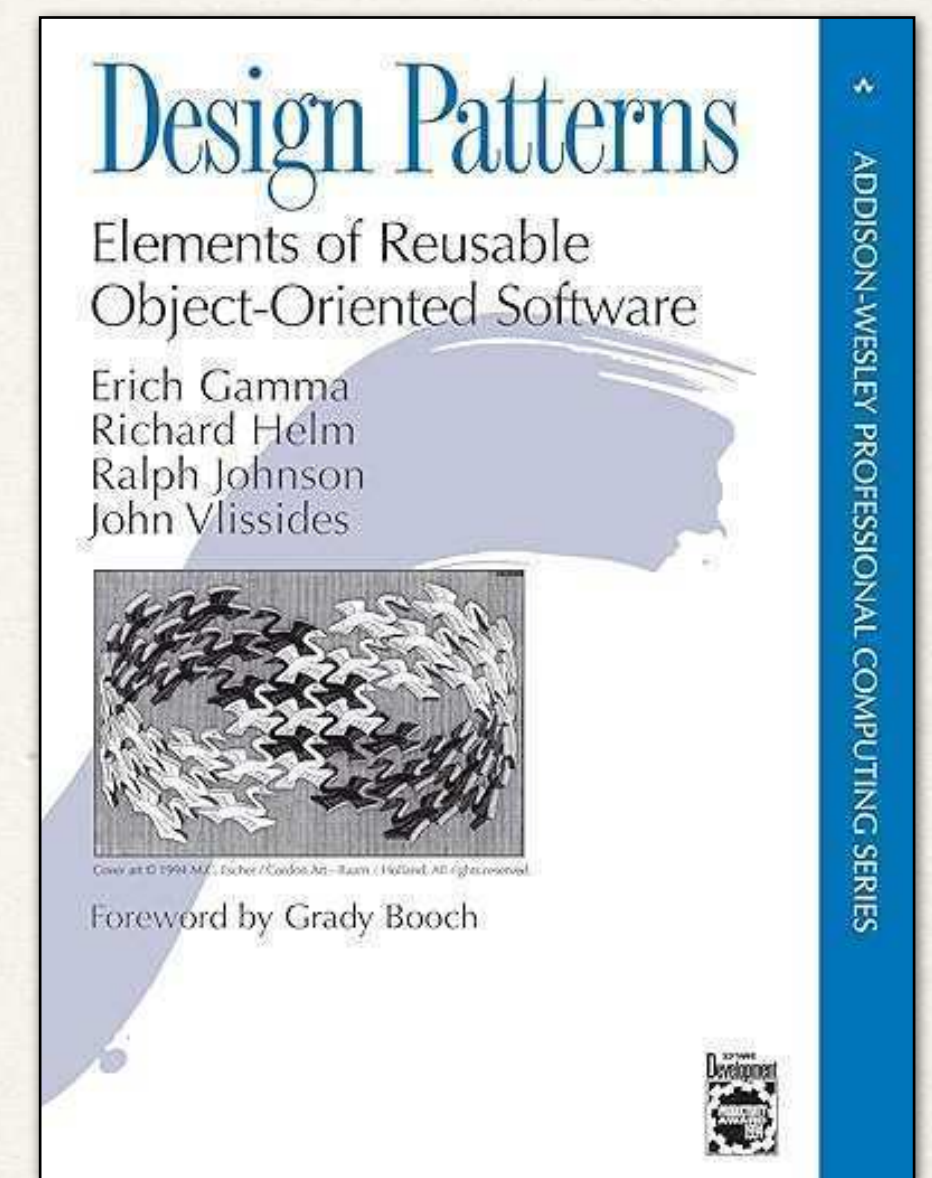


What Are Design Patterns?

- Design patterns are a simple way to solve common recurring problems
- They help you write clean code that is easy to understand
- Patterns are used in the software industry for enterprise applications
- Provides a common vocabulary for software architectures

Popular Design Patterns Book

- *Design Patterns: Elements of Reusable Object-Oriented Software*
- Authors: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides
- Affectionately known as the **Gang of Four (GoF)** book
- Frequently referenced by the developer community



Java Design Patterns

Creational

Factory
Singleton
Prototype

...

Structural

Adapter
Bridge
Composite

...

Behavioral

Iterator
Observer
Strategy

...

Describing Design Patterns

- When describing the patterns, we can use a general format
- **Intent**
 - What is the pattern meant to do?
 - *What is the goal of this pattern?*
- **Problem**
 - The recurring issue the pattern solves
 - *Why would I need this pattern?*
- **Solution**
 - The general approach used to solve the problem
 - *How does this pattern resolve the issue?*

Intent

Problem

Solution

Creational Patterns

Creational

Factory
Singleton
Prototype

...

- Focus on **how objects are created**
- Helps to decouple object creation from object usage
- Provides flexibility in deciding which object is needed for a given use case
- Makes the system independent of how objects are created, composed and used
- Abstracts away the complexity of the object request
 - Just tell the system what you need

Creational Patterns - Analogies

- Restaurant
 - Ordering from a menu ... I'll have the fish and chips
- Auto dealership
 - Tell the salesperson ... I want to buy a new mini-van
- Home building
 - Tell construction lead ... build me a three bedroom house

Creational

Factory
Singleton
Prototype

...

Creational patterns abstract away the complexity of object creation

Factory Design Pattern

Intent

Problem

Solution

- **Intent**

- Provide a way to create objects using a common interface ... without exposing creation logic
- Allows the actual object type to be decided at runtime by a subclass or factory method

- **Problem**

- When you have multiple related classes
- You want to localize / centralize the object creation

- **Solution**

- Use a factory class with a method that decides which object to create, based on input

Benefits

Centralizes object creation
Simplifies client code
Easy to add new types later

Factory Design Pattern Example: Drawing App

- **Use Case**

- A drawing app needs to create different shapes based on user input

- **Solution**

- Factory name: ShapeFactory
- Interface: Shape
- ShapeFactory creates different types of Shapes: Circle, Square, ...

- **Why Factory?**

- User selects a Shape from toolbar and passes it to ShapeFactory
- Based on user input, the correct Shape is returned

Benefits

Centralizes object creation
Simplifies client code
Easy to add new types later

Factory Design Pattern Example: Payment Processor

- **Use Case**

- An e-commerce app needs to support different payment methods (Credit Card, Apple Pay, ...)

- **Solution**

- Factory name: `PaymentProcessorFactory`
- Interface: `PaymentProcessor`
- `PaymentProcessorFactory` creates different types of `PaymentProcessor`:
 - `CreditCardProcessor`, `ApplePayProcessor`, ...

Benefits

Centralizes object creation
Simplifies client code
Easy to add new types later

- **Why Factory?**

- Based on user selection at checkout, the correct `PaymentProcessor` is returned

Factory Design Pattern Example: Notifications

- **Use Case**

- An app needs to send different types of notifications (Email, SMS, WhatsApp ...)

- **Solution**

- Factory name: `NotificationFactory`
- Interface: `Notification`
- `NotificationFactory` creates different types of `Notification`:
 - `EmailNotification`, `SmsNotification`, ...

- **Why Factory?**

- Client code doesn't care how notification is created, just give me the desired notification

Benefits

Centralizes object creation
Simplifies client code
Easy to add new types later

Factory Design Pattern Notification Example



Factory Design Pattern Example: Notifications

- **Use Case**

- An app needs to send different types of notifications (Email, SMS, WhatsApp ...)

- **Solution**

- Factory name: `NotificationFactory`
- Interface: `Notification`
- `NotificationFactory` creates different types of `Notification`:
 - `EmailNotification`, `SmsNotification`, ...

- **Why Factory?**

- Client code doesn't care how notification is created, just give me the desired notification

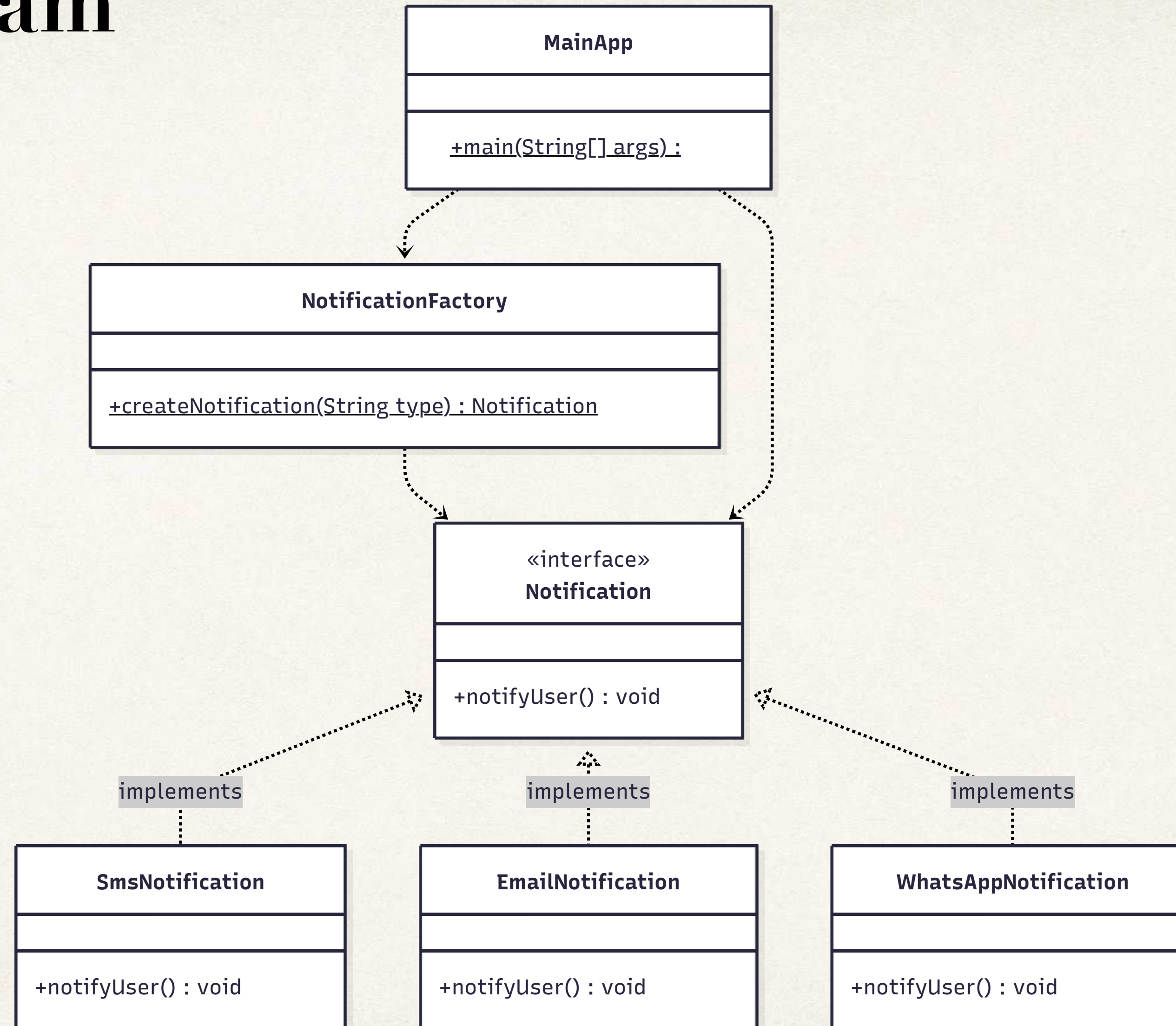
Benefits

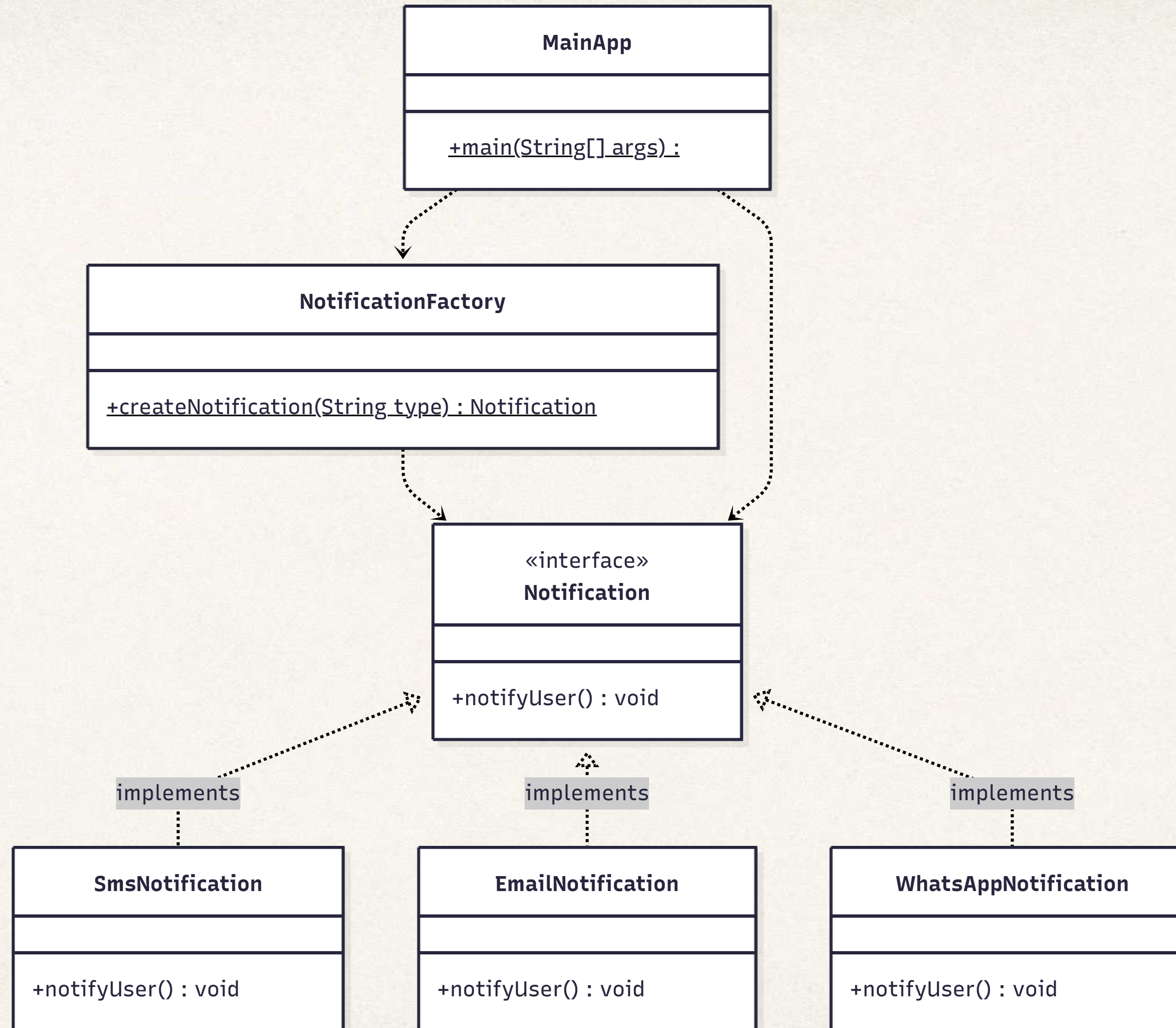
Centralizes object creation
Simplifies client code
Easy to add new types later

Components

Name	Description
Notification	Defines a common method: <code>notifyUser()</code> for all notification types
SmsNotification	Class that implements <code>Notification</code> , sends an SMS
EmailNotification	Class that implements <code>Notification</code> , sends an email
WhatsAppNotification	Class that implements <code>Notification</code> , sends WhatsApp message
NotificationFactory	Factory class that creates and returns the correct <code>Notification</code> object
MainApp	Client code that uses the factory to request a notification and call the <code>notifyUser()</code> method

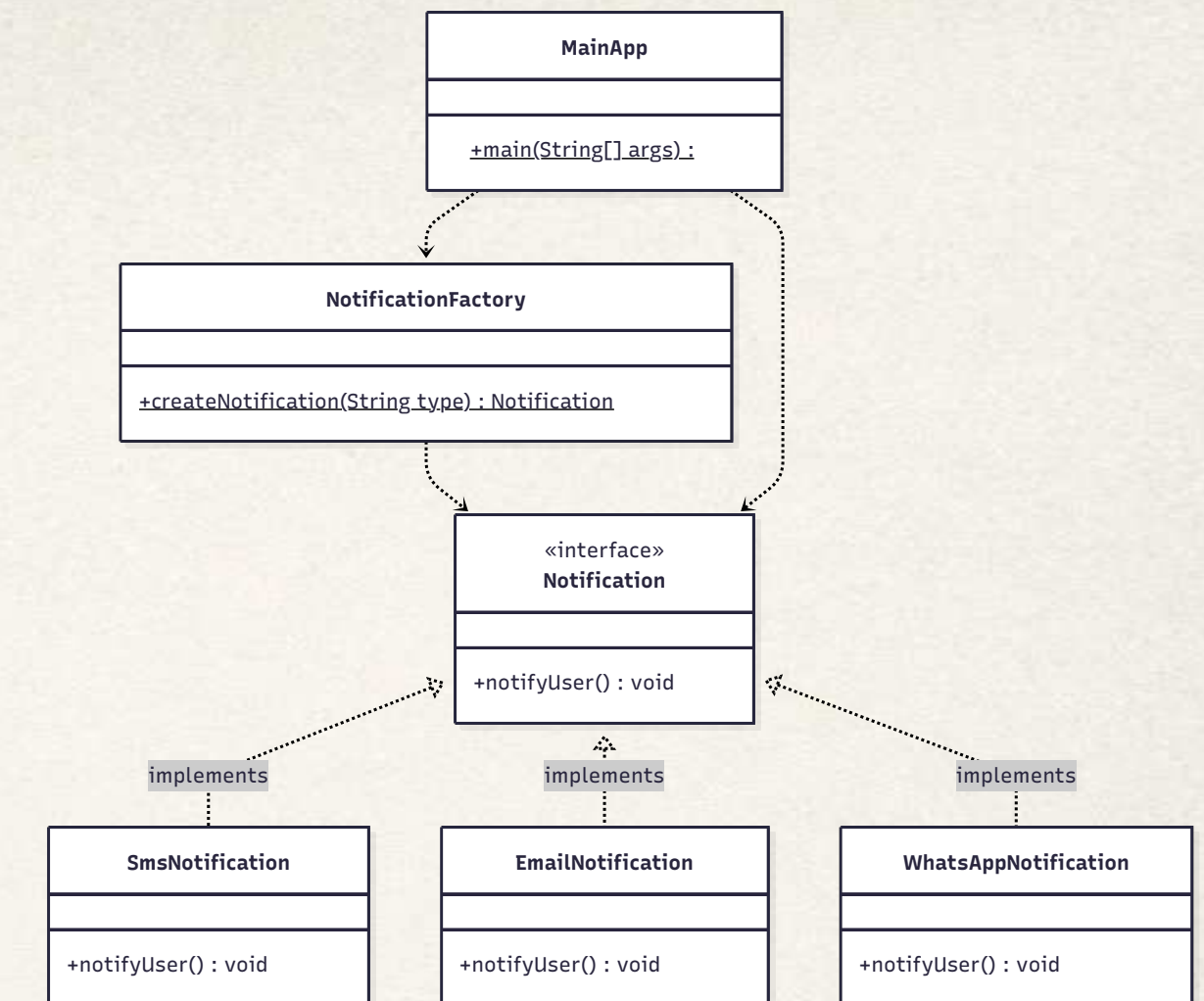
UML Diagram





Notification Interface and Implementations

```
public interface Notification {  
  
    void notifyUser();  
  
}
```

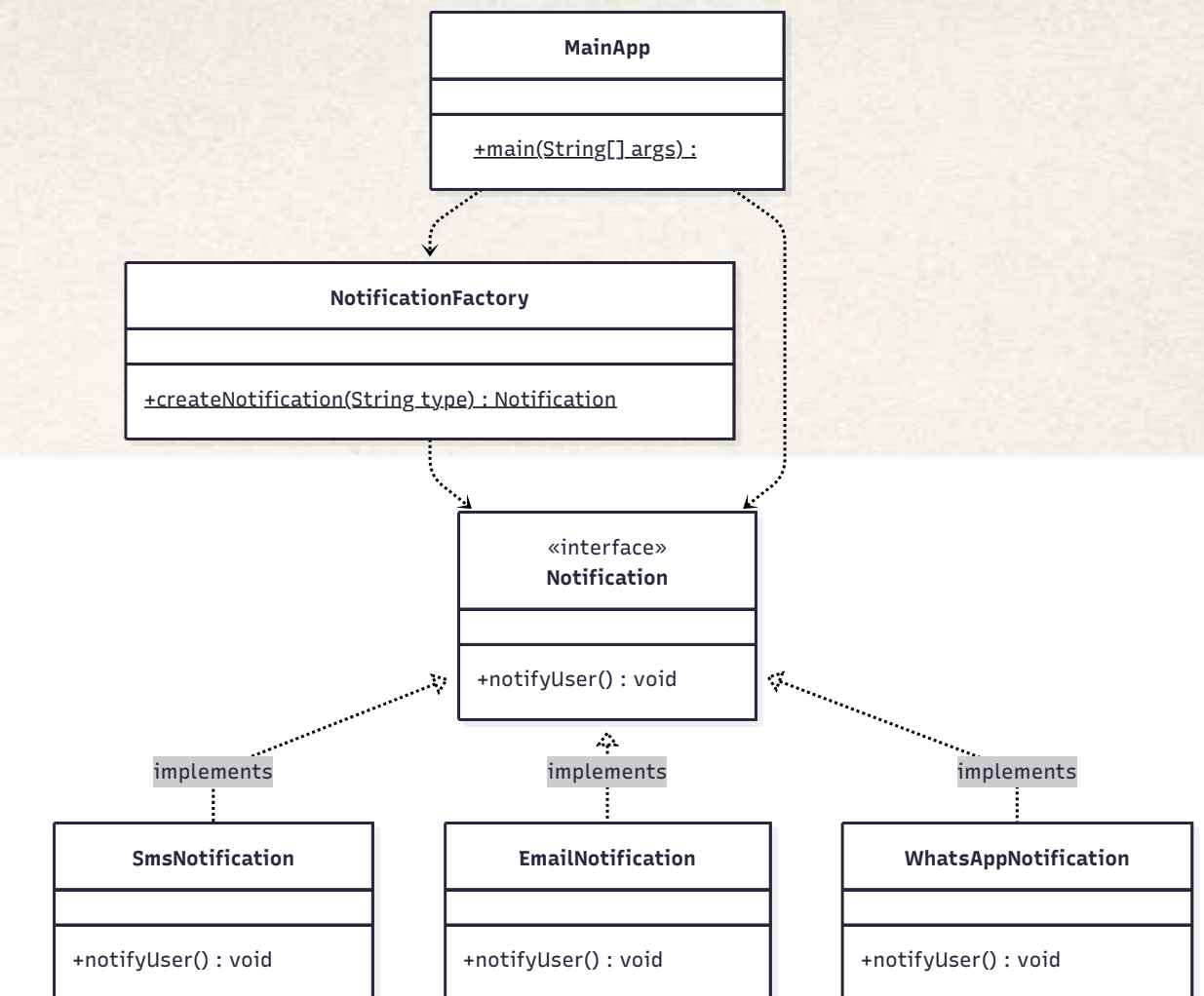


```
public class SmsNotification implements Notification {  
  
    @Override  
    public void notifyUser() {  
        System.out.println("Sending sms ...");  
    }  
  
}
```

```
public class EmailNotification implements Notification {  
  
    @Override  
    public void notifyUser() {  
        System.out.println("Sending email ...");  
    }  
  
}
```


NotificationFactory

```
public class NotificationFactory {  
  
    public static Notification createNotification(String type) {  
  
        if (type == null || type.isEmpty()) {  
            return null;  
        }  
  
        switch (type.toLowerCase()) {  
            case "email":  
                return new EmailNotification();  
            case "sms":  
                return new SmsNotification();  
            case "whatsapp":  
                return new WhatsAppNotification();  
            default:  
                throw new IllegalArgumentException("Unknown notification type: " + type);  
        }  
    }  
}
```



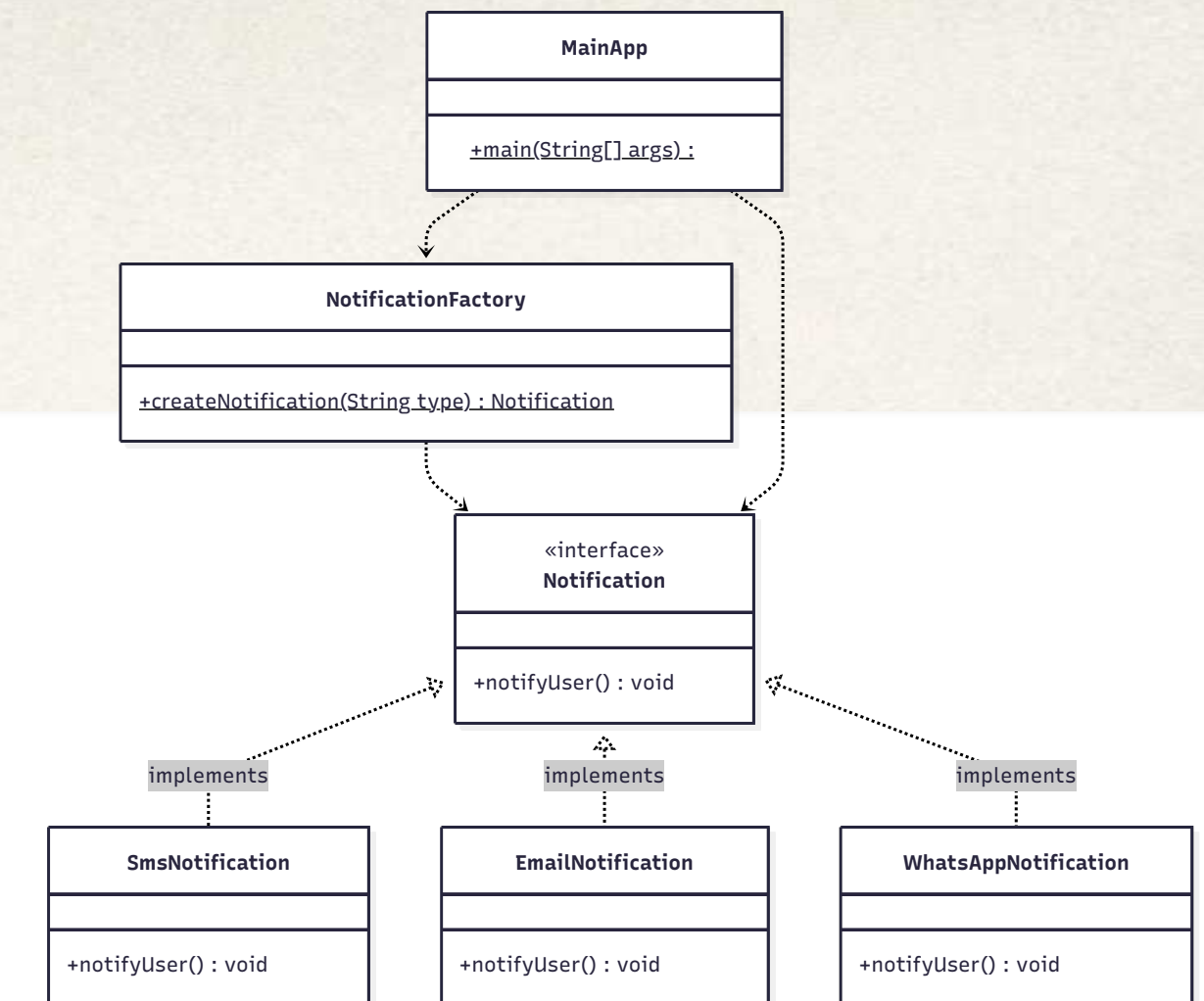
Benefits

Centralizes object creation
Simplifies client code
Easy to add new types later

MainApp

Let the factory create the objects

```
public class MainApp {  
  
    public static void main(String[] args) {  
  
        Notification emailNotification = NotificationFactory.createNotification("email");  
        emailNotification.notifyUser();  
  
        Notification smsNotification = NotificationFactory.createNotification("sms");  
        smsNotification.notifyUser();  
  
        Notification whatsappNotification = NotificationFactory.createNotification("whatsapp");  
        whatsappNotification.notifyUser();  
  
    }  
}
```



Client code doesn't care how notification is created, just give me the desired notification.

Development Process

- **Initial implementation**
 - Use a trivial implementation that WILL NOT use Factory design pattern
- **Refactor our code**
 - Update the application to use the Factory design pattern

Abstract Factory Design Pattern



Abstract Factory Design Pattern

Intent

Problem

Solution

- **Intent**

- Create **related object families** using an interface ... without binding to specific classes

- **Problem**

- When you have **sets of related classes** that must work together
- Don't want to bind code to specific implementations

- **Solution**

- Use an abstract factory with methods to build each product type
- Concrete factories return platform- or role-specific objects
- Client code stays clean and unaware of class details

Benefits

Keeps code modular and consistent
Supports swapping of object families
Reduces hardcoded logic

Abstract Factory Example: Portable UI Framework

- **Use Case**

- Building a portable UI framework that will work on multiple platforms: MS Windows, Mac, Linux
- Each platform has its own look-and-feel (LAF) for UI elements: buttons, text fields, windows

- **Solution**

- Factory name: `UIComponentFactory`
- Concrete Factories: `WindowsUIComponentFactory`, `MacUIComponentFactory`, ...
- Interfaces: `Button`, `TextField`
- Each factory returns platform specific components:
 - `WindowsButton`, `MacButton`, ...

- **Why Abstract Factory?**

- Keeps UI creating consistent across platforms
- Let's you switch platforms by swapping factories

Benefits

Keeps code modular and consistent
Supports swapping of object families
Reduces hardcoded logic

Abstract Factory Example: Travel App

- **Use Case**

- Building a travel app with multiple providers (Expedia, Priceline, ...)
- Each provider returns different formats for flights, hotels, car ... we need a unified format

- **Solution**

- Factory name: `ItineraryFactory`
- Concrete Factories: `ExpediaItineraryFactory`, `PricelineItineraryFactory`, ...
- Interfaces: `FlightBooking`, `HotelBooking`, `CarRentalBooking`
- Each factory returns provider specific components:
 - `ExpediaFlightBooking`, `PricelineFlightBooking`, ...

- **Why Abstract Factory?**

- Provide consistent structure for integrating various APIs
- Allows switching providers

Benefits

Keeps code modular and consistent
Supports swapping of object families
Reduces hardcoded logic

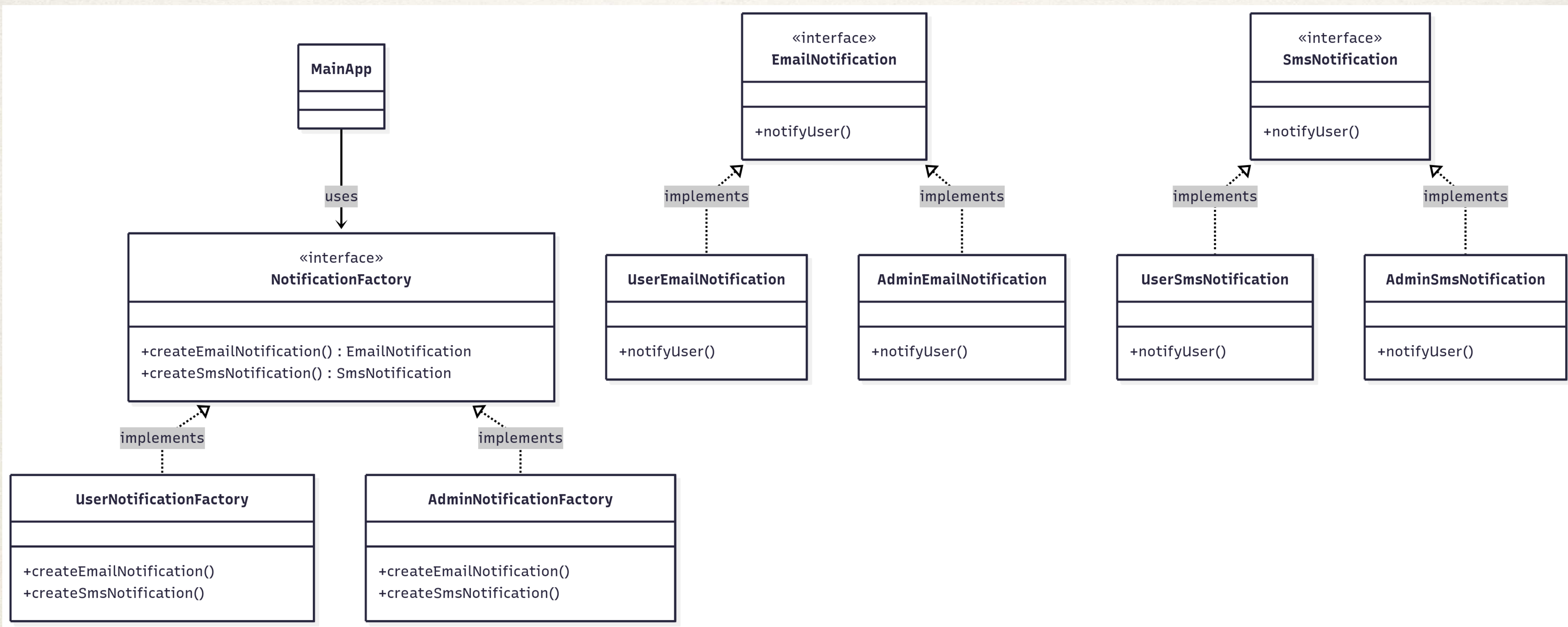
Abstract Factory Example: Notifications

- **Use Case**
 - Building a notification system to send notifications to different user roles: Admin, User etc ...
 - Each role has its own custom formatting for email and SMS messages
- **Solution**
 - Factory name: `NotificationFactory`
 - Concrete Factories: `AdminNotificationFactory`, `UserNotificationFactory`, ...
 - Interfaces: `AdminNotification`, `UserNotification`
 - Each factory returns specific notifications:
 - `AdminEmailNotification`, `UserEmailNotification`, ...
- **Why Abstract Factory?**
 - Cleanly separates role-based logic and simplifies adding new roles

Benefits

Keeps code modular and consistent
Supports swapping of object families
Reduces hardcoded logic

UML Diagram



Development Process

Step-By-Step

1. Create Email components

- EmailNotification, UserEmailNotification and AdminEmailNotification

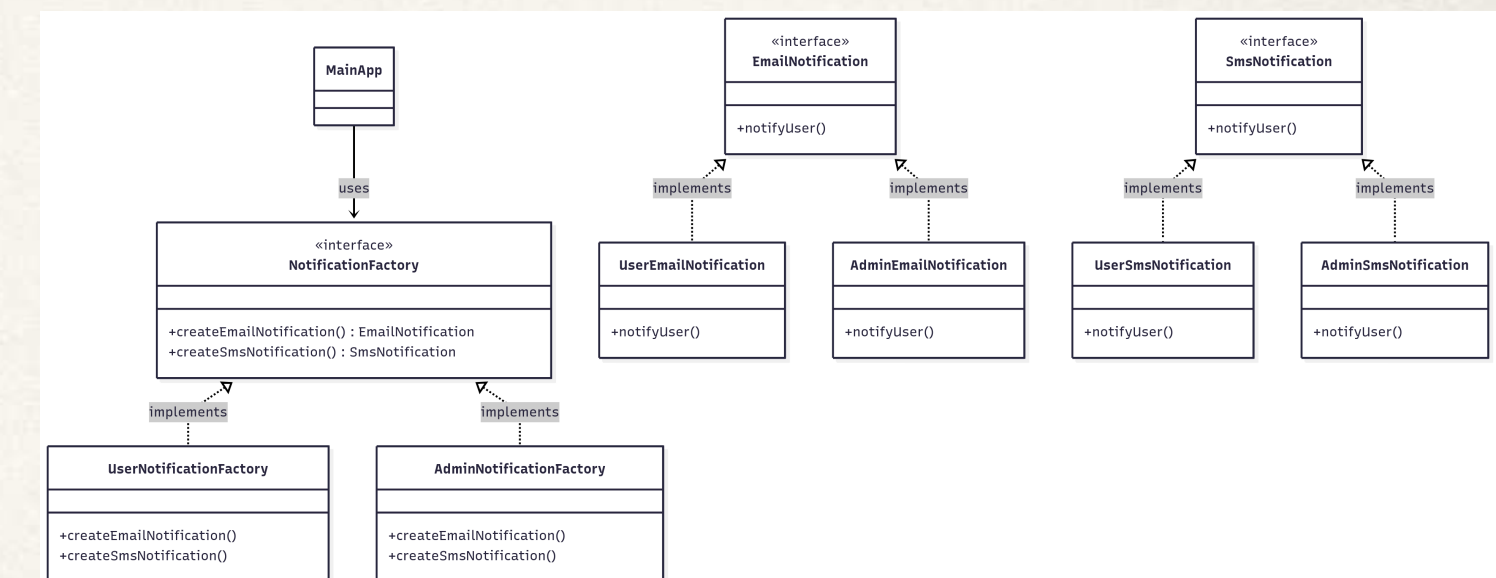
2. Create SMS components

- SmsNotification, UserSmsNotification and AdminSmsNotification

3. Create the Factory components

- NotificationFactory, UserNotificationFactory and AdminNotificationFactory

4. Create the Main App



Singleton Design Pattern



Singleton Design Pattern

Intent

Problem

Solution

- **Intent**

- Ensure a class has only one instance and provide easy access to the instance

- **Problem**

- When you want to control the number of object instances
- For example, you only want one logger or one repository

Benefits

Controls object creation
Reduces resource usage
Easy access to the instance

- **Solution**

- Make the constructor private
- Provide a static method that returns the single shared instance

Singleton Example: Java Runtime class

- **Use Case**

- The Java Runtime class provides system-level operations such as memory stats, process execution ...
- Only one Runtime instance is available in a Java Virtual Machine (JVM)

- **Solution**

- Class: Runtime
- Singleton method: `Runtime.getRuntime()`
- Provides a single access point to OS-level features

- **Why Singleton?**

- Prevents multiple runtime environments from being created
- Simplifies access and ensures consistent system control

Benefits

Controls object creation
Reduces resource usage
Easy access to the instance

Singleton Example: Spring Application Context

- **Use Case**
 - Spring apps manage many components (beans)
 - Most beans should be shared and reused during the application lifecycle
- **Solution**
 - Class: `ApplicationContext`
 - Manages bean instantiation and lifecycle
 - By default, returns one shared instance (singleton)
- **Why Singleton?**
 - Promotes loose coupling and testability
 - Efficient memory and lifecycle management

Benefits

Controls object creation
Reduces resource usage
Easy access to the instance

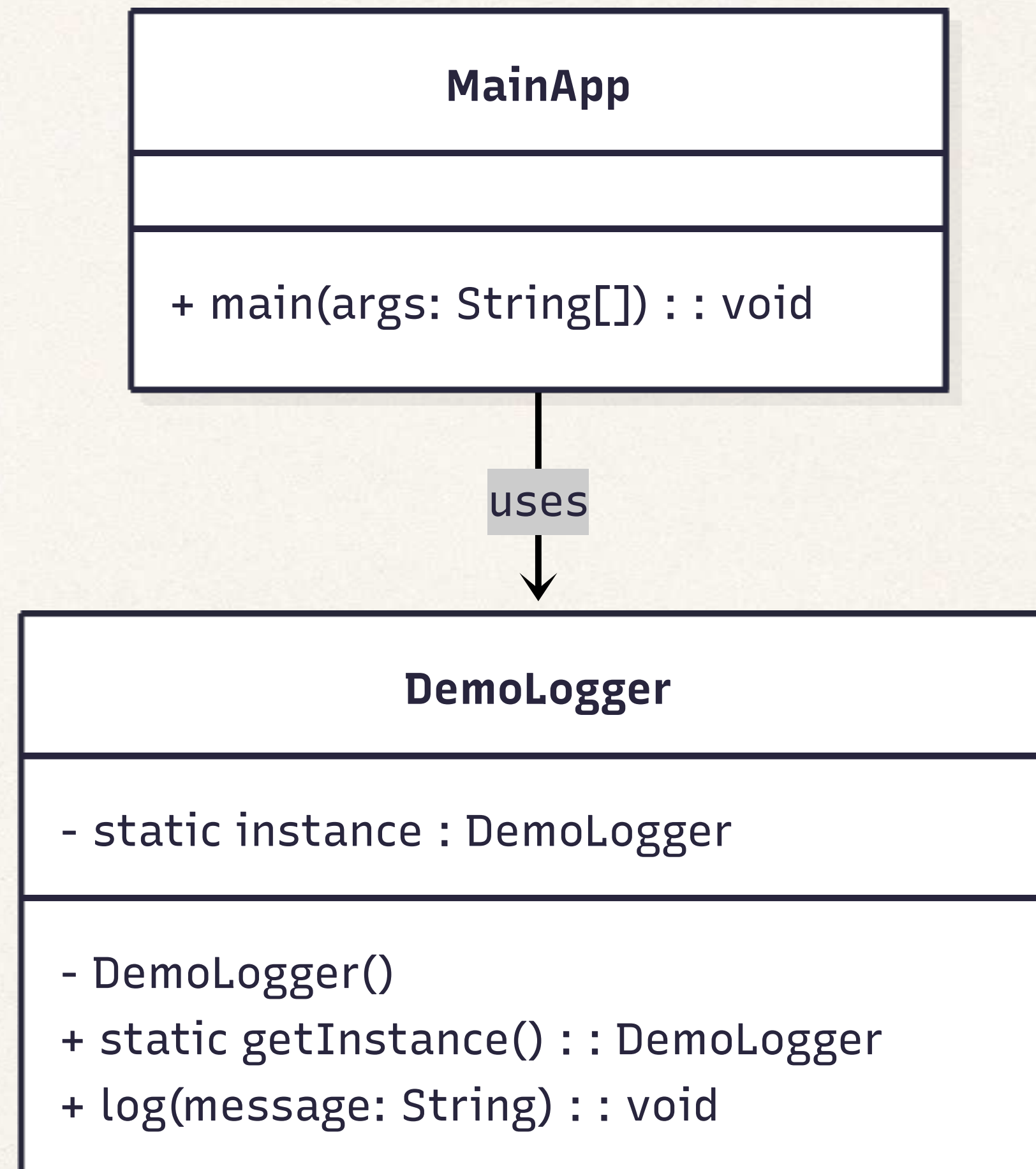
Singleton Example: Logger

- **Use Case**
 - Logging is required across many classes
 - A single logger is sufficient
- **Solution**
 - Class: DemoLogger
 - Singleton method: `DemoLogger.getInstance()`
 - Provides a single access point to the shared logger
- **Why Singleton?**
 - Centralizes logger creation

Benefits

Controls object creation
Reduces resource usage
Easy access to the instance

UML Diagram



Development Process

Step-By-Step

1. Create a static field to hold the single shared instance
2. Make the constructor private
3. Provide a static method that returns the single shared instance

Singleton Implementation Variations

Name	Description
Lazy Initialization	Creates the instance only when it is first requested
Eager Initialization	Creates the instance at class loading time, regardless of usage
Thread Safe	Ensures only one instance is created in a multi-threaded environment using synchronized access
Bill Pugh Implementation	Uses a static inner helper class to achieve lazy loading and thread safety without synchronization overhead

Builder Design Pattern



Builder Design Pattern

Intent

Problem

Solution

- **Intent**

- Build complex objects step-by-step
- Keep the building process separate from the final object

- **Problem**

- Need to create objects with multiple fields without multiple overloaded constructors

- **Solution**

- Use a Builder that accepts fields in fluent-style ... chained method calls
- Call `build()` to create the final object

Benefits

Easy to create objects with many options
Readable fluent API
Makes objects immutable once built

Examples

- Custom Meal Ordering App
- Java JDK Locale
- OpenAI chat completions request

Builder Example: Custom Meal Ordering App

Bob's

- **Use Case**
 - A Meal has required items (burger) and optional items (drink, fries).
 - Multiple overloaded constructors may hurt code clarity and maintainability.
- **Solution**
 - Class: `Meal`
 - Builder class: `Meal.Builder`
 - Required fields in constructor, optional fields via fluent methods ... chained method calls
- **Why Builder?**
 - Flexible combinations without overloaded constructors
 - Readable, fluent API and immutable final object

Custom Meal app - Code Example

Benefits

Easy to create objects with many options
Readable fluent API
Makes objects immutable once built

```
// Meal with required burger and one optional field (drink)
```

```
Meal meal1 = new Meal.Builder(Burger.VEG)  
            .drink(Drink.COKE)  
            .build();
```

Example of fluent API
Chained method calls

```
System.out.println("Meal 1: " + meal1);
```

```
// Meal with required burger and two optional fields (drink, fries)
```

```
Meal meal2 = new Meal.Builder(Burger.CHEESE_BURGER)  
            .drink(Drink.LEMONADE)  
            .fries(Fries.FRENCH_FRIES)  
            .build();
```

Example of fluent API
Chained method calls

```
System.out.println("Meal 2: " + meal2);
```


Builder Example: Java JDK Locale class

- **Use Case**
 - The Java Locale API lets you define regional settings such as language and country
 - The `Builder` allows you to configure a `Locale` step-by-step before creating it
- **Solution**
 - Class: `java.util.Locale`
 - Builder class: `Locale.Builder`
 - Set the language, region, and other attributes ... then call `build()` to create the final `Locale` object
- **Why Builder?**
 - Builds a `Locale` in multiple steps without requiring all information upfront
 - Keeps construction separate from the final object

Java Locale class - Code Example

Benefits

Easy to create objects with many options
Readable fluent API
Makes objects immutable once built

// Create a Locale for French language in Canada

```
Locale locale = new Locale.Builder()  
                .setLanguage("fr")  
                .setRegion("CA")  
                .build();
```

Example of fluent API
Chained method calls

Builder Example: OpenAI Java SDK

- **Use Case**
 - The OpenAI Java SDK lets you build and send requests to AI models
 - The `Builder` allows you to set parameters step-by-step before sending
- **Solution**
 - Class: `com.openai.models.chat.completions.ChatCompletionCreateParams`
 - Builder class: `ChatCompletionCreateParams.Builder`
 - Specify model, max tokens, messages, then call `build()`
- **Why Builder?**
 - Configures a complex request in stages
 - Separates setup logic from the API call

Builder Example: OpenAI Java SDK

Benefits

Easy to create objects with many options
Readable fluent API
Makes objects immutable once built

// Build a chat completion request to generate a song about Java Duke

```
ChatCompletionCreateParams createParams = ChatCompletionCreateParams.builder()  
    .model(<< insert latest model name >>)  
    .maxCompletionTokens(2048)  
    .addSystemMessage("You are an award-winning songwriter. Hit maker.")  
    .addUserMessage("Write me a song about Java Duke and his adventures.")  
    .build();
```

Example of fluent api
chained method calls

Builder Design Pattern - Meal Example



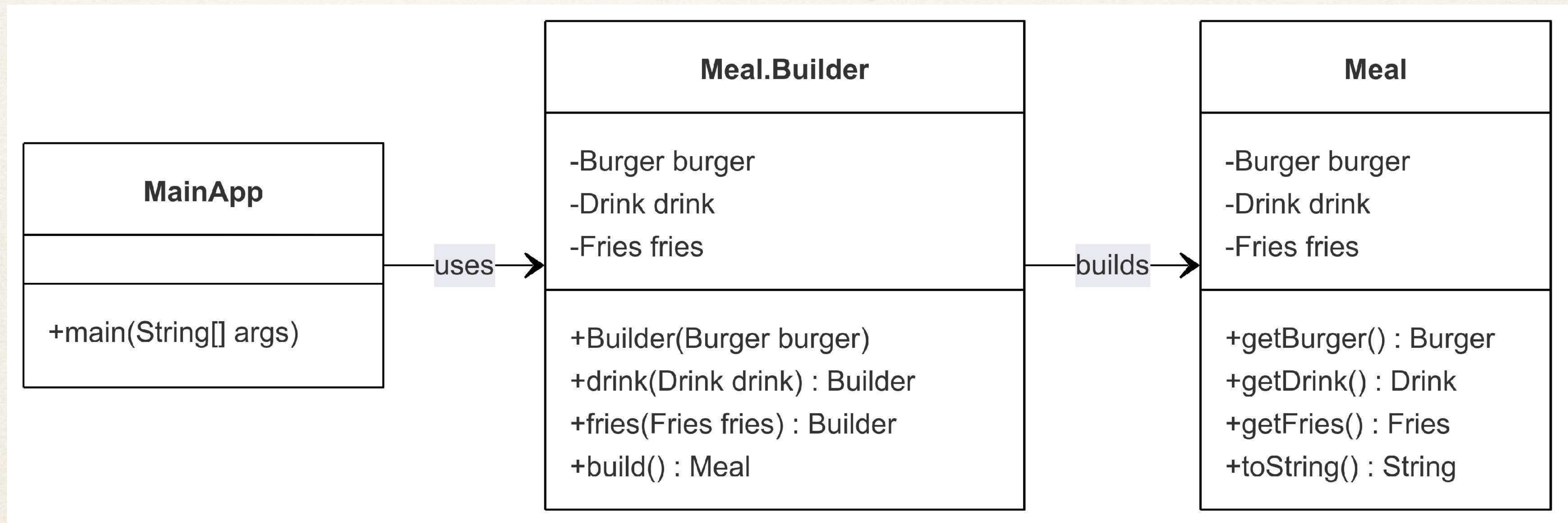
Custom Meal app - Code Example

Bob's

```
// Meal with required burger and one optional field (drink)  
  
Meal meal1 = new Meal.Builder(Burger.VEG)  
    .drink(Drink.COKE)  
    .build();  
  
System.out.println("Meal 1: " + meal1);
```

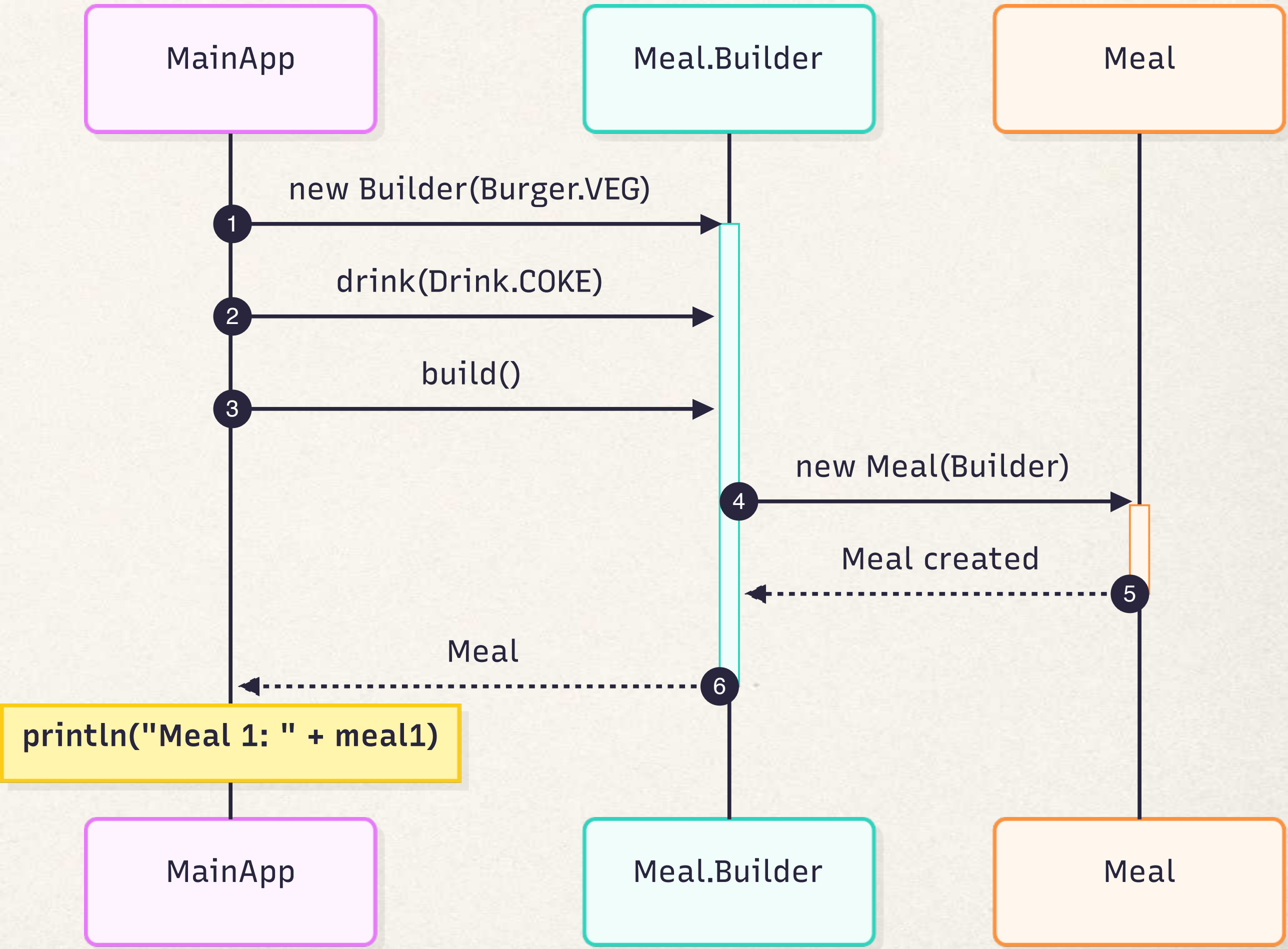

UML Diagram - Key Classes

Builder is a static inner class of Meal



Sequence Diagram

```
// Meal with required burger and one optional field (drink)
Meal meal1 = new Meal.Builder(Burger.VEG)
    .drink(Drink.COKE)
    .build();
System.out.println("Meal 1: " + meal1);
```



Development Process

Step-By-Step

1. Create enums: `Burger`, `Drink` and `Fries`
2. Define fields for `Meal` class
3. Add a static inner class `Meal.Builder`
4. In `Builder`, add a constructor for required fields: `Builder(Burger burger)`
5. Add fluent methods for optional fields (`drink()`, `fries()`), returning `this`
6. Make the `Meal` constructor private, taking a `Builder`
7. Implement `build()` to return new `Meal(this)`

Prototype Design Pattern



Prototype Design Pattern

Intent

Problem

Solution

- **Intent**

- Create new objects by copying existing objects
- Avoid using the `new` keyword for similar objects

- **Problem**

- Need to create many objects with similar state
- Using the `new` keyword each time is costly

- **Solution**

- Use a Prototype object that can clone itself
- Call `clone()` to create a new object with the same state

Benefits

Quickly create new objects by cloning existing ones
Reduces repetitive initialization and use of `new` keyword
Faster creation of complex objects

Examples

- Video Game Characters
- Data Analytics Reports
- Email Newsletter App

Prototype Example: Video Game Characters

- **Use Case**
 - A game has multiple characters that share the same base stats and abilities
 - Creating each character with new keyword requires reinitializing same fields repeatedly
- **Solution**
 - Class: `GameCharacter`
 - Prototype: Create a base character with default stats
 - Clone & Customize: Use `clone ( )` to create new characters and only modify unique attributes
- **Why Prototype?**
 - Faster creation of multiple similar characters
 - Avoids repetitive initialization logic

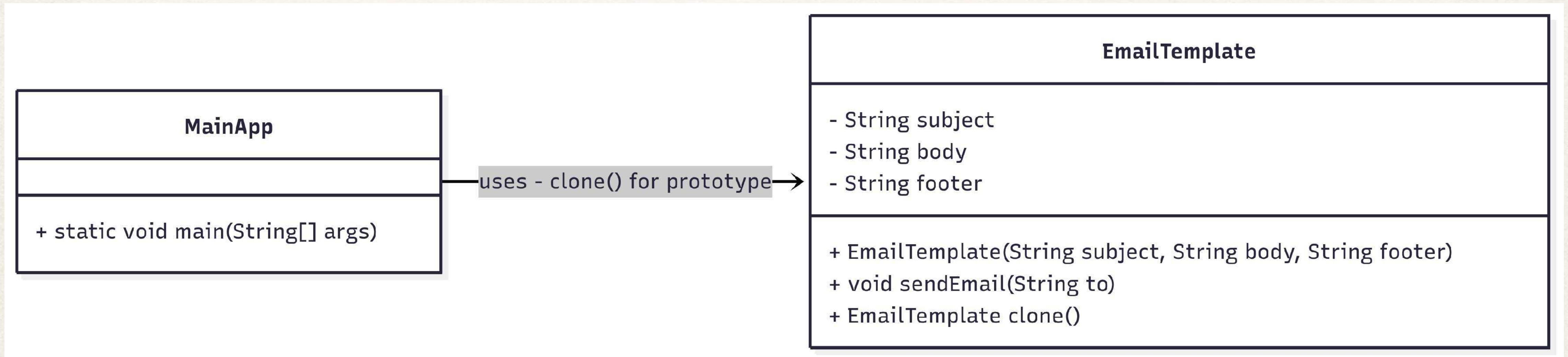
Prototype Example: Data Analytics Reports

- **Use Case**
 - A data analytics system frequently creates similar report objects containing charts, tables and styles
 - Reinitializing all report fields manually is inefficient
- **Solution**
 - Class: `ReportTemplate`
 - Prototype: Define a base report layout with default styles
 - Clone & Customize: Use `clone ( )` to generate new reports and modify only the data source
- **Why Prototype?**
 - Quickly create reports with uniform formatting
 - Reduces boilerplate setup code

Prototype Example: Email Newsletter App

- **Use Case**
 - A company sends monthly email newsletters with a standard template
 - Manually creating each email object with new keyword is repetitive and error-prone
- **Solution**
 - Class: `EmailTemplate`
 - Prototype: Creates a base email template
 - Clone & Customize: Use `clone ( )` to copy the base template and modify the subject
- **Why Prototype?**
 - Avoid repeatedly creating email object using new keyword
 - Quickly creates customized emails from a single base template

UML Diagram



Development Process

Step-By-Step

1. Create EmailTemplate class
2. Implement the `java.lang.Cloneable` interface
3. Override the `clone()` method
4. In the MainApp ... clone and customize the prototype

Structural Design Patterns



Java Design Patterns

Creational

Factory
Singleton
Prototype

...



Structural

Adapter
Bridge
Composite

...

Behavioral

Iterator
Observer
Strategy

...

Structural Patterns

Structural

Adapter
Bridge
Composite

...

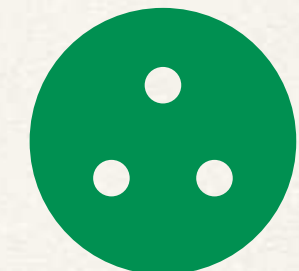
- Focus on **how objects are composed and related**
- Facilitates building larger structures from objects
- Makes code more flexible and easier to change
- Add functionality without major modifications

Structural Patterns - Analogies

Structural

Adapter
Bridge
Composite
...

- Power Adapter Plug
 - Allow US laptop to plug into outlet in another country (Europe, India, etc ...)
- Television, Home Speakers and Remote Control
 - Different components, but they can work together as a unit
 - Can swap components



Structural patterns adds functionality without modifications

Adapter Design Pattern

Intent

Problem

Solution

- **Intent**

- Provide a way for incompatible interfaces to work together using a common interface
- Allow clients to interact with different APIs without knowing the differences
- Adapter convert requests from the client into a format the adaptee understands

- **Problem**

- Payment providers (Stripe, PayPal) have different APIs and data formats
- Without adapters, client must handle provider specific logic, resulting in tight-coupling

- **Solution**

- Define a common interface, PaymentProcessor used by the client
- Use adapters (StripeAdapter, PayPalAdapter) to translate requests and delegate to each SDK

Benefits

Unifies different API with common interface
Decouples client code from third-party systems
Easy to add new providers with an adapter

Examples

- Sports League Stats Dashboard
- Legacy Health Records Integration
- Unified Payment Gateway

Sports League Stats Dashboard

Benefits

Unifies different API with common interface
Decouples client code from third-party systems
Easy to add new providers with an adapter

- **Use Case**

- A sports analytics company wants a single dashboard showing status for: cricket, rugby and football
- However, each league provides data in different formats

- **Solution**

- Interface: `LeagueStatsProvider: getStats(teamId)`
- Adapters: `CricketStatsAdapter`, `RugbyStatsAdapter`, `FootballStatsAdapter`
- Each adapter translates raw data (Runs, Tries, Goals) into a unified data model

- **Why Adapter?**

- Unifies incompatible league APIs into one consistent format
- If we need to add a new league, just add a new adapter

Legacy Health Records Integration

Benefits

Unifies different API with common interface
Decouples client code from third-party systems
Easy to add new providers with an adapter

- **Use Case**

- Hospitals need to share patient data with regional health systems
- Regional systems use XML/SOAP while the modern hospital app uses JSON/REST

- **Solution**

- Interface: `HealthRecordService`: `getPatientRecord(patientId)`
- Adapter: `LegacyHealthAdapter`
- The adapter converts JSON $\longleftrightarrow$ XML for patient demographics, vitals and lab results

- **Why Adapter?**

- Keeps the app decoupled from legacy health records systems
- If we need to add a new legacy health system, just add a new adapter

Unified Payment Gateway

Benefits

Unifies different API with common interface
Decouples client code from third-party systems
Easy to add new providers with an adapter

- **Use Case**

- An e-commerce platform supports multiple payment providers: Stripe, PayPal, etc ...
- Each provider has different APIs and incompatible data formats

- **Solution**

- Interface: `PaymentProcessor: pay(amount)`
- Adapter: `StripeAdapter, PayPalAdapter`
- The adapter converts method calls to support the underlying SDK: `StripeSDK` and `PayPalSDK`

- **Why Adapter?**

- Provides a single unified interface for payments
- If we need to add a new payment provider, just add a new adapter

Adapter Design Pattern

Unified Payment Gateway Example



Unified Payment Gateway

Benefits

Unifies different API with common interface
Decouples client code from third-party systems
Easy to add new providers with an adapter

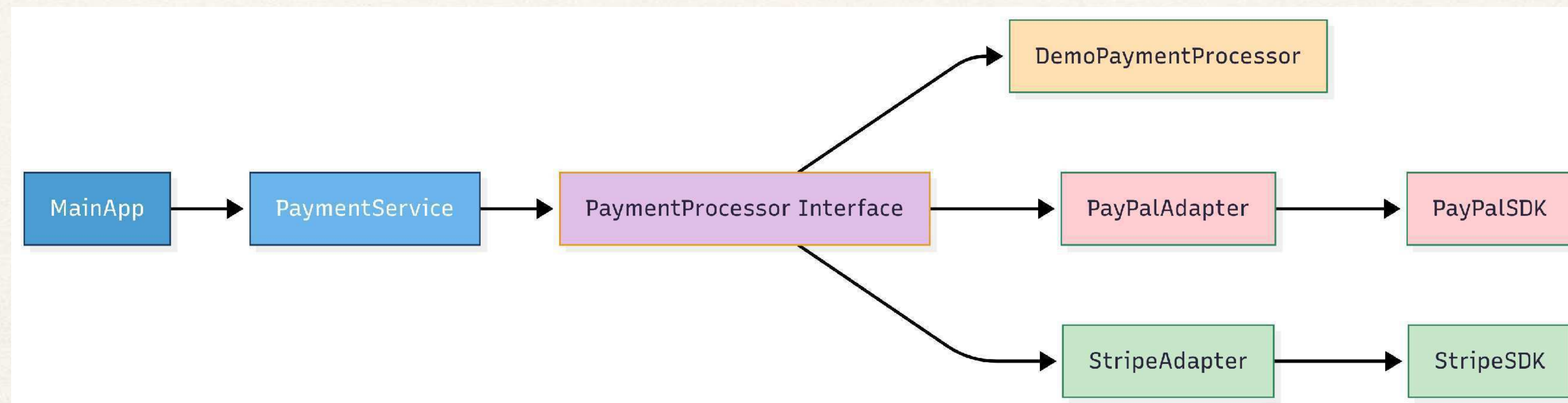
- **Use Case**
 - An e-commerce platform supports multiple payment providers: PayPal, Stripe, etc ...
 - Each provider has different APIs and incompatible data formats

Three Payment Methods

Name	Description
Demo Payment	In-house demo payment, no adapter required
PayPal	Requires an adapter because PayPal SDK has different method signature
Stripe	Requires an adapter because Stripe SDK has different method signature

Unified Payment Gateway

- **Solution**
 - Interface: `PaymentProcessor: pay(amount)`
 - Adapter: `PayPalAdapter, StripeAdapter`
 - The adapter converts method calls to support the underlying SDK: `PayPalSDK` and `StripeSDK`

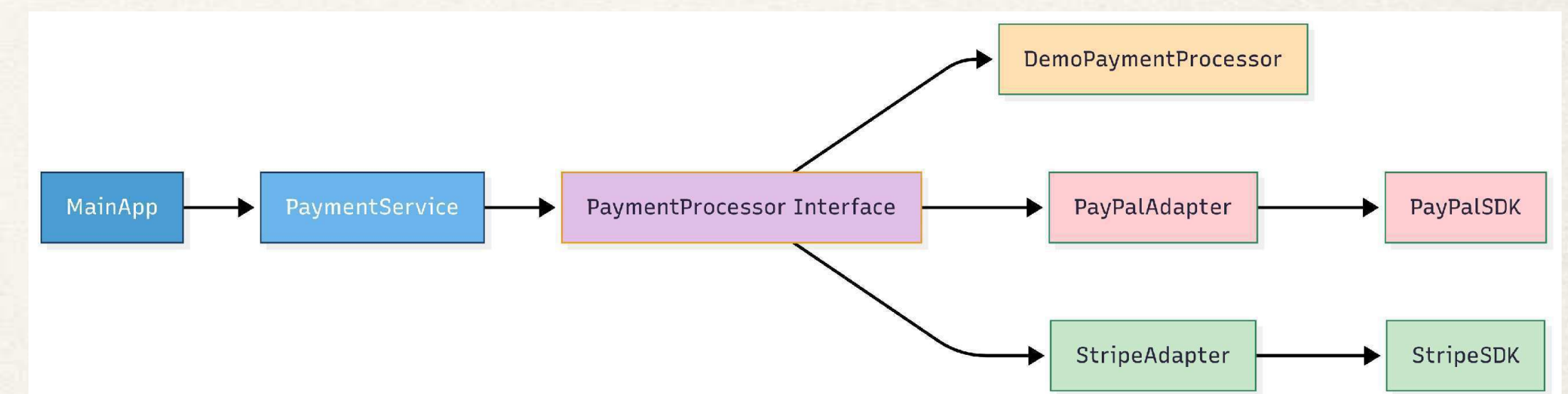


PaymentProcessor Interface

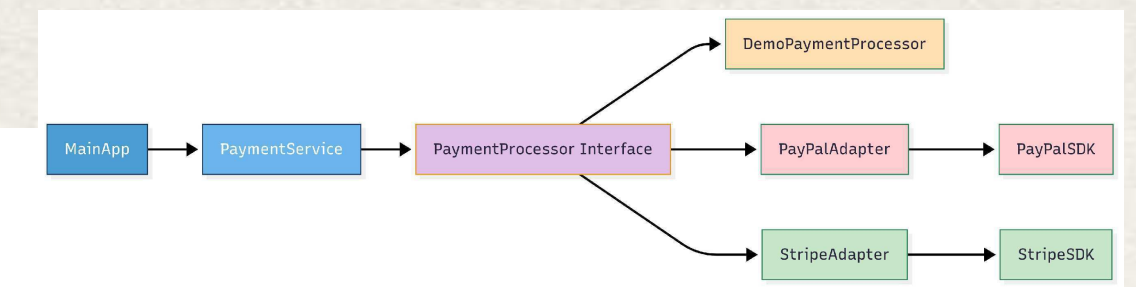
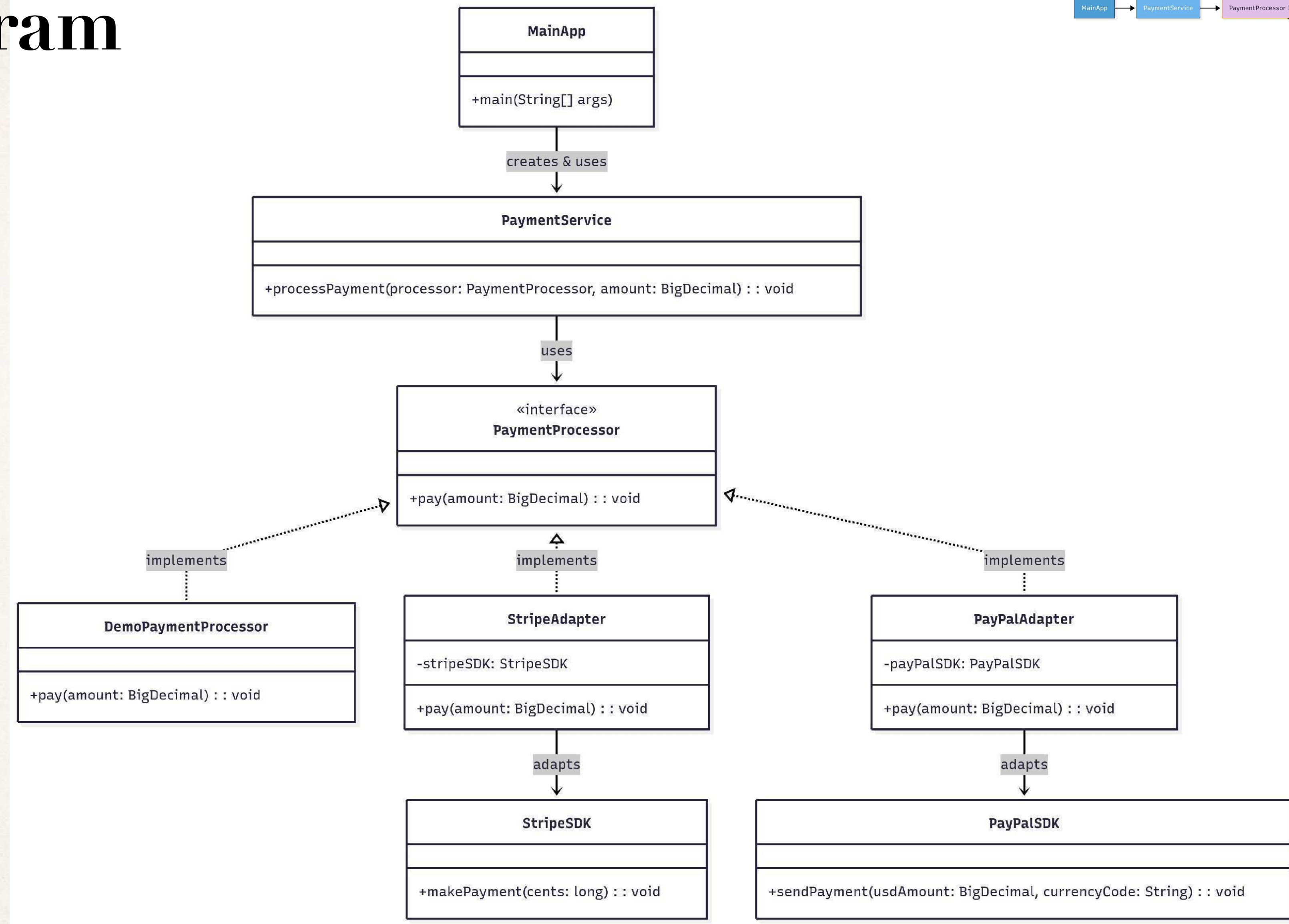
```
/**
 * Target Interface (PaymentProcessor)
 *
 * Defines the standard interface expected by the client (PaymentService)
 *
 * All adapters will implement this interface to make compatible APIs work
 */
public interface PaymentProcessor {

    void pay(BigDecimal amount);

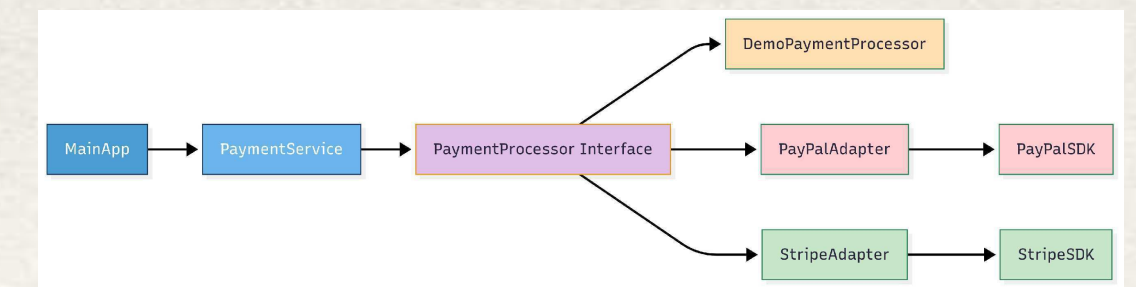
}
```



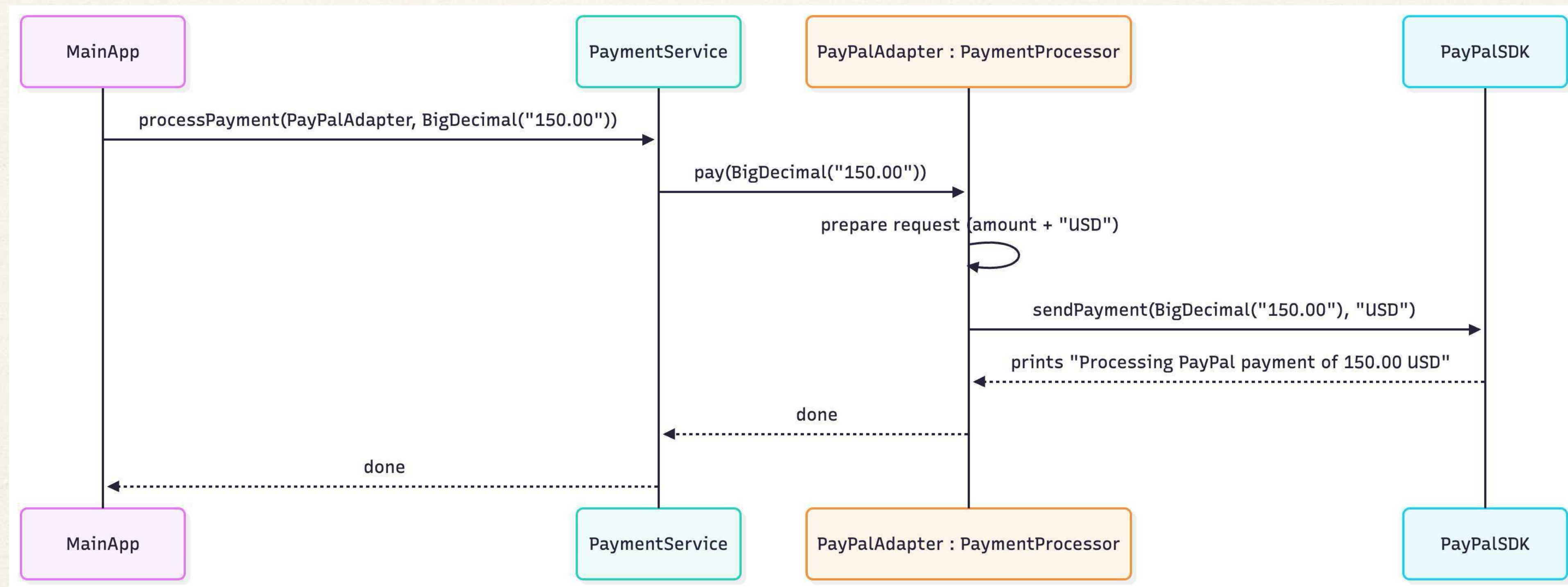
UML Diagram



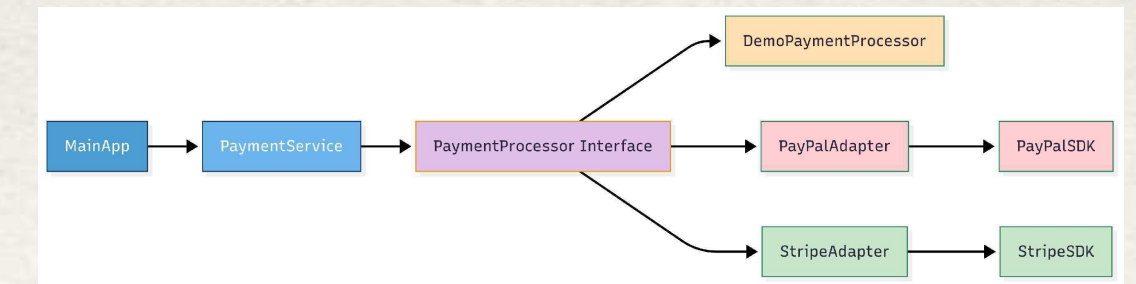
Sequence Diagram - PayPal



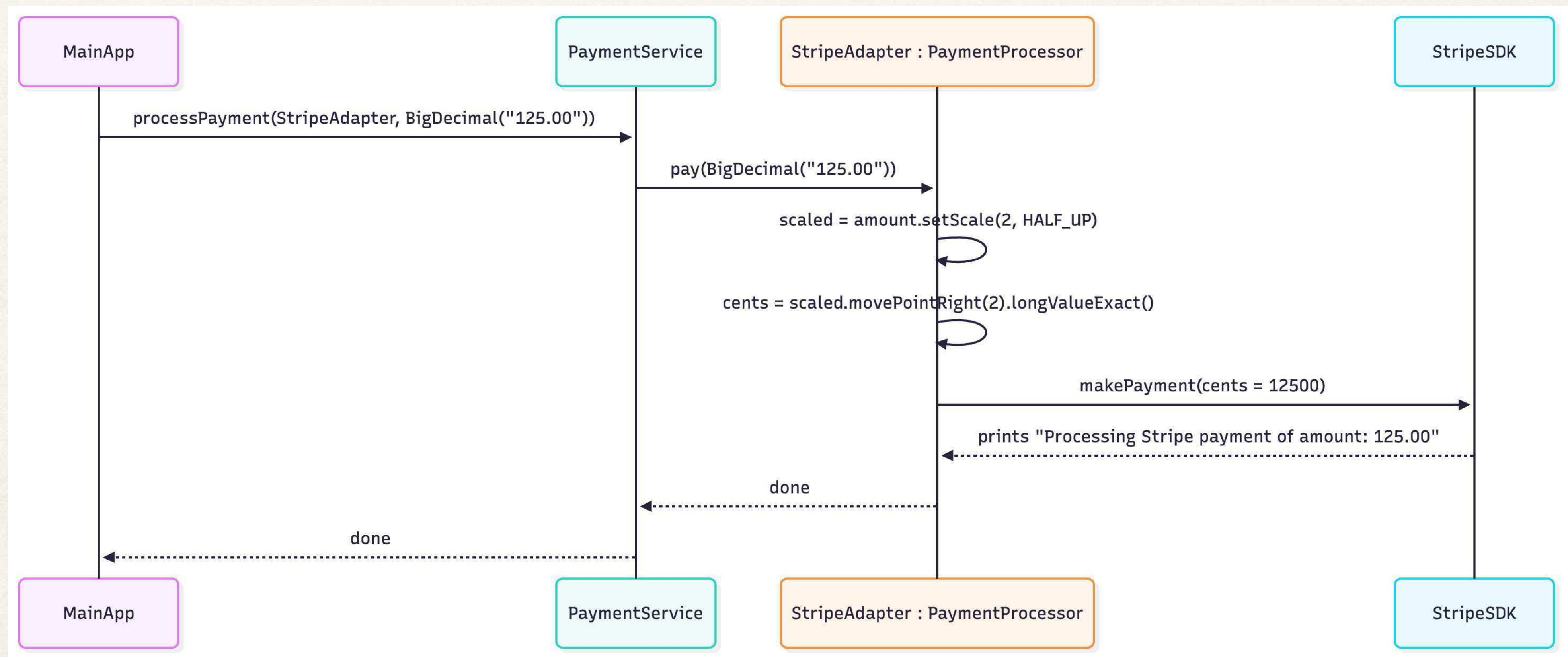
- PayPal SDK requires amount and currency code (this differs from our common interface)
- The adapter converts method calls to support the underlying SDK: PayPalSDK



Sequence Diagram - Stripe



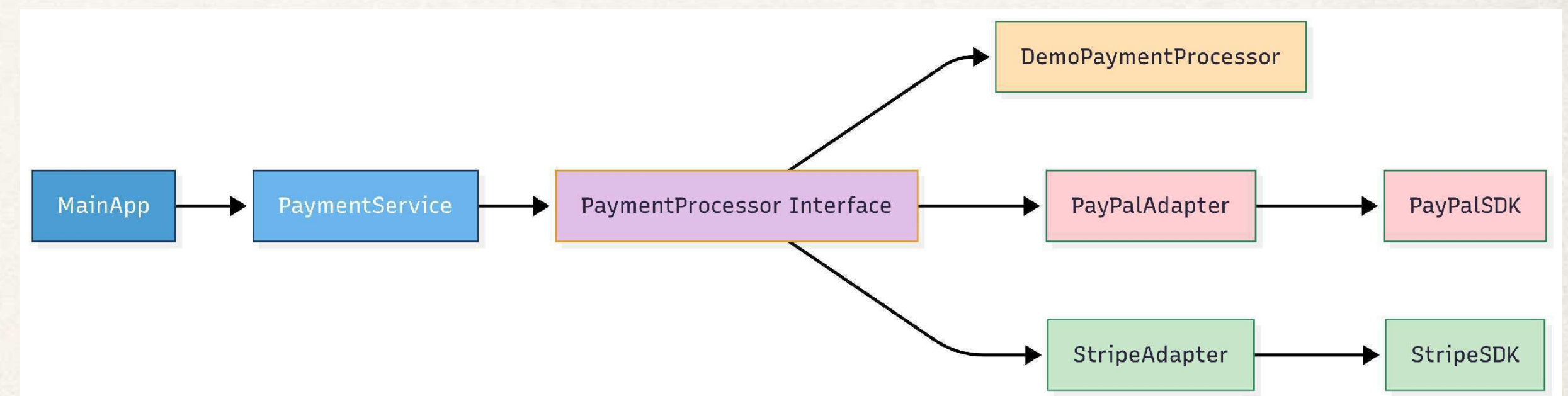
- Stripe SDK requires **amount in total cents** instead of dollars+cents (this differs from our common interface)
- The adapter converts method calls to support the underlying SDK: StripeSDK



Development Process

Step-By-Step

1. Create PaymentProcessor (common interface)
2. Create DemoPaymentProcessor (direct implementation)
3. Create PayPalAdapter (translate BigDecimal + currency → PayPalSDK)
4. Create StripeAdapter (translate BigDecimal → cents → StripeSDK)



Bridge Design Pattern



Bridge Design Pattern

Intent

Problem

Solution

- **Intent**
 - Decouple abstraction (high-level logic) from implementation (low-level details)
 - Allows both abstraction and implementation to vary independently
 - Avoid class explosion when combining multiple dimensions (e.g. Payment methods x Currencies)

Bridge Design Pattern

Intent

Problem

Solution

- Problem
 - A payment system must support different payment methods
 - Must also support different currencies (USD, EUR, INR, ...)
 - Without Bridge ... every combination requires a new class:
 - CreditCardUsdPayment
 - CreditCardEurPayment
 - NetBankingUsdPayment, ...
 - This leads to tight coupling and poor scalability

Bridge Design Pattern

Intent

Problem

Solution

- **Solution**

- Define an **Abstraction** hierarchy
- Define an **Implementation** hierarchy
- Link them via a bridge (composition instead of inheritance)
- When we need new payment methods and currencies
 - Can be added independently, without changing existing code

Benefits

Extensible: Add new types independently
Decoupled: Separate logic from details
Scalable: Avoids class explosion

Examples

- Employees and Compensation Plans
- Vehicles and Service Centers
- Payments and Currencies

Employees and Compensation Plans

Benefits

Extensible: Add new types independently
Decoupled: Separate logic from details
Scalable: Avoids class explosion

- **Use Case**

- Human Resources system must handle different types of employees (full-time, part-time, contractor, ...)
- Each employee has a different compensation plan (Salaried, Hourly, ...)
- Creating separate classes for each combination leads to class explosion, duplication and complexity

- **Solution**

- Abstraction: `Employee: FullTimeEmployee, PartTimeEmployee, Contractor`
- Implementation: `CompensationPlan: SalaryPlan, HourlyPlan, ...`
- Bridge: Employee delegates pay calculation to compensation plans

- **Why Bridge?**

- Employees and compensation plans evolve independently
- Add new employee types or plans without major modifications to code

Vehicles and Service Centers

Benefits

Extensible: Add new types independently
Decoupled: Separate logic from details
Scalable: Avoids class explosion

- **Use Case**

- A service company repairs different types of vehicles (cars, trucks, ...)
- Each vehicle can be serviced at different service centers (repair, inspection, painting, ...)

- **Solution**

- Abstraction: `Vehicle`: `Car`, `Truck`
- Implementation: `ServiceCenter`: `RepairCenter`, `InspectionCenter`, `PaintCenter`
- Bridge: Vehicle delegates servicing to a service center dynamically

- **Why Bridge?**

- Vehicles and service centers evolve independently
- Supports flexible combination of vehicles and services

Payments and Currencies

Benefits

Extensible: Add new types independently
Decoupled: Separate logic from details
Scalable: Avoids class explosion

- **Use Case**
 - An e-commerce platform supports multiple payment methods and currencies
- **Solution**
 - Abstraction: `Payment: CreditCardPayment, NetBankingPayment`
 - Implementation: `CurrencyProcessor: UsdCurrencyProcessor, EurCurrencyProcessor`
 - Bridge: Each payment delegates to a currency processor at runtime
- **Why Bridge?**
 - Decouples payment methods from currency processing
 - Add new payment methods or currencies independently

Bridge Design Pattern

Payments and Currencies Example



Payments and Currencies

Benefits

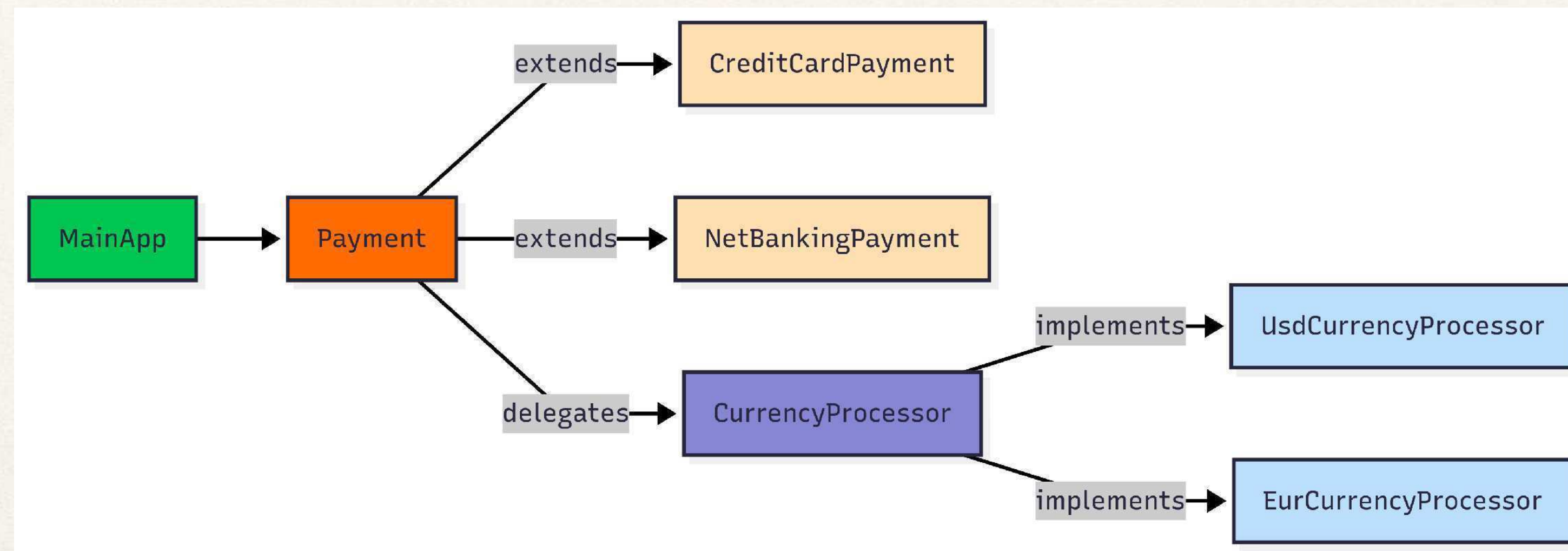
Extensible: Add new types independently
Decoupled: Separate logic from details
Scalable: Avoids class explosion

- **Use Case**
 - An e-commerce platform supports multiple payment methods and currencies

Payments and Currencies

- **Solution**

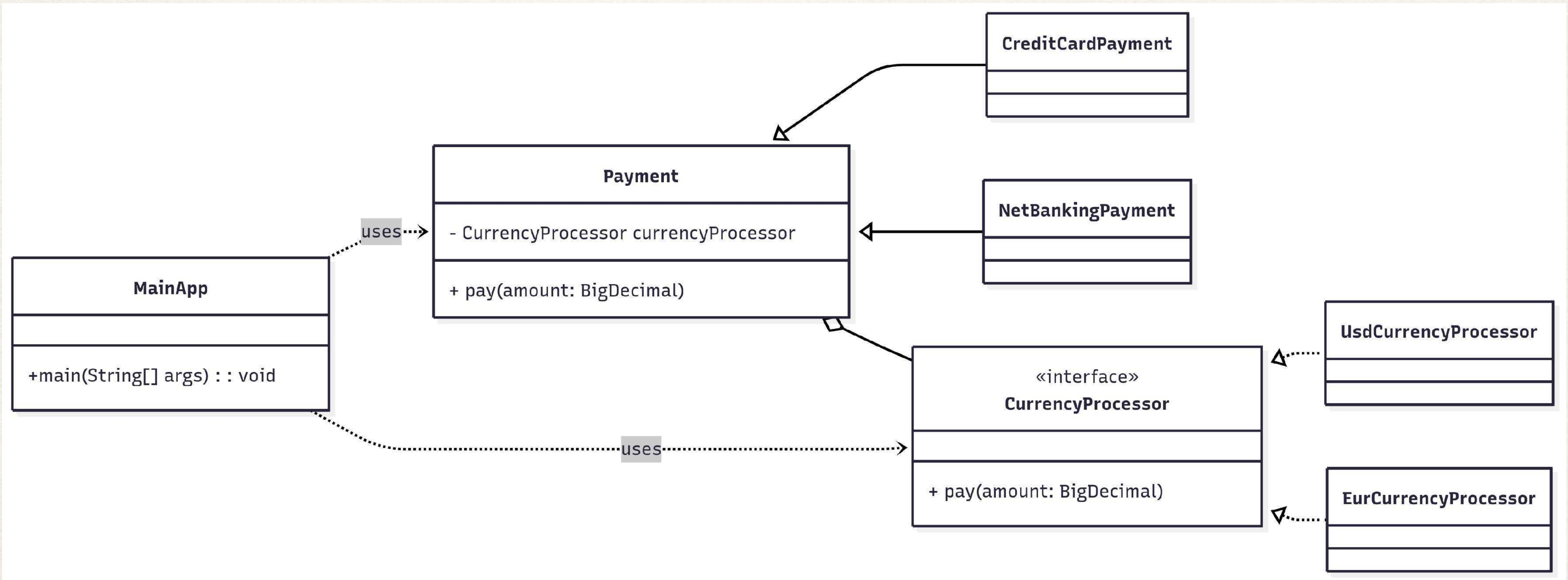
- Abstraction: `Payment`: `CreditCardPayment`, `NetBankingPayment`
- Implementation: `CurrencyProcessor`: `UsdCurrencyProcessor`, `EurCurrencyProcessor`
- Bridge: Each payment delegates to a currency processor at runtime



The Components

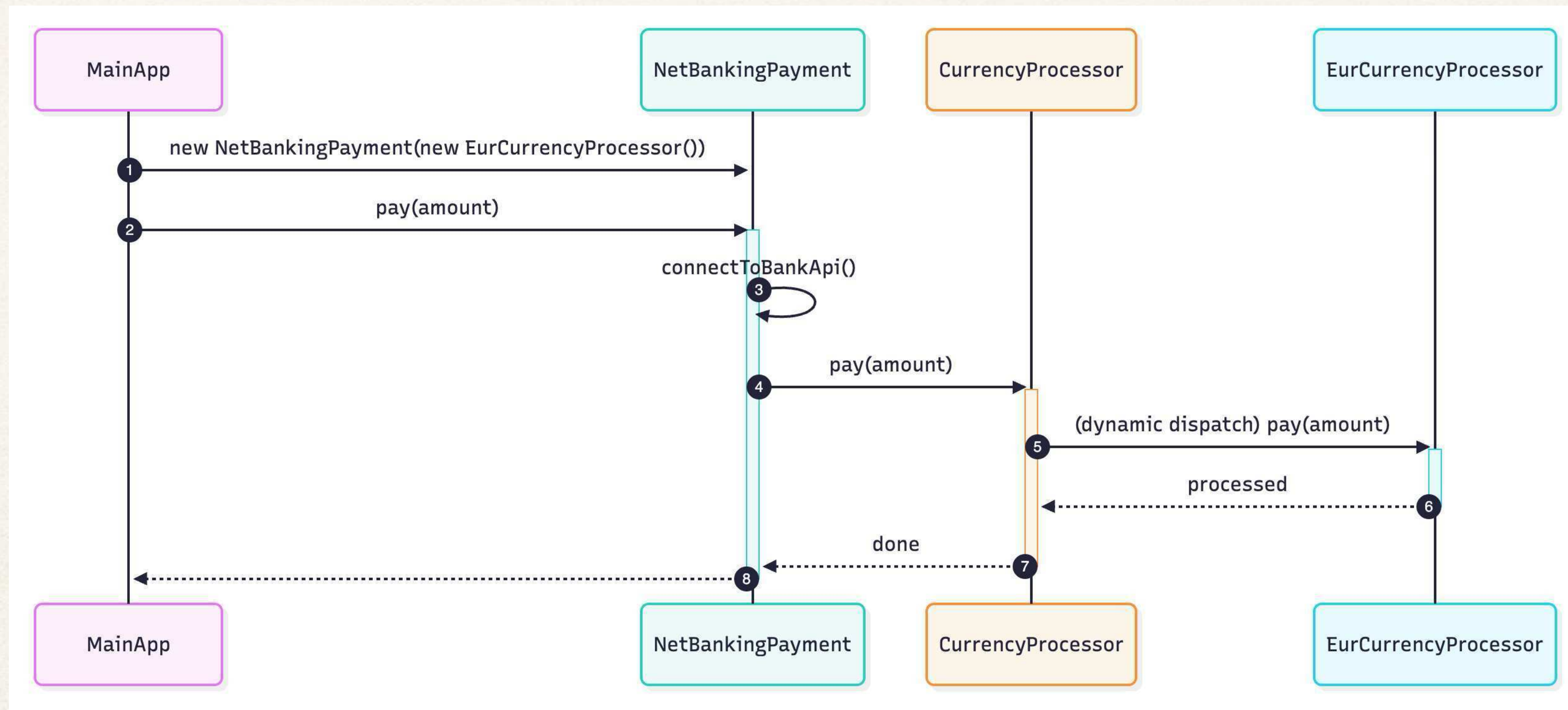
Component	Role	Description
Payment (interface)	Abstraction	Defines the high-level interface and maintains a reference to the implementor.
CreditCardPayment, NetBankingPayment	Refined Abstraction	Extend or implement the abstraction to provide specific behavior while still delegating to the implementor.
CurrencyProcessor	Implementor	Declares the interface for the implementation side of the bridge.
UsdCurrencyProcessor, EurCurrencyProcessor	Concrete Implementors	Provide the actual implementation details for processing in different contexts.

UML Diagram



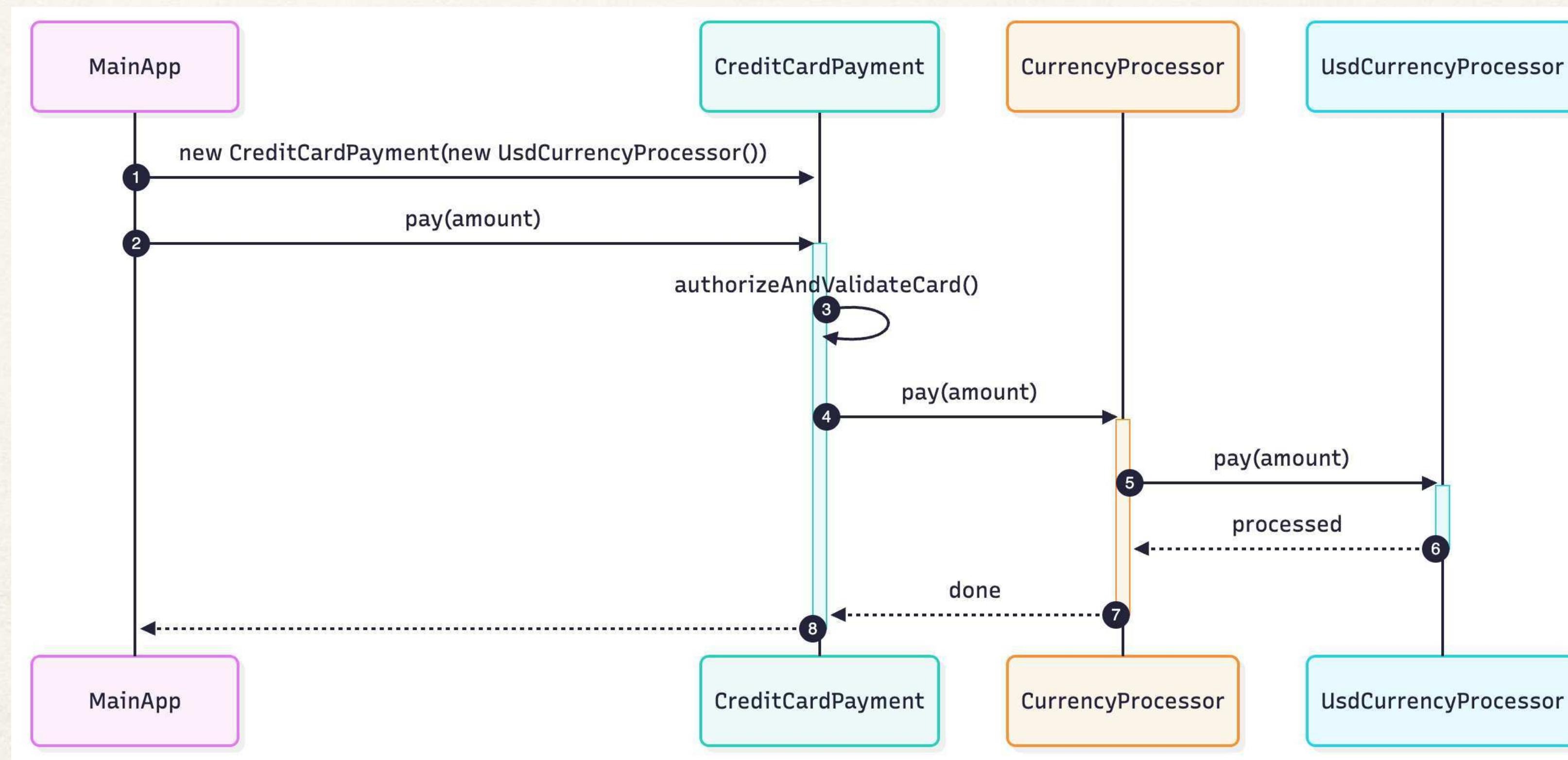
Sequence Diagram - NetBanking and EUR

- Initiate a call to NetBankingPayment, which delegate currency processing to EurCurrencyProcessor
- Demonstrates the Bridge pattern: payment logic and currency processing are decoupled



Sequence Diagram - CreditCard and USD

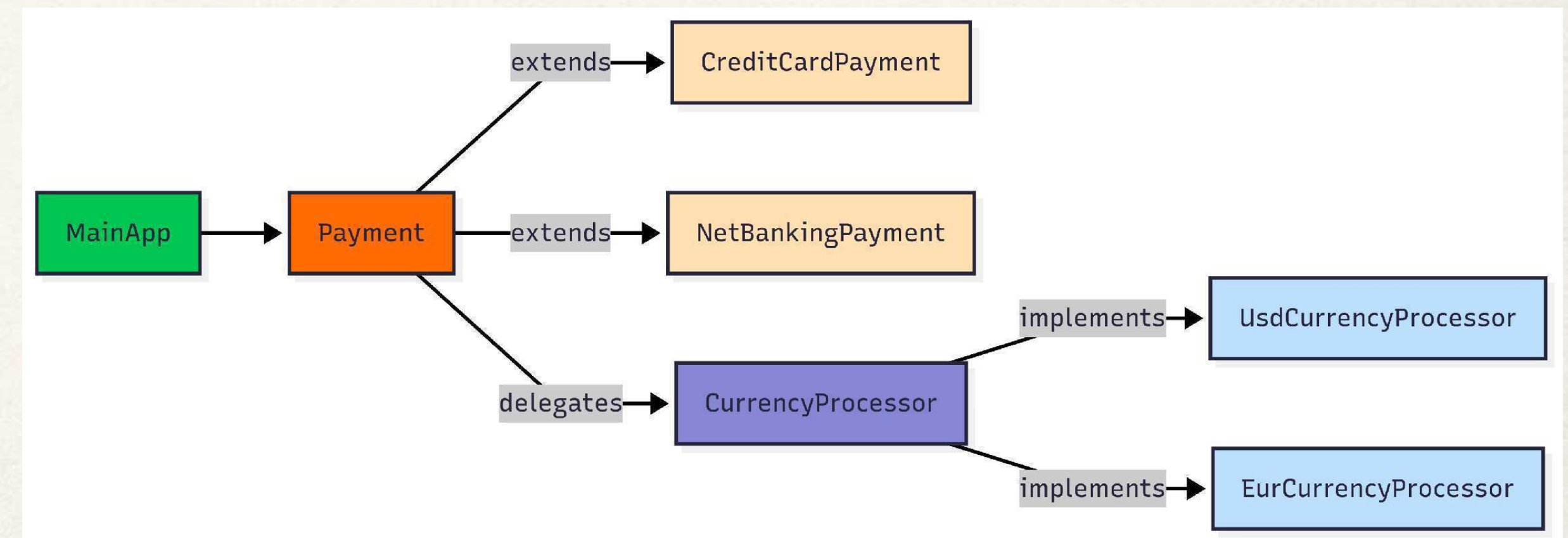
- Initiate a call to CreditCardPayment, which delegates currency processing to UsdCurrencyProcessor
- Demonstrates the Bridge pattern: payment logic and currency processing are decoupled



Development Process

Step-By-Step

1. Create CurrencyProcessor (implementor)
2. Create EurCurrencyProcessor (concrete implementor)
3. Create Payment (abstraction, high-level interface)
4. Create NetBankingPayment (refined abstraction)
5. Repeat for others ...



Decorator Design Pattern



Decorator Design Pattern

Intent

Problem

Solution

- **Intent**

- Add responsibilities to objects dynamically
- Flexible alternative to subclassing
- Combine behaviors at runtime

- **Problem**

- Need to extend message notifications with features (priority, signature etc)
- Using inheritance cause class explosion
- Rigid, tightly coupled design that is hard to maintain

- **Solution**

- Use an abstract decorator that wraps another component
- Add concrete decorators to provide behavior

Benefits

Extensible: Add new features easily
Decoupled: Mix behaviors at runtime
Scalable: Avoids class explosion

Examples

- Coffee Shop
- Data I/O
- Notification

Coffee Shop

Benefits

Extensible: Add new features easily
Decoupled: Mix behaviors at runtime
Scalable: Avoids class explosion

- **Use Case**

- Customers can add extras to their coffee (milk, sugar, whipped cream, caramel, ...)
- Creating subclasses for each combination (MilkCoffee, SugarMilkCoffee, CaramelWhippedCoffee, ...) leads to class explosion

- **Solution**

- Component: `Coffee` interface.
- Concrete Component: `SimpleCoffee`.
- Decorators: `MilkDecorator`, `SugarDecorator`, `WhippedCreamDecorator`, etc
- Each decorator adds cost/description and delegates to the wrapped object

- **Why Decorator?**

- Combine features dynamically at runtime
- Avoid subclass explosion and keep design flexible

Data I/O

Benefits

Extensible: Add new features easily
Decoupled: Mix behaviors at runtime
Scalable: Avoids class explosion

- **Use Case**
 - An application saves and loads data to files
 - Data may require extra features like compression or encryption
- **Solution**
 - Component: `DataSource` interface
 - Concrete Component: `FileDataSource`
 - Decorators: `CompressionDecorator`, `EncryptionDecorator`
 - Each decorator adds functionality while delegating to the wrapped object
- **Why Decorator?**
 - Add new data-processing features without modifying existing code
 - Flexibly stack behaviors (encrypt + compress, compress + encrypt)

Notification

Benefits

Extensible: Add new features easily
Decoupled: Mix behaviors at runtime
Scalable: Avoids class explosion

- **Use Case**

- A system sends notifications (Email, SMS, WhatsApp).
- Extra features may be needed, like marking messages as high priority, adding a signature, or emphasizing text.

- **Solution**

- Component: `Notification` interface.
- Concrete Components: `EmailNotification`, `SmsNotification`, `WhatsAppNotification`.
- Decorators: `PriorityDecorator`, `SignatureDecorator`, `UpperCaseDecorator`.
- Each decorator modifies the message and then delegates to the wrapped object.

- **Why Decorator?**

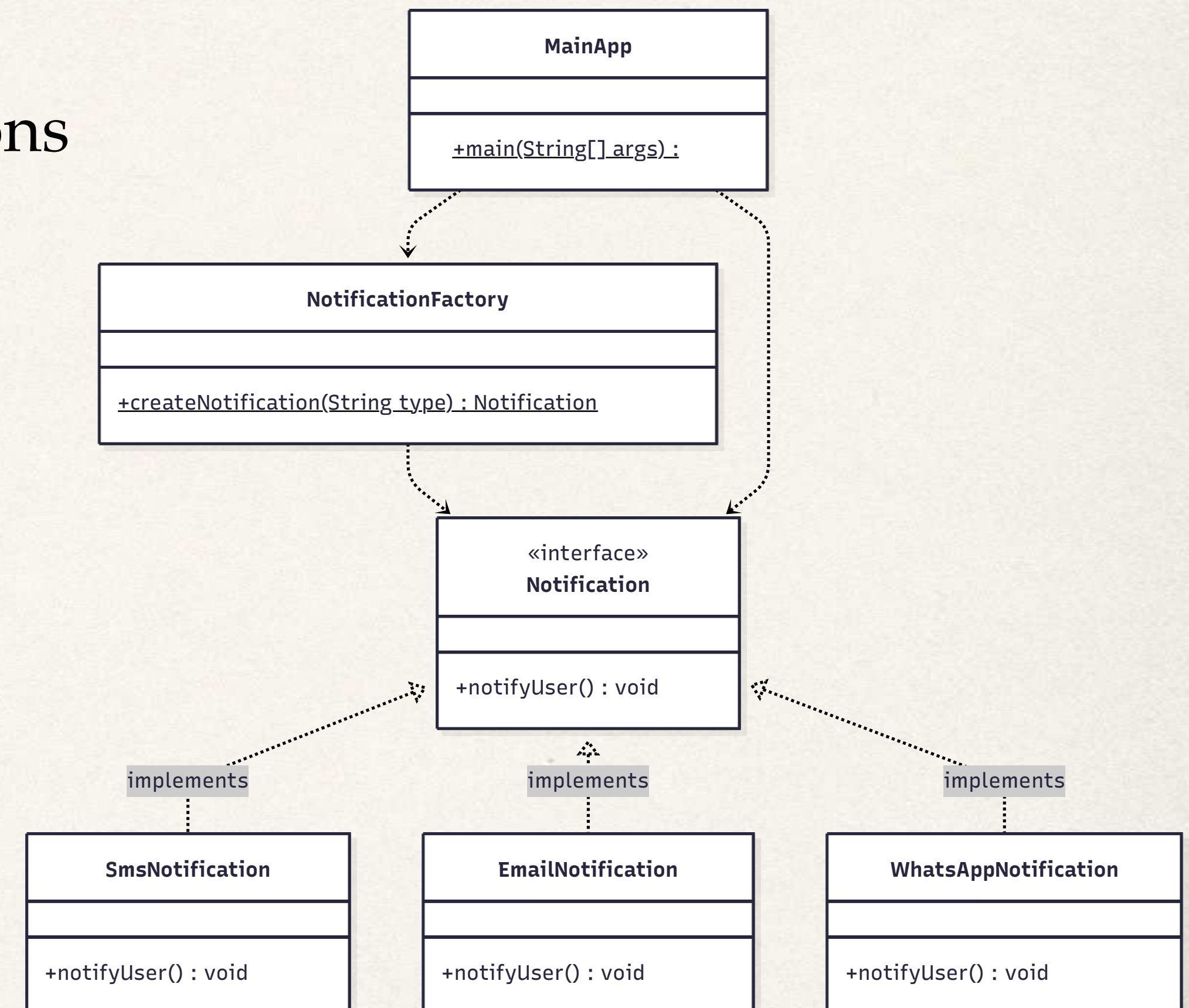
- Add new notification features without changing base classes.
- Combine behaviors dynamically at runtime in different orders.

Decorator Design Pattern Notification Example



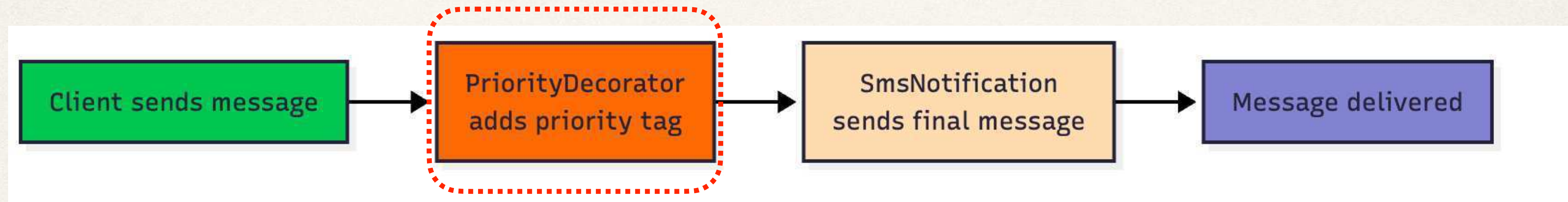
Notifications

- Use Case
 - An app needs to send different types of notifications
 - Email
 - SMS
 - WhatsApp ...



Notifications - Decorators

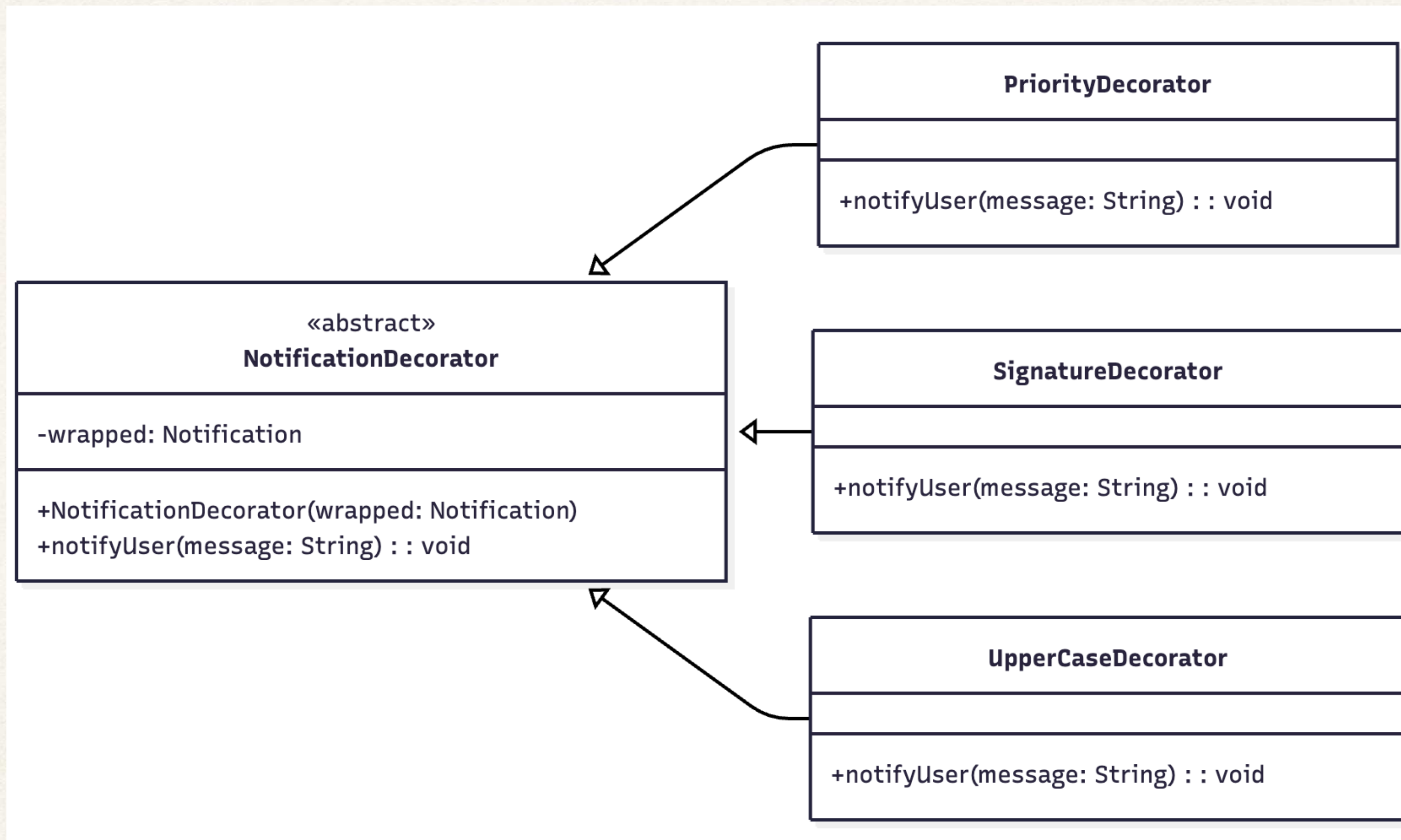
- **Solution**
 - Decorators: PriorityDecorator, SignatureDecorator, UpperCaseDecorator
 - Each decorator modifies the message and then delegates to the notification object



The Components

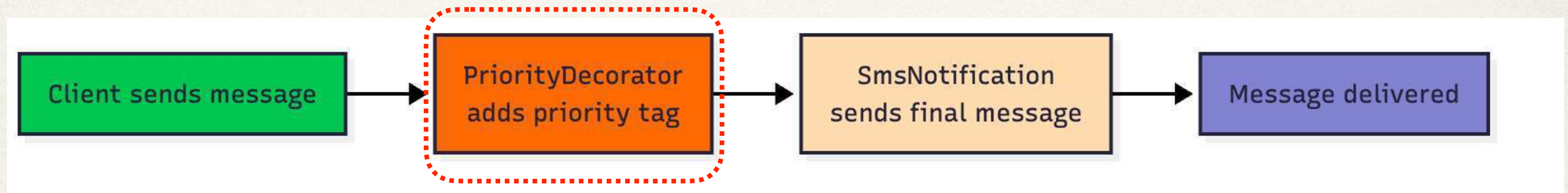
Component	Role	Description
NotificationDecorator	Abstract Decorator	Wraps a Notification and delegates calls Base class for all decorators
PriorityDecorator	Concrete Decorator	Marks a message as high priority by prepending a priority tag
SignatureDecorator	Concrete Decorator	Adds a signature or footer to the end of the message
UpperCaseDecorator	Concrete Decorator	Converts the message text to uppercase for emphasis

UML Diagram

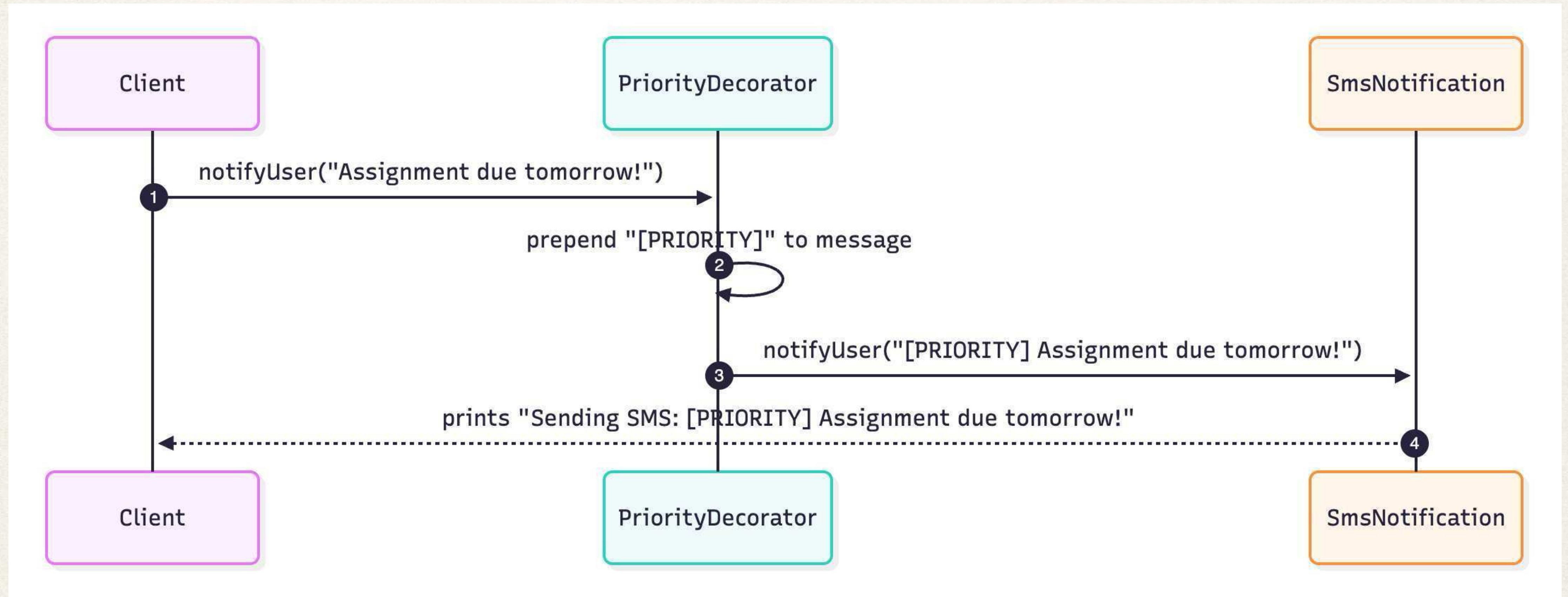


Component Diagram - PriorityDecorator

- Initiate a call to `PriorityDecorator`, which adds a priority tag
- Delegates the message to a notification object
- Demonstrates the Decorator pattern
 - New behavior is layered dynamically without changing the base notification class

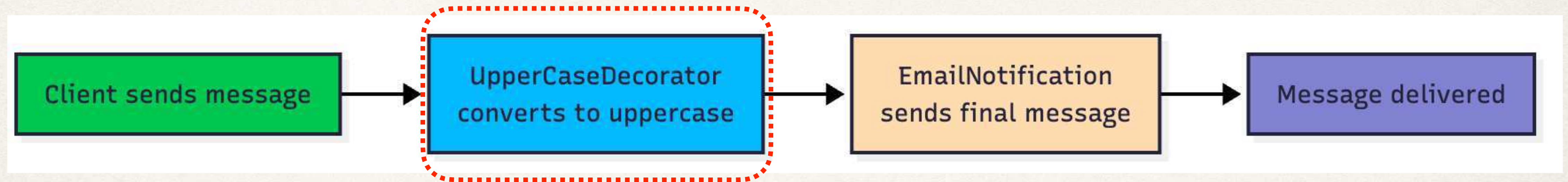


Sequence Diagram - PriorityDecorator

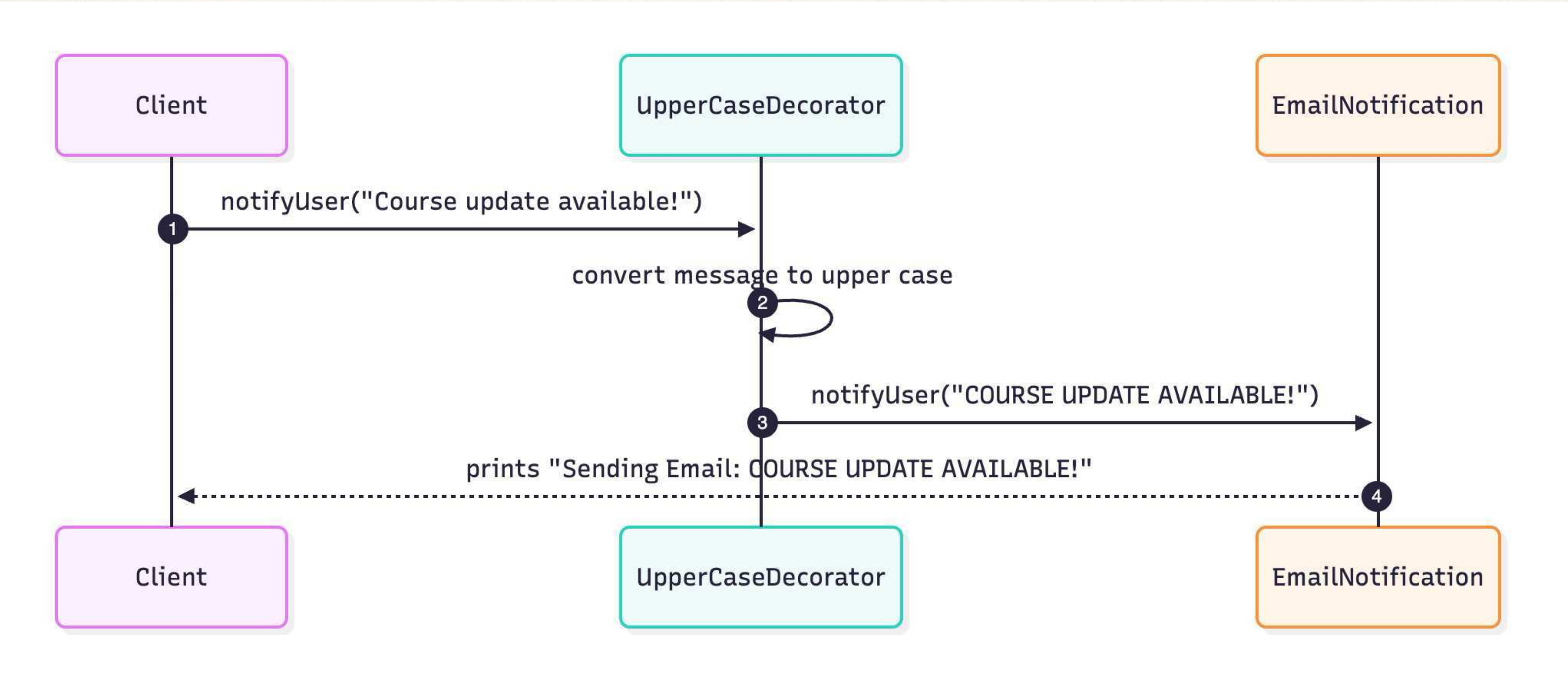


Component Diagram - UpperCaseDecorator

- Initiate a call to UpperCaseDecorator, which converts the message text to uppercase for emphasis
- Delegates the message to the notification object



Sequence Diagram - UpperCaseDecorator

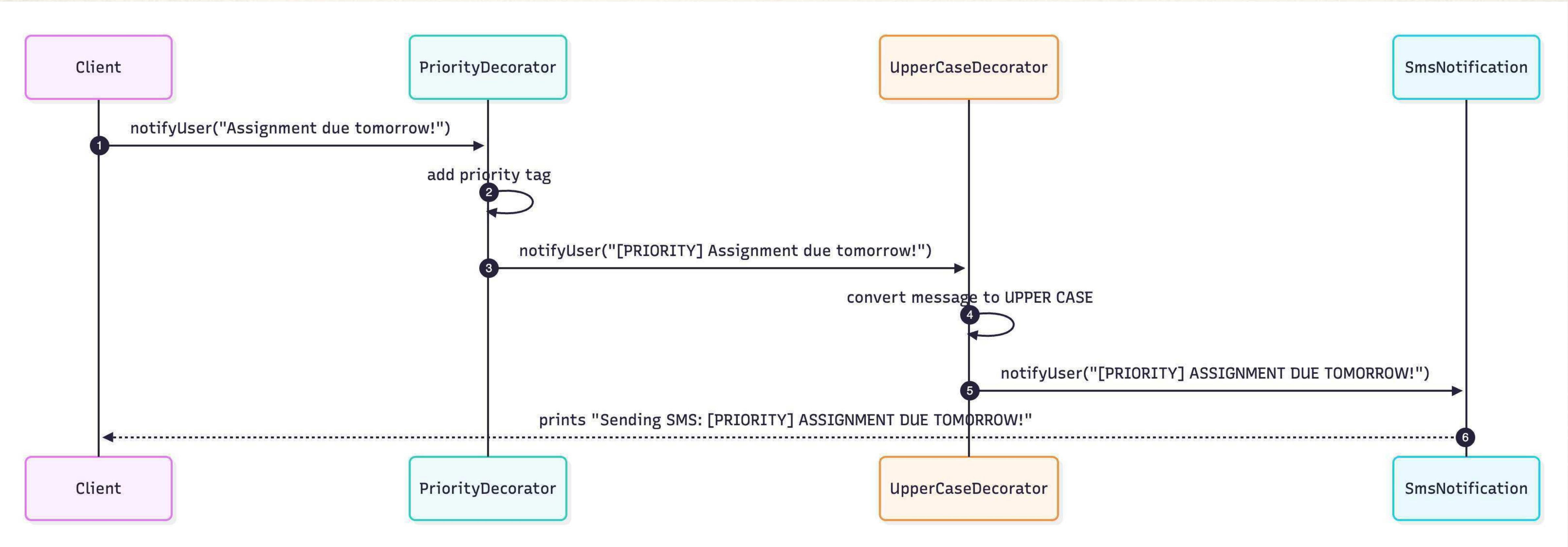


Chaining Decorators

- Chaining decorators allow us to wrap an object with multiple decorators
- Each decorator adds behavior and delegates to the next
- In our example, each decorator will modify the message and pass it along the chain
- A notification can be marked as **priority**, then **uppercased** before being sent



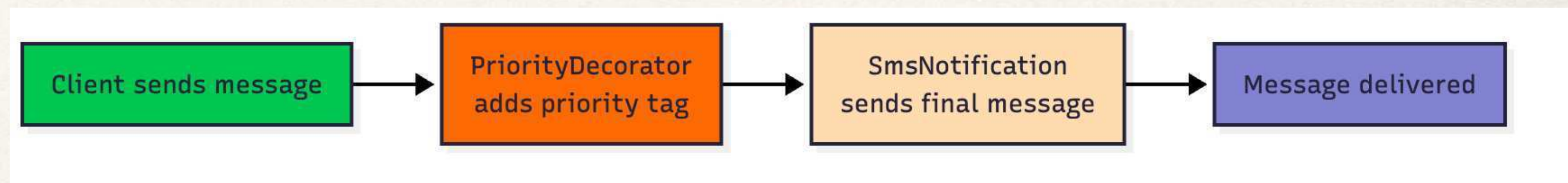
Sequence Diagram - Chaining Decorators



Development Process

Step-By-Step

1. Refactor Notifications to accept a string message
2. Create NotificationDecorator (abstract decorator, base class)
3. Create PriorityDecorator (concrete decorator)
4. Integrate into main app
5. Repeat for other decorators ...



Composite Design Pattern



Composite Design Pattern

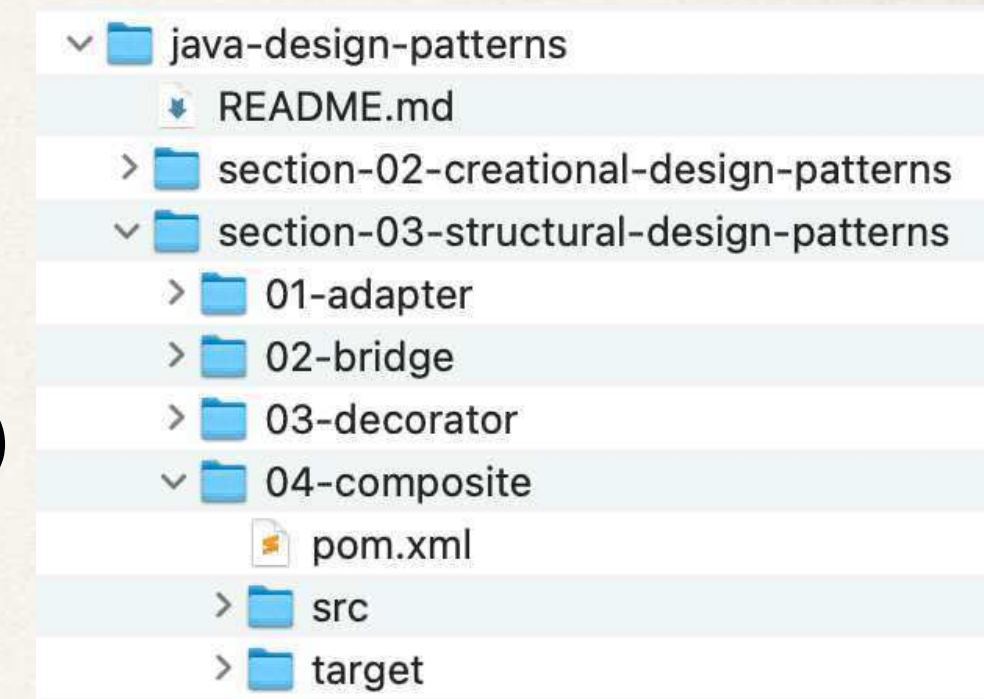
Intent

Problem

Solution

Leaf: terminal object, no children
Composite: container object, has children

- **Intent**
 - Compose objects into tree structures
 - Treat leaves and composites uniformly
- **Problem**
 - Applications need to represent hierarchies (folders / files, orgs / departments, clusters / nodes)
 - Clients must distinguish between leaf vs composite types ... ugly conditional code
 - Direct approach leads to awkward, tightly coupled, hard-to-extend design
- **Solution**
 - Component interface defines common operations for both leaves and composites
 - Leaf vs Composite: leaf does work directly; composite holds children and delegates recursively
 - Client treats all objects uniformly through a defined component interface



Composite Design Pattern

Benefits

Uniform: Treat single and group objects alike
Extensible: Add new types without changes
Simple: Recursion replaces conditionals

Intent

Problem

Solution

- **Intent**

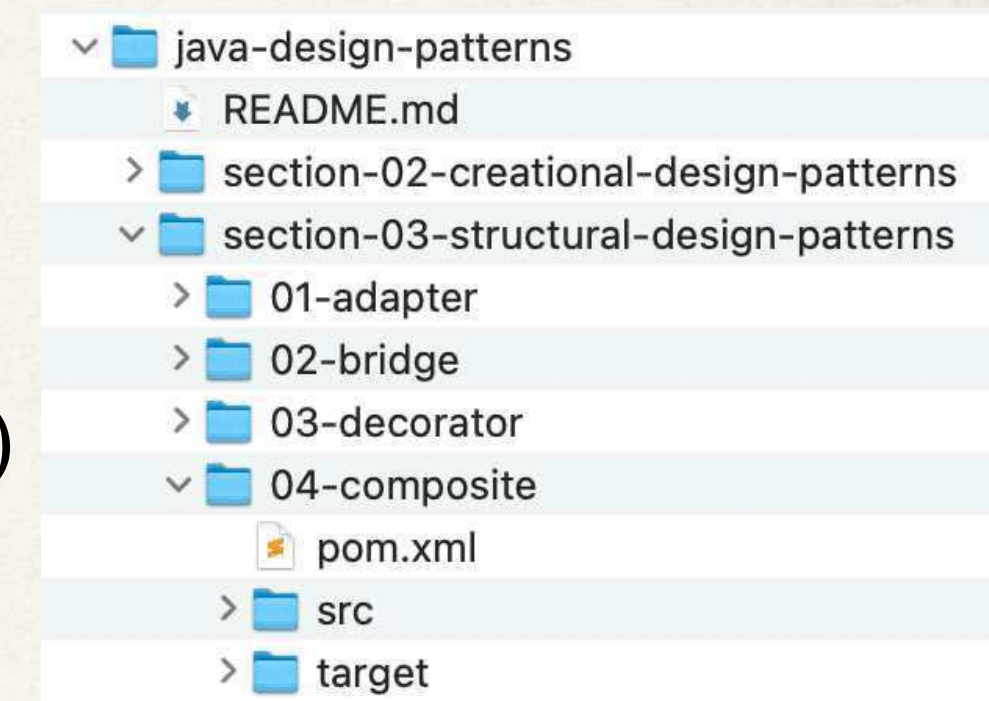
- Compose objects into tree structures
- Treat leaves and composites uniformly

- **Problem**

- Applications need to represent hierarchies (folders / files, orgs / departments, clusters / nodes)
- Clients must distinguish between leaf vs composite types ... ugly conditional code
- Direct approach leads to awkward, tightly coupled, hard-to-extend design

- **Solution**

- Component interface defines common operations for both leaves and composites
- Leaf vs Composite: leaf does work directly; composite holds children and delegates recursively
- Client treats all objects uniformly through a defined component interface



Examples

- Game Engine
- Workflow Management System
- File System (Folders and Files)

Game Engine

Benefits

Uniform: Treat single and group objects alike
Extensible: Add new types without changes
Simple: Recursion replaces conditionals

- **Use Case**
 - A scene has single objects (player, tree) and groups (level, world)
 - Managing separately makes rendering/updates complex
- **Solution**
 - Component: `GameObject` interface - `render()`, `update()`
 - Leaf: `Sprite`, `Mesh`, `Light`
 - Composite: `GameObjectGroup` that holds children
 - Composite calls `render/update` on children recursively
- **Why Composite?**
 - Uniform: treat object and hierarchies the same
 - Recursive: simplify scene graph traversal

Workflow Management System

Benefits

Uniform: Treat single and group objects alike
Extensible: Add new types without changes
Simple: Recursion replaces conditionals

- **Use Case**

- Workflows have simple tasks (send email) and groups (onboarding workflow process)
- Managing both separately complicates execution and tracking

- **Solution**

- Component: Task interface: `execute()`, `status()`
- Leaf: `EmailTask`, `ValidationTask`
- Composite: `Workflow`
- Composite delegates to children recursively

- **Why Composite?**

- Uniform: treats tasks and workflows the same
- Recursive: execute and track easily

File System (Files and Folders)

Benefits

Uniform: Treat single and group objects alike
Extensible: Add new types without changes
Simple: Recursion replaces conditionals

- **Use Case**

- File systems have files and folders
- Managing separately complicates code for file size, display, attributes etc

- **Solution**

- Component: `FileSystemComponent` interface: `getSize()`, `print()`
- Leaf: `File`
- Composite: `Folder`
- Composite delegates to children recursively

- **Why Composite?**

- Uniform: treats files and folders the same
- Recursive: calculate size and display easily



Composite Design Pattern

File System Example

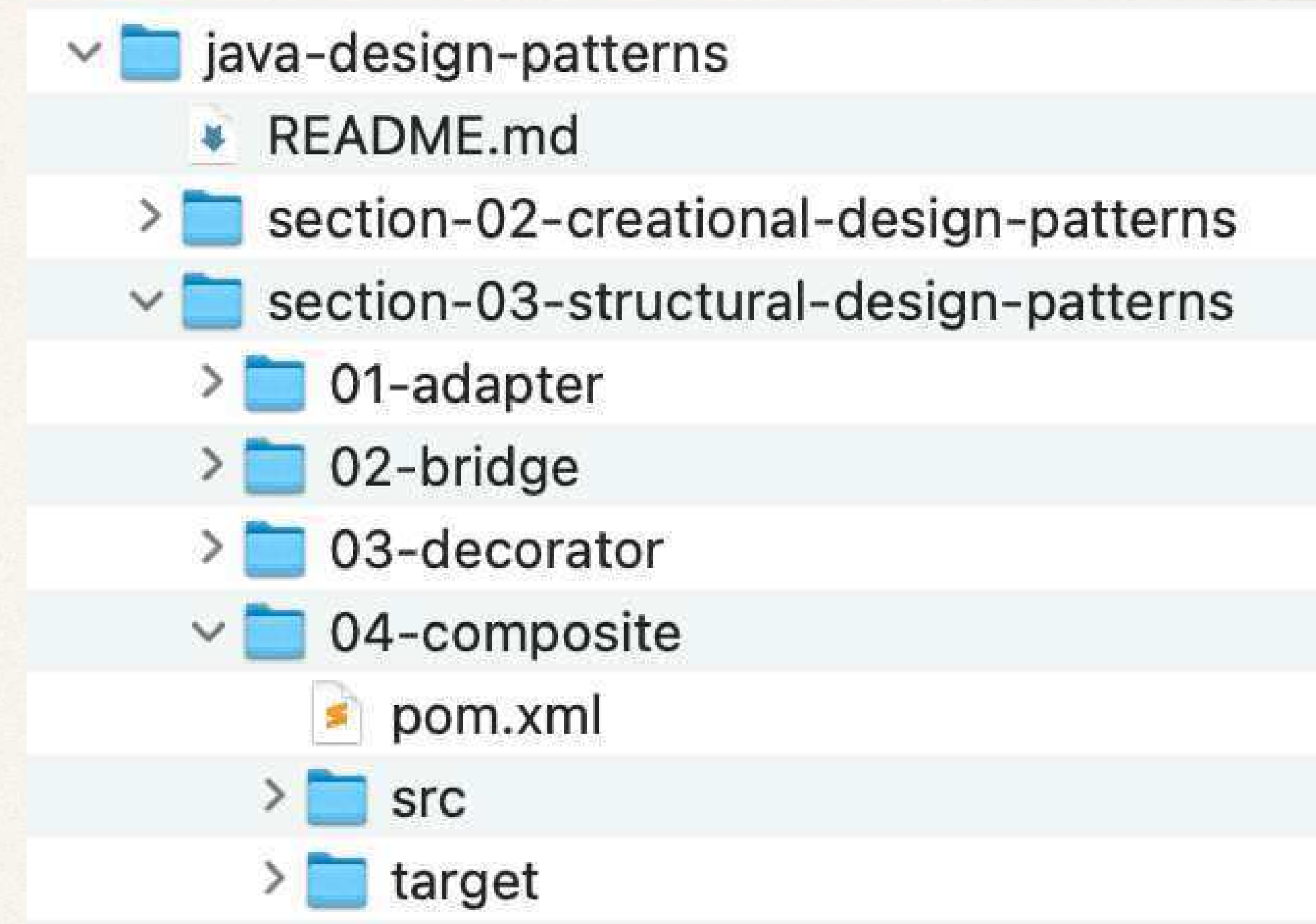


File System (Files and Folders)

Benefits

Uniform: Treat single and group objects alike
Extensible: Add new types without changes
Simple: Recursion replaces conditionals

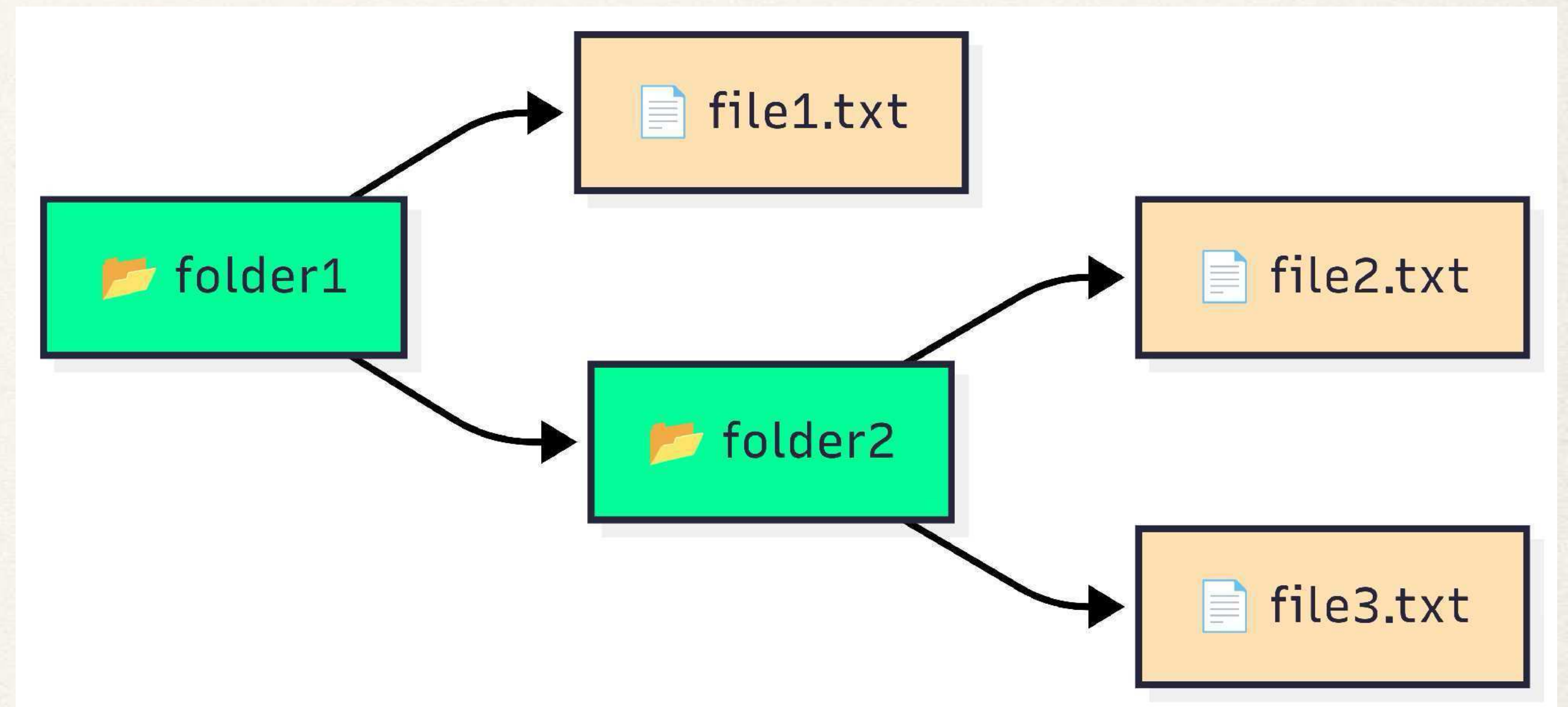
- Use Case
 - File systems have files and folders
 - Managing separately complicates code for file size, display, attributes etc



Composite Design Pattern

- **Solution**

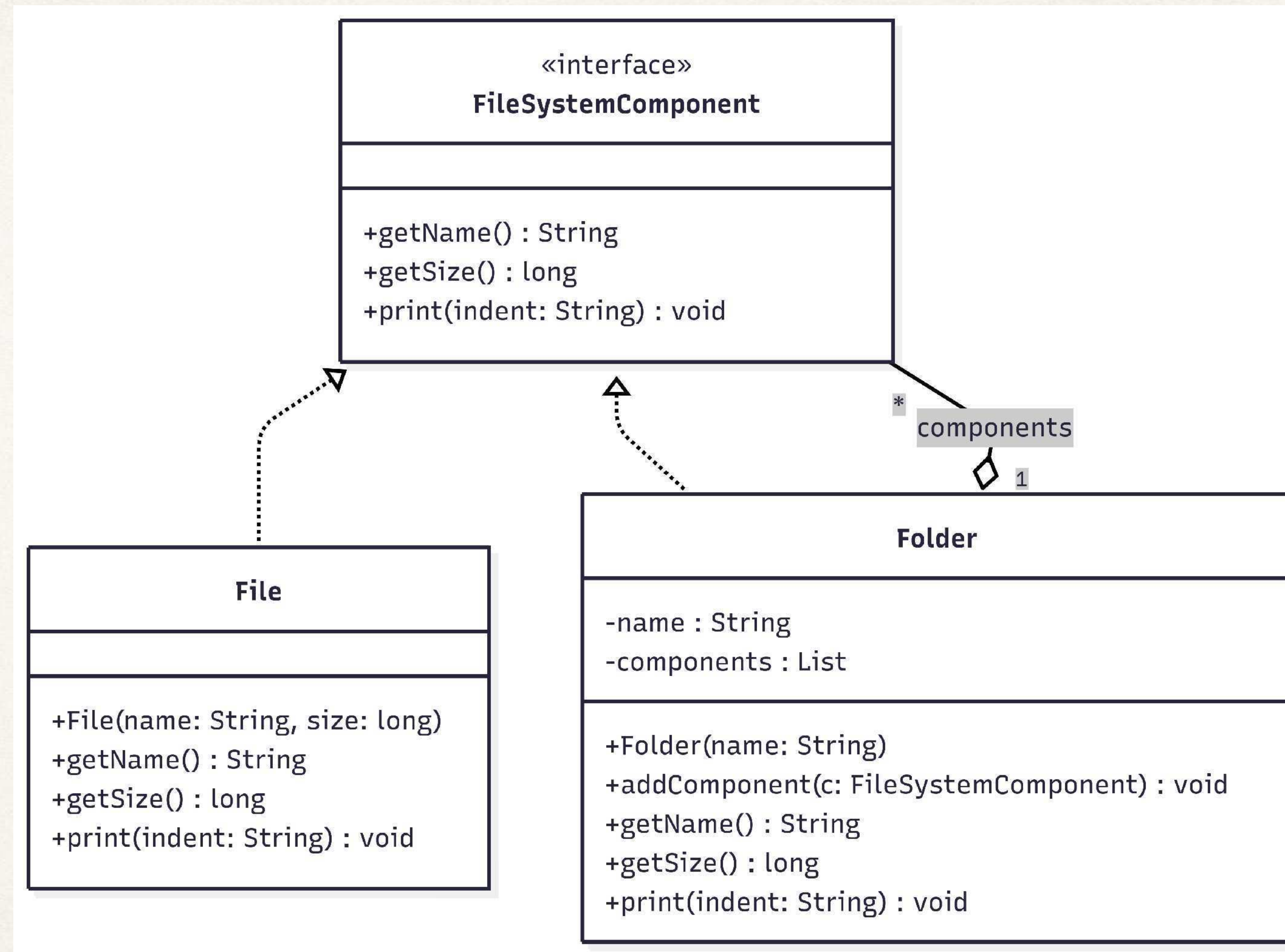
- Component: `FileSystemComponent` interface: `getSize()`, `print()`
- Leaf: `File`
- Composite: `Folder`
- Composite delegates to children recursively



The Components

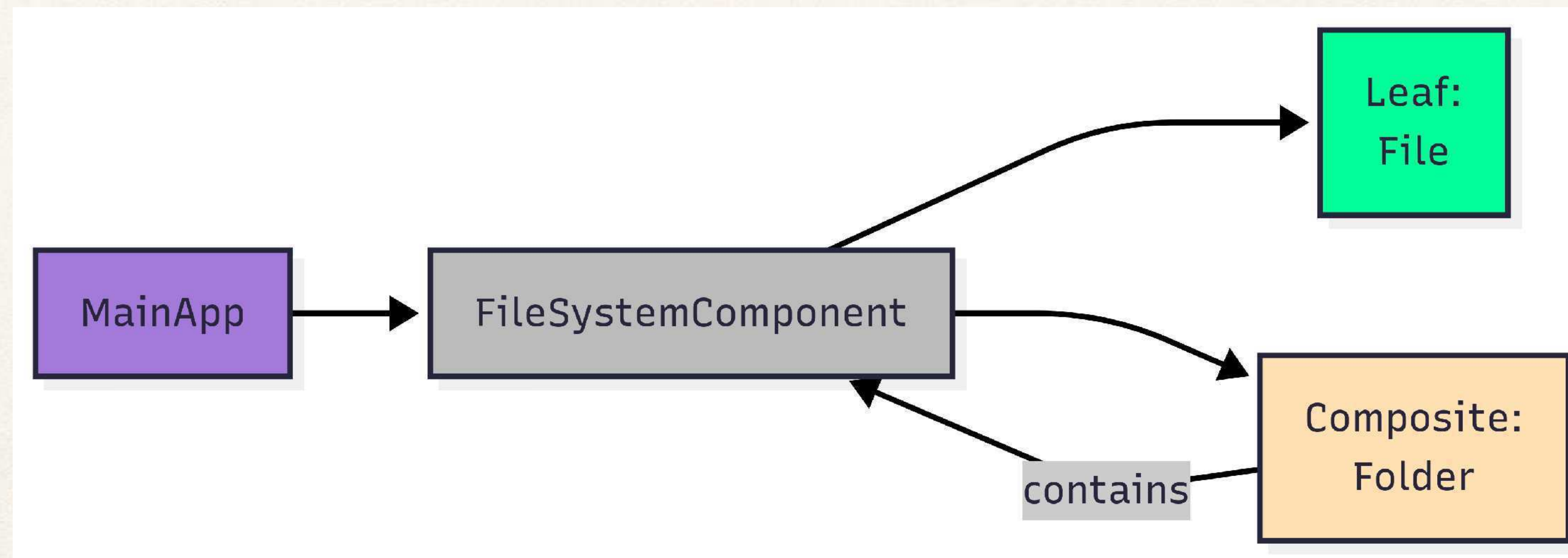
Component	Role	Description
FileSystemComponent	Component	Common interface with methods like getSize() and print()
File	Leaf	Represents an individual file
Folder	Composite	Represents a folder Contains a collect of children (files or folders) Delegates operations recursively

UML Diagram

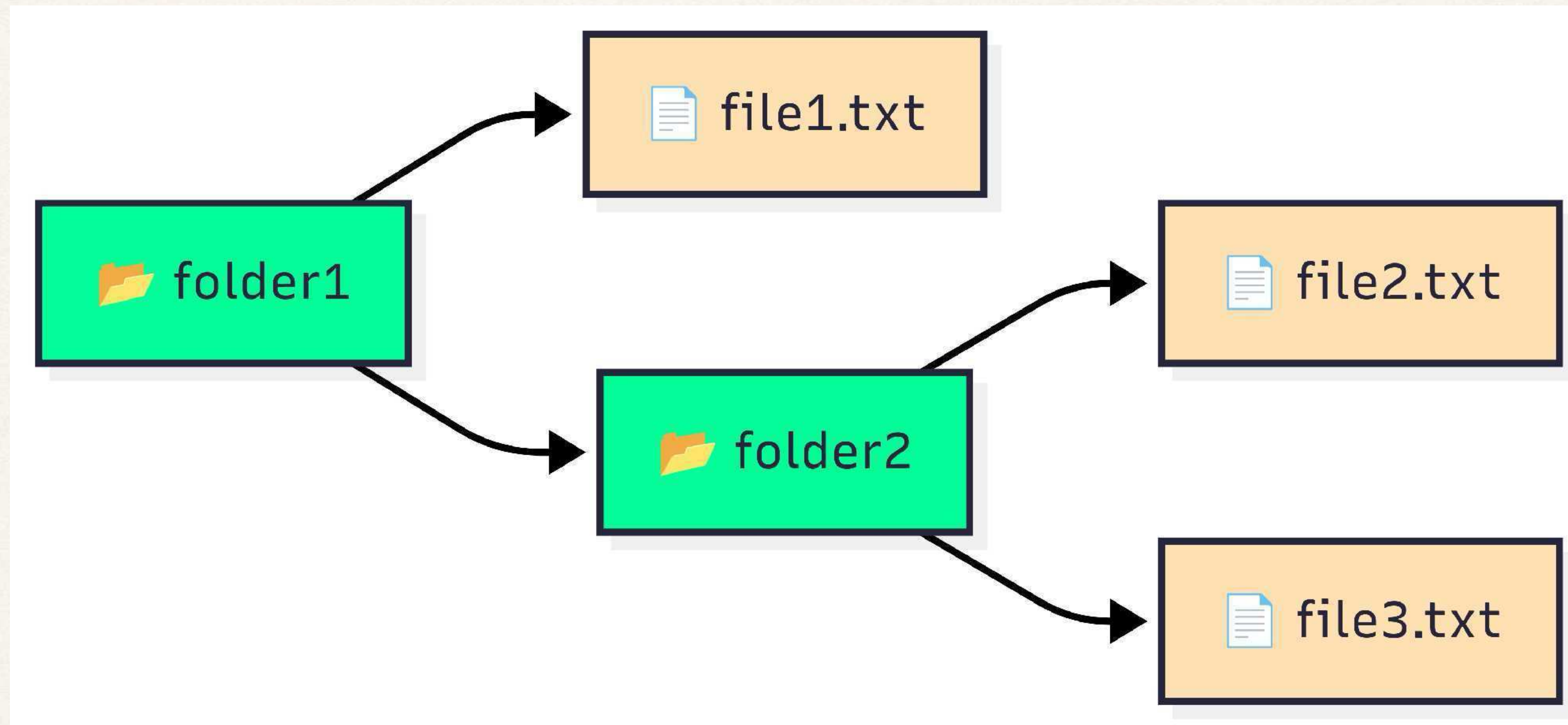


Component Diagram - Composite

- The main app performs actions on a `FileSystemComponent`
- For component that is a **Leaf: File**, it returns its size or prints itself
- For component that is a **Composite: Folder**, it delegates the call to its children recursively
- Demonstrates the Composite pattern ... uniform treatment of files and folders

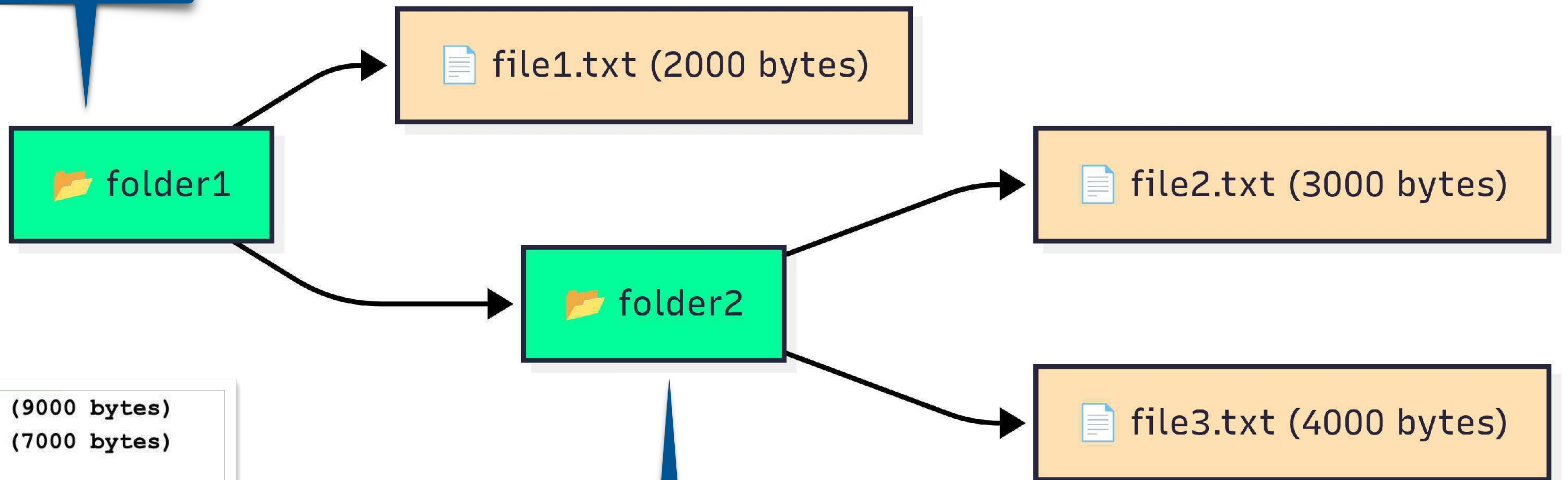


Files and Folders Example



Files and Folders Example

Total size: (9000 bytes)

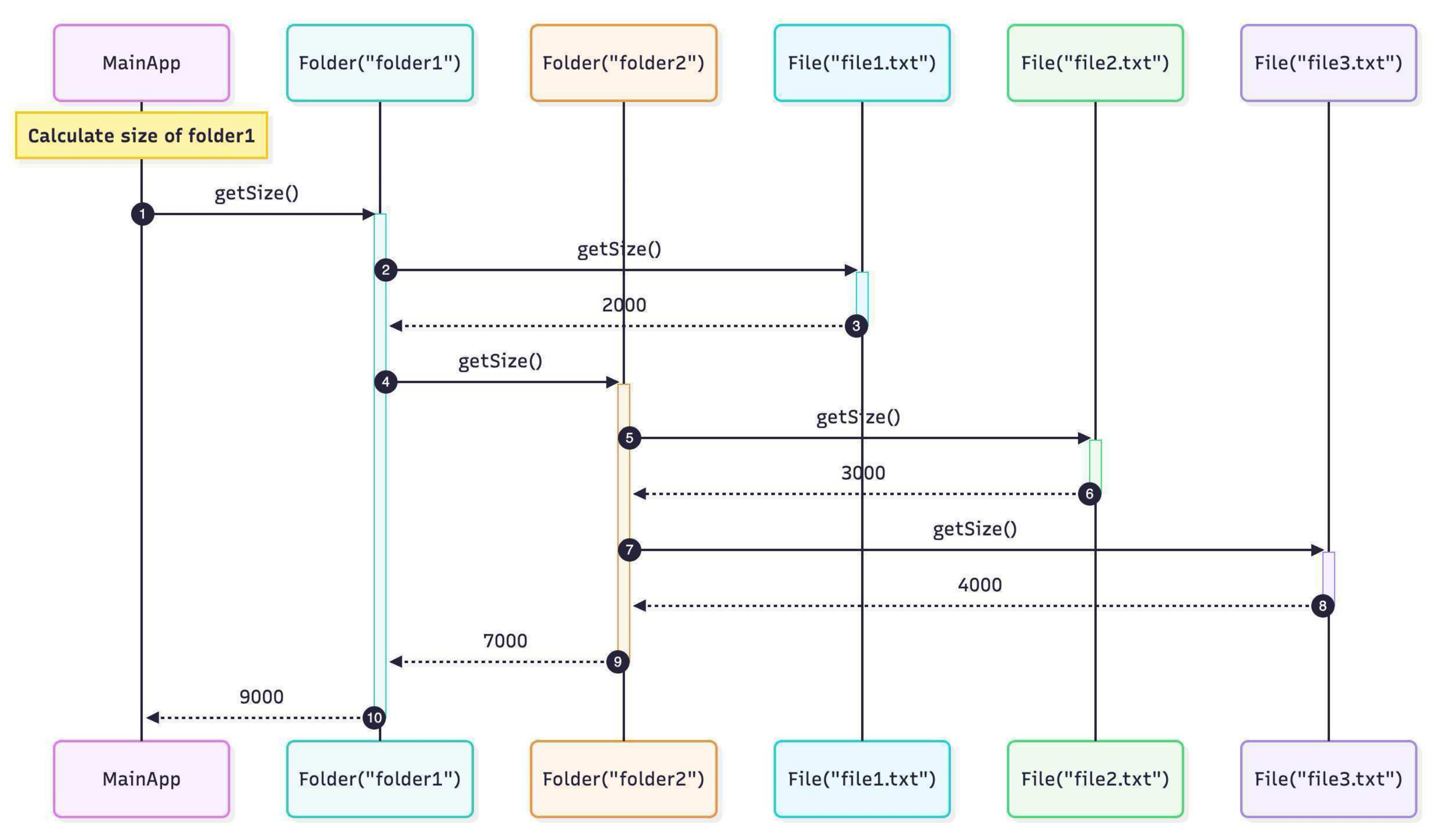
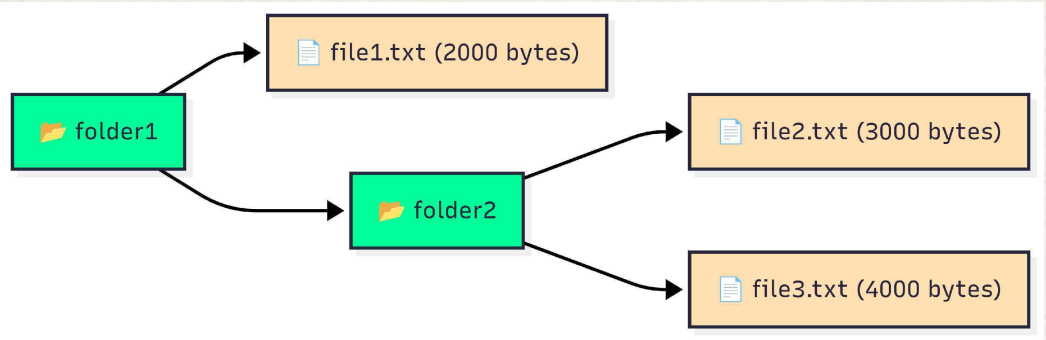


```
folder1 size: (9000 bytes)
folder2 size: (7000 bytes)
```

```
+ folder1/ (9000 bytes)
- file1.txt (2000 bytes)
+ folder2/ (7000 bytes)
- file2.txt (3000 bytes)
- file3.txt (4000 bytes)
```

Total size: (7000 bytes)

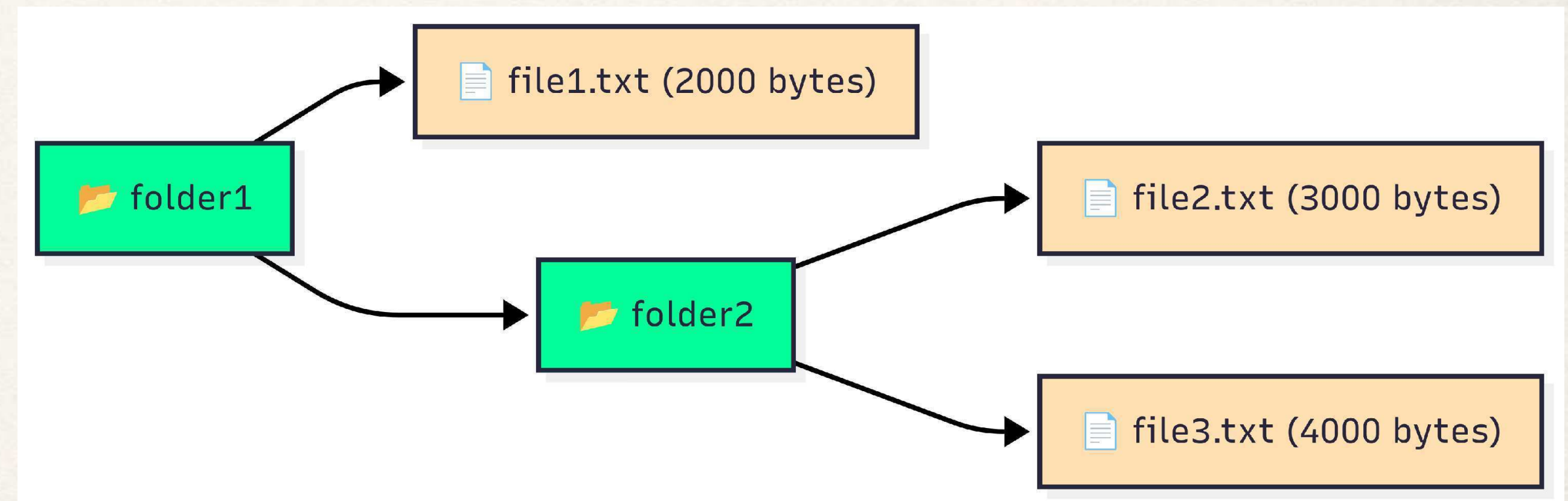
Sequence Diagram - Composite



Development Process

Step-By-Step

1. Create component: `FileSystemComponent`
2. Create leaf: `File`
3. Create composite: `Folder`
4. Integrate into main app



Facade Design Pattern



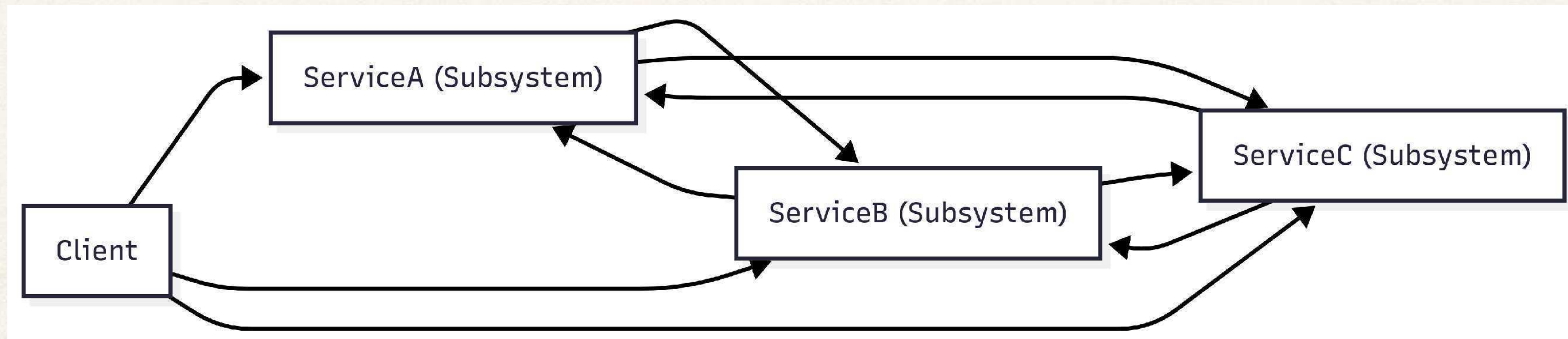
Facade Design Pattern

Intent

Problem

Solution

- **Intent**
 - Provide a unified interface to a set of interfaces in a subsystem
 - Define a higher-level interface that makes the subsystem easier to use
- **Problem**
 - Applications depend on complex subsystems (frameworks, libraries, many classes)
 - Clients must initialize/configure objects, track dependencies, and call methods in the right order
 - Leads to tight coupling, verbose, hard-to-maintain code



Facade Design Pattern

Benefits

Simple: One unified interface
Decoupled: Less client dependency
Flexible: Subsystems can change freely

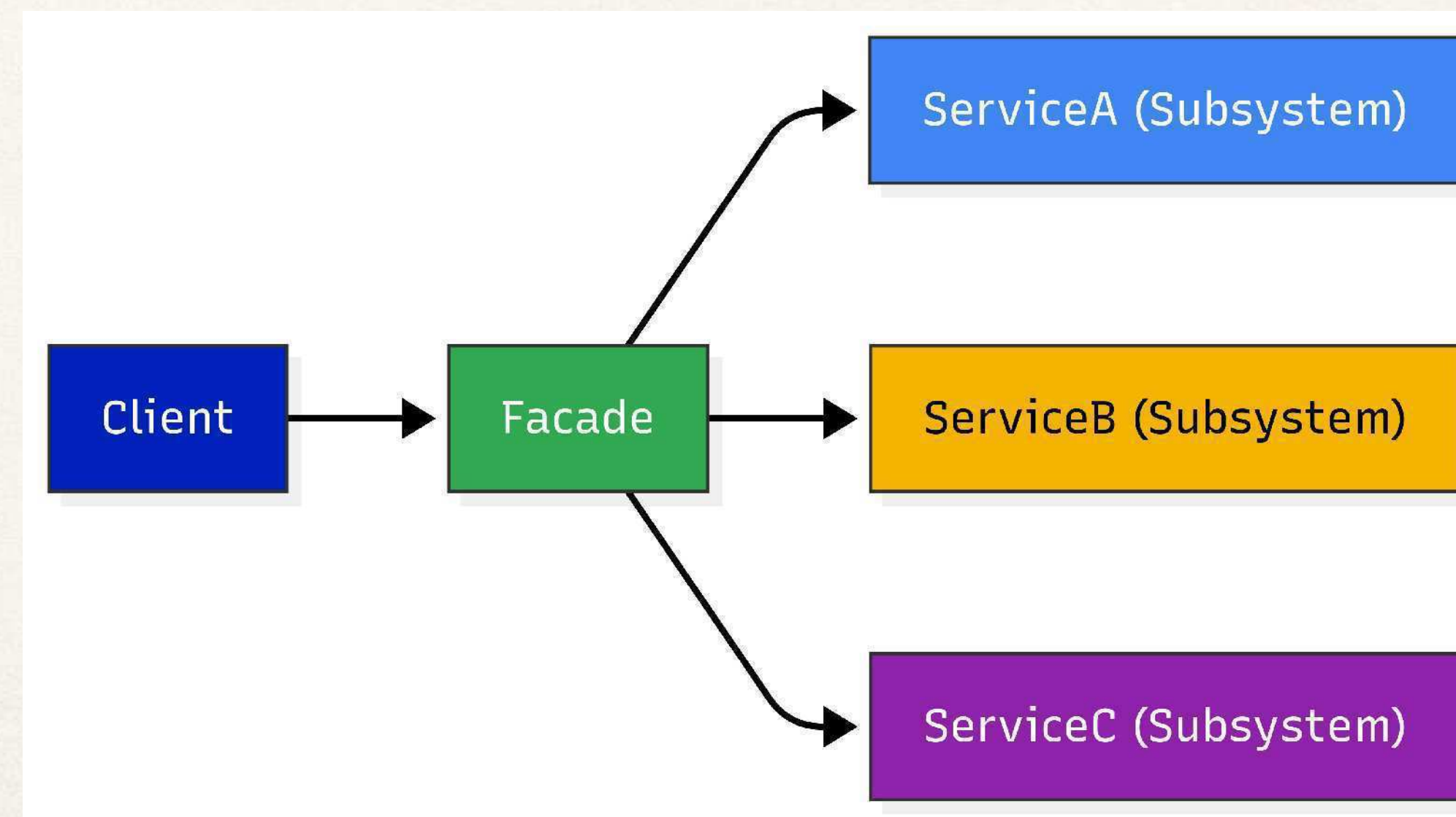
Intent

Problem

Solution

- **Solution**

- Introduce a Facade class that exposes only the features clients care about
- Facade delegates requests to appropriate subsystem classes
- Simplifies client code and shields it from subsystem complexity



Examples

- Vehicle Dashboard
- Travel Booking App

Vehicle Dashboard

Benefits

Simple: One unified interface
Decoupled: Less client dependency
Flexible: Subsystems can change freely

- **Use Case**

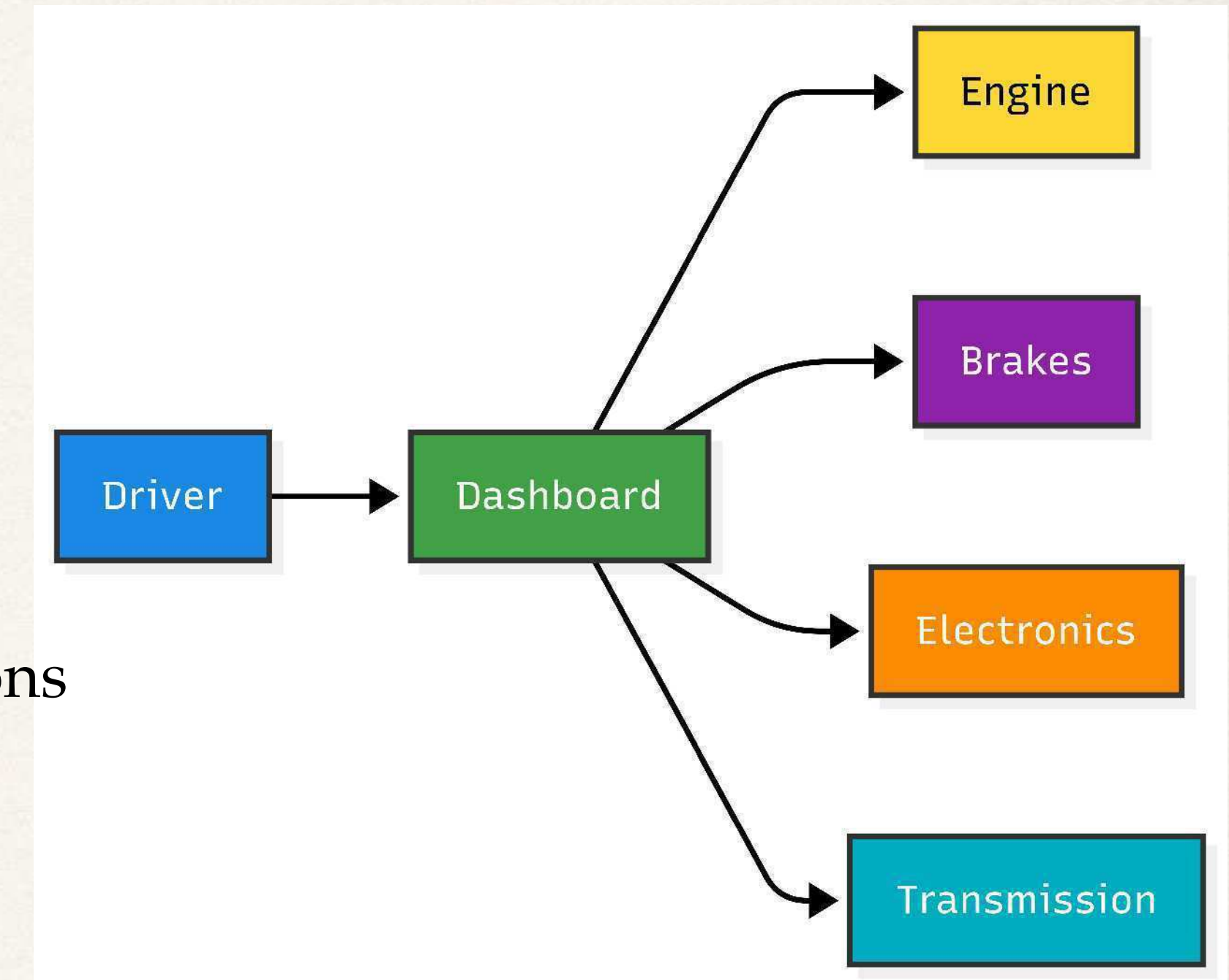
- Cars have many subsystems (engine, brakes, electronics, transmission)
- Directly controlling each would be overwhelming for the driver

- **Solution**

- Facade: Dashboard with steering wheel, pedals, and buttons
- Subsystems: Engine, brakes, electronics and transmission
- Dashboard translates simple user actions into complex subsystem operations

- **Why Facade?**

- Simple: One interface for many systems
- Decoupled: Driver doesn't manage technical details
- Safe: Consistent, error-free control



Travel Booking App

Benefits

Simple: One unified interface
Decoupled: Less client dependency
Flexible: Subsystems can change freely

- **Use Case**

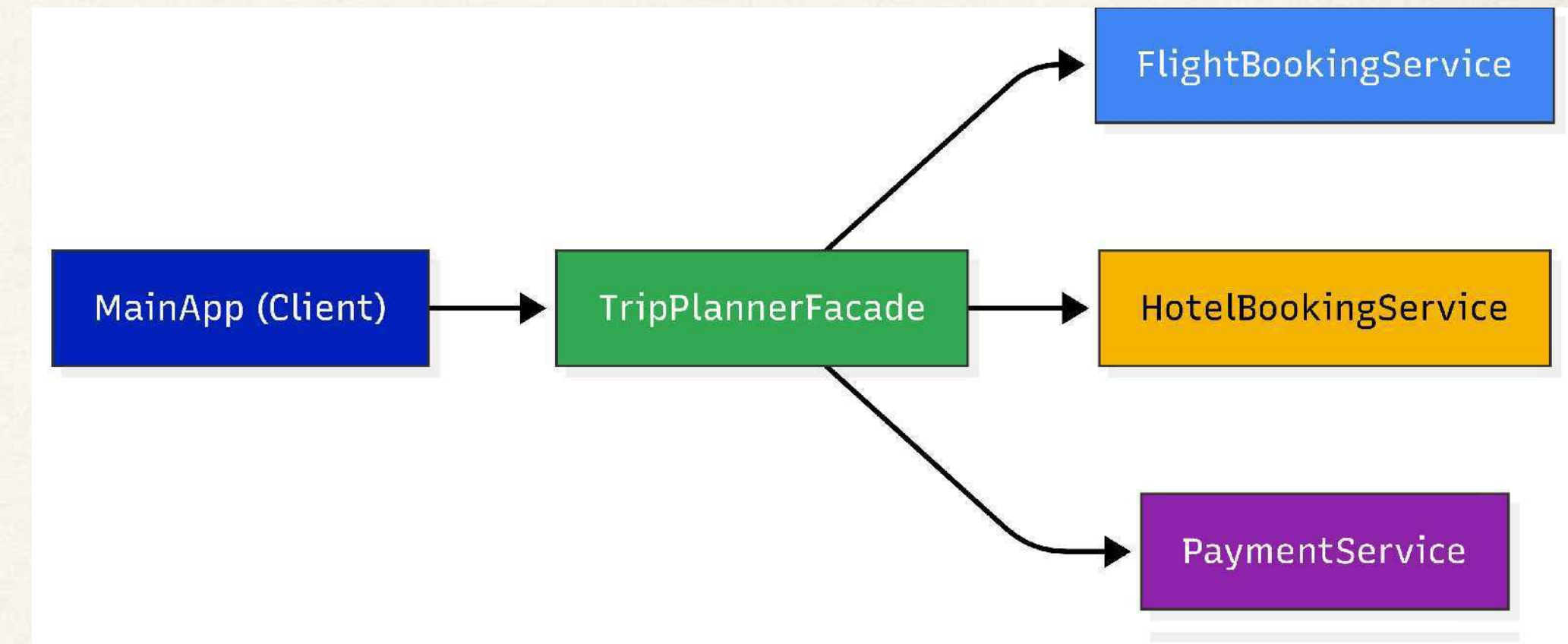
- Booking a trip involves multiple services (flights, hotels, payments)
- Manually coordinating each service is repetitive and error-prone

- **Solution**

- Facade: `TripPlannerFacade` with `bookCompleteTrip()`
- Subsystems: Flight service, hotel service, payment service
- Facade sequences calls and returns a unified confirmation

- **Why Facade?**

- Simple: One call replaces many
- Decoupled: Client avoids subsystem complexity
- Unified: Single confirmation aggregates results



Facade Design Pattern

Travel Booking App Example



Travel Booking App

Benefits

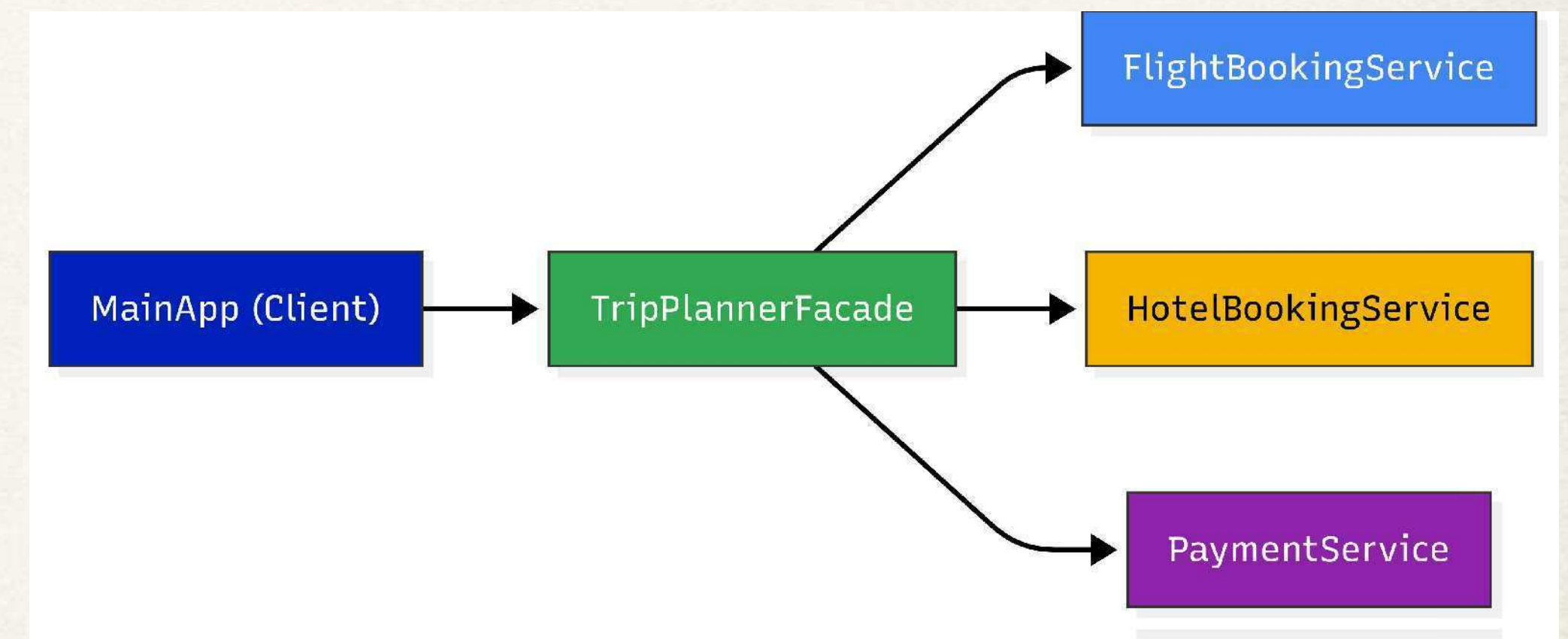
Simple: One unified interface
Decoupled: Less client dependency
Flexible: Subsystems can change freely

- **Use Case**

- Booking a trip involves multiple services (flights, hotels, payments)
- Manually coordinating each service is repetitive and error-prone

- **Solution**

- Facade: `TripPlannerFacade` with `bookCompleteTrip()`
- Subsystems: Flight service, hotel service, payment service
- Facade sequences calls and returns a unified confirmation



The Components

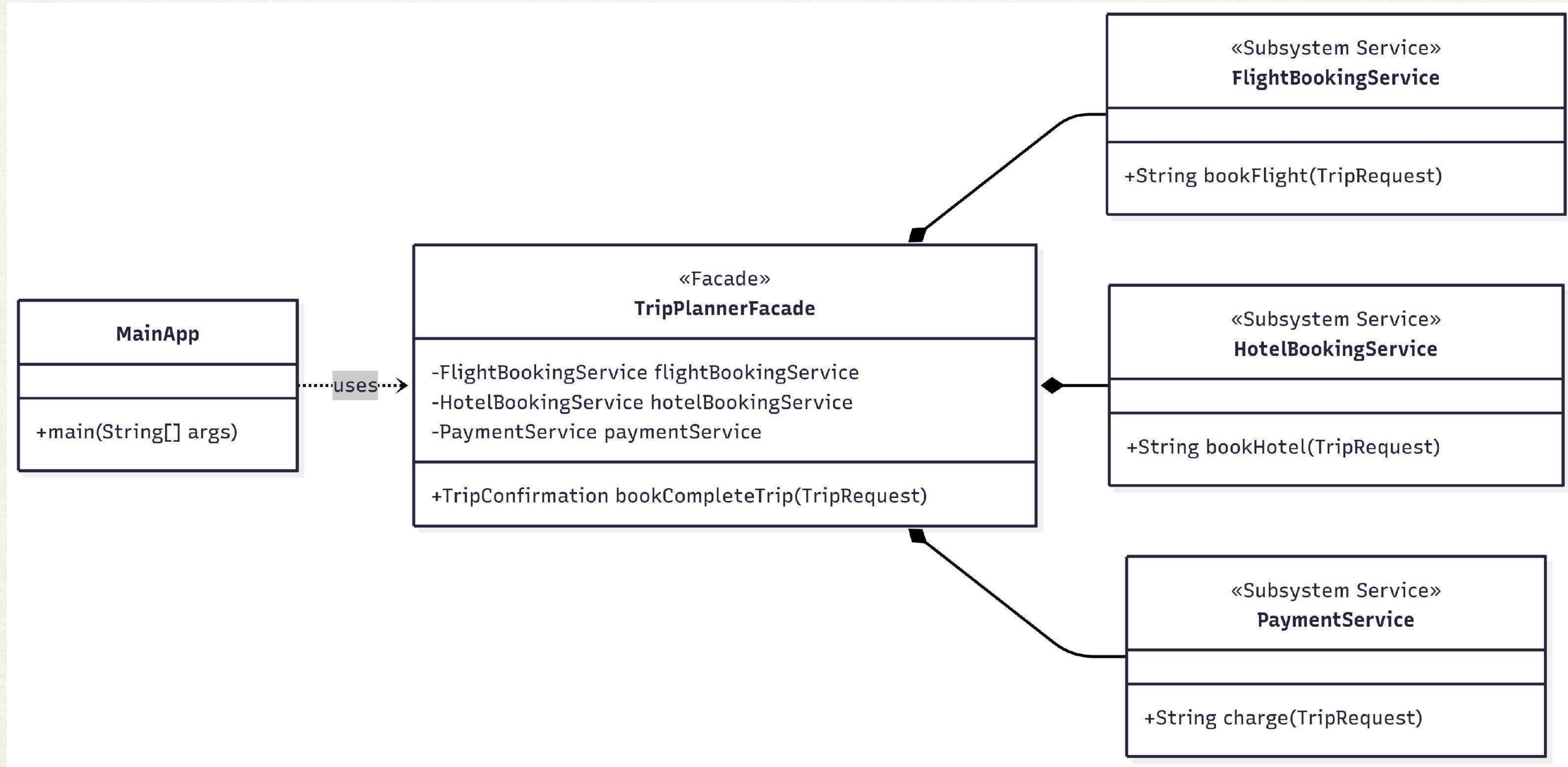
Component	Role	Description
MainApp	Client	Entry point: creates a trip request and calls the Facade
TripPlannerFacade	Facade	Provides a unified interface; coordinates flights, hotels, and payments.
FlightBookingService	Subsystem	Handles flight reservations (origin, destination, dates, guests).
HotelBookingService	Subsystem	Manages hotel bookings (check-in, check-out, guests).
PaymentService	Subsystem	Processes payments, hides transaction details from the client.

Supporting Components - Value Objects

- **Value Object**
 - Simple data holder that never changes once it's created (immutable)
 - Modeled as a domain concept: `TripRequest`, `TripConfirmation`
 - Passed between layers (client -> facade -> subsystems) to encapsulate state cleanly

Component	Role	Description
<code>TripRequest</code>	Value Object	Input details: origin, destination, dates, guests, payment info.
<code>TripConfirmation</code>	Value Object	Aggregates results from all subsystems into a single confirmation.

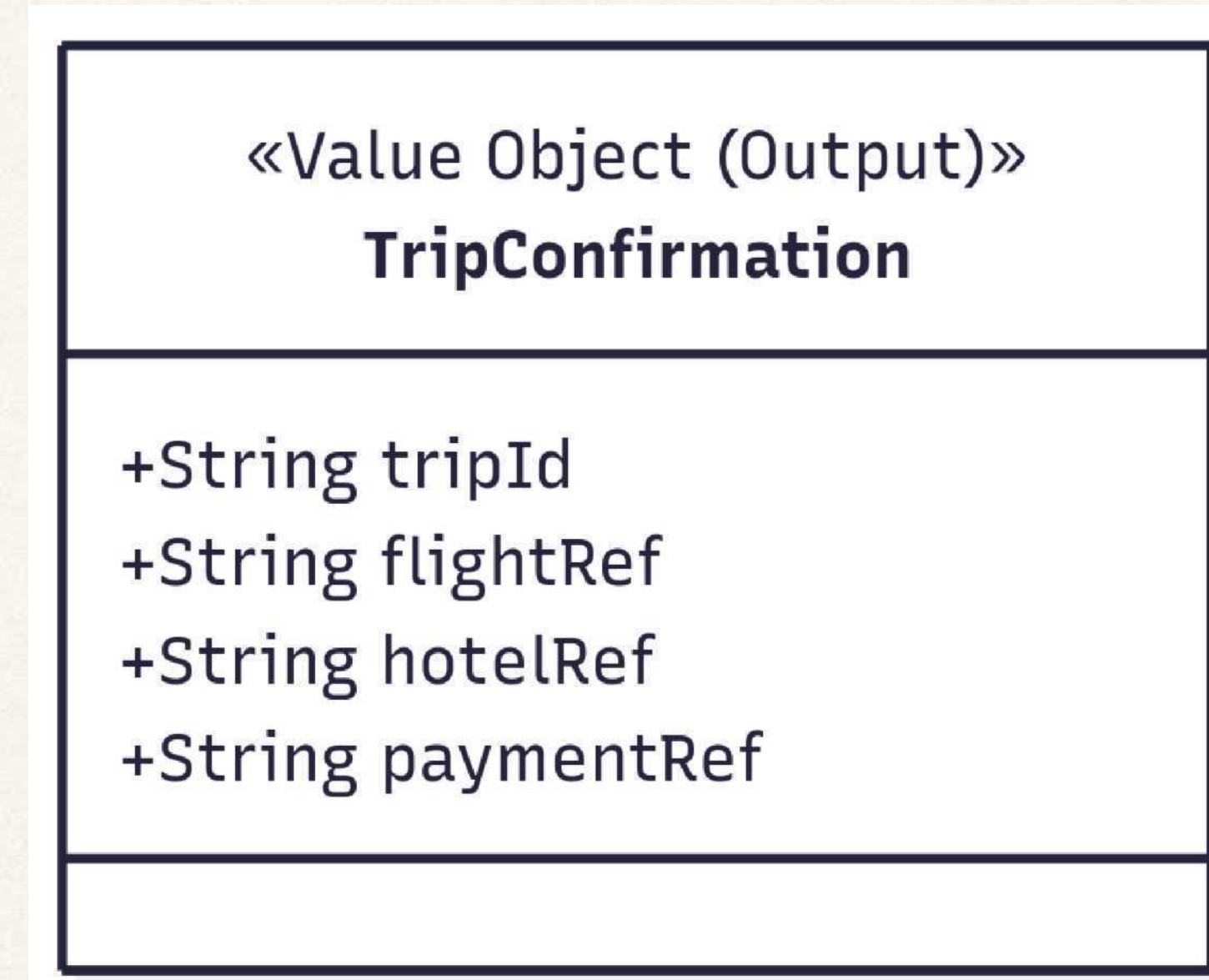
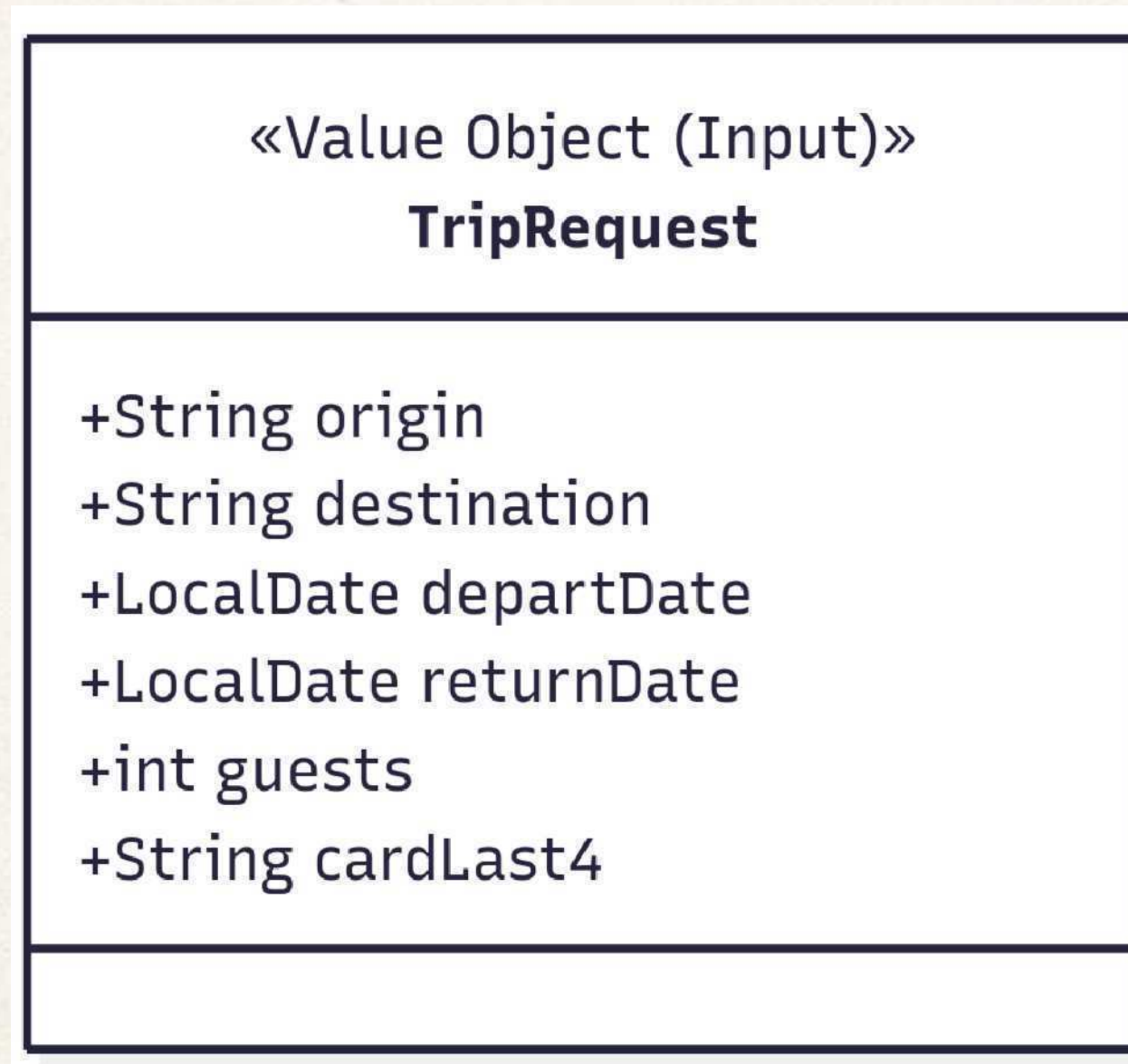
UML Diagram - Key Classes



UML Diagram - Value Objects

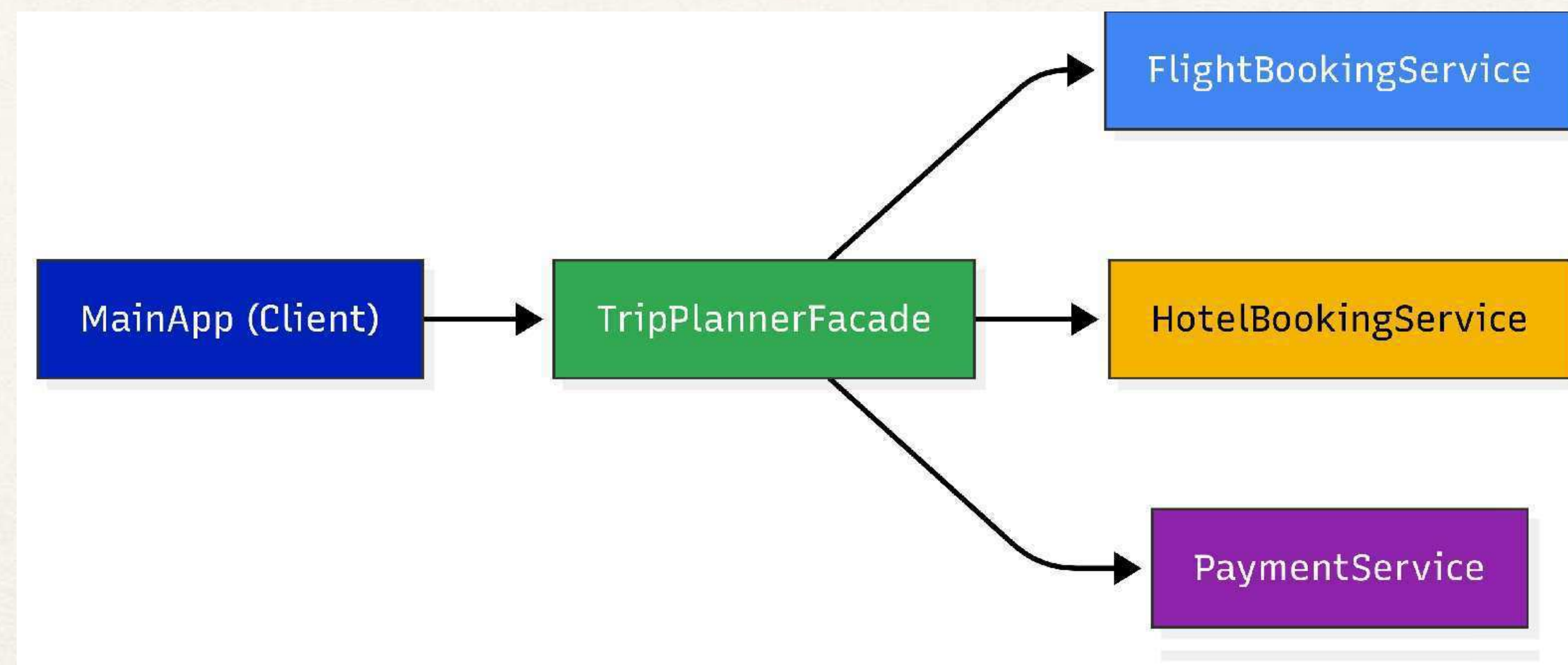
Value Objects

Data carrier: passed between components to hold state
Domain concept: TripRequest, TripConfirmation

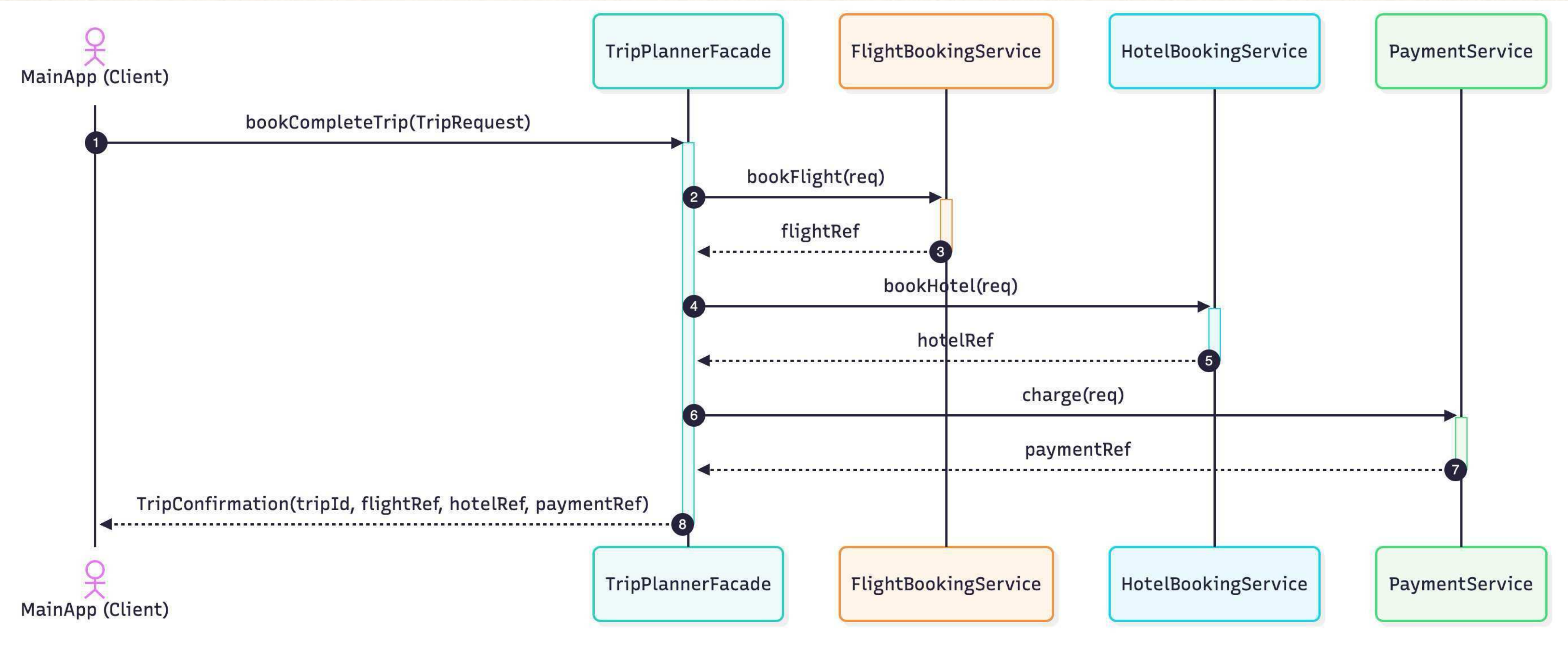
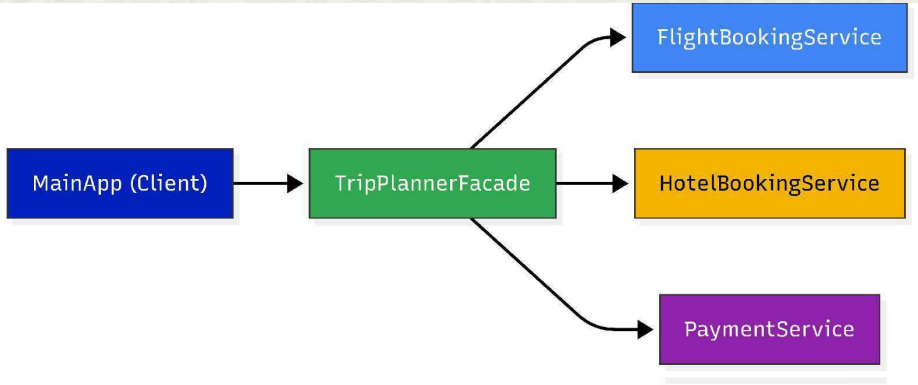


Component Diagram - Facade

- Main app uses Facade: `TripPlannerFacade.bookCompleteTrip()` instead of calling subsystems directly
- Facade orchestrates services in order: flights, hotels, and payments
- Subsystems stay hidden ... client never manages dependencies or sequencing
- Unified confirmation returned, showing how Facade simplifies and reduces coupling



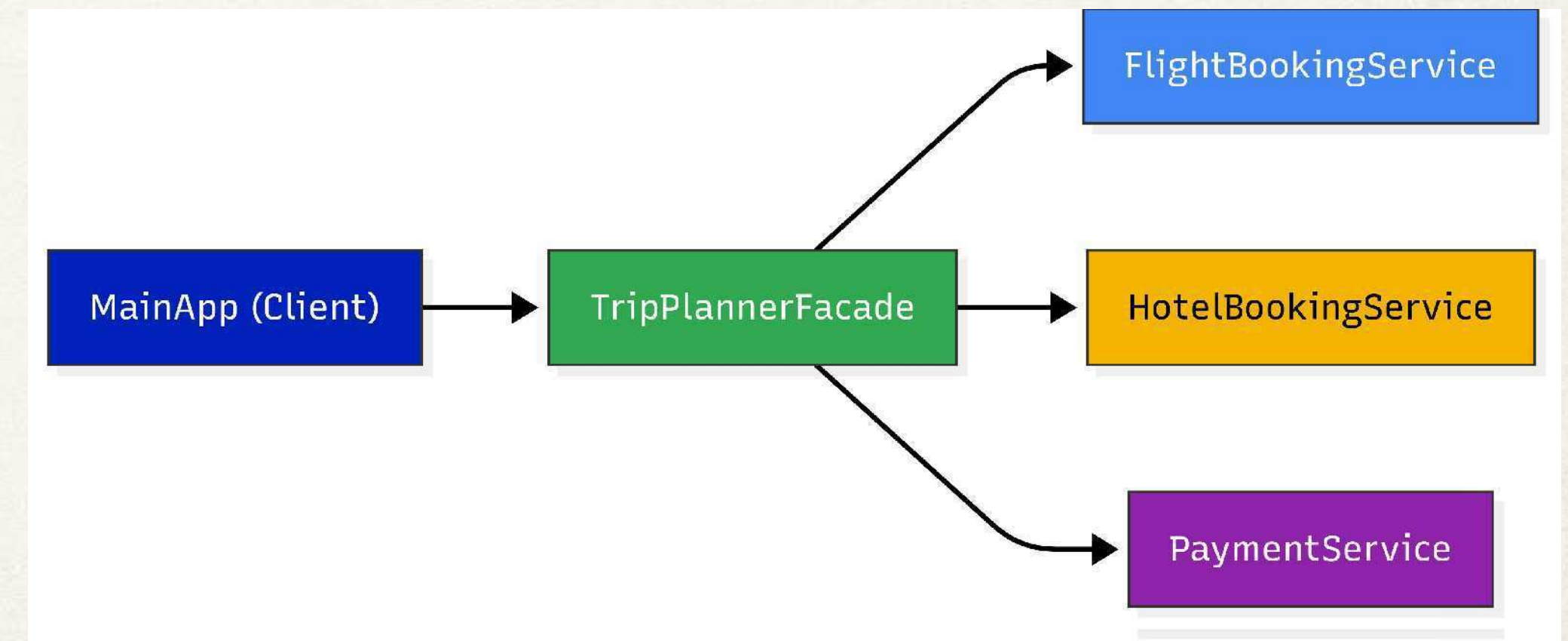
Sequence Diagram - Facade



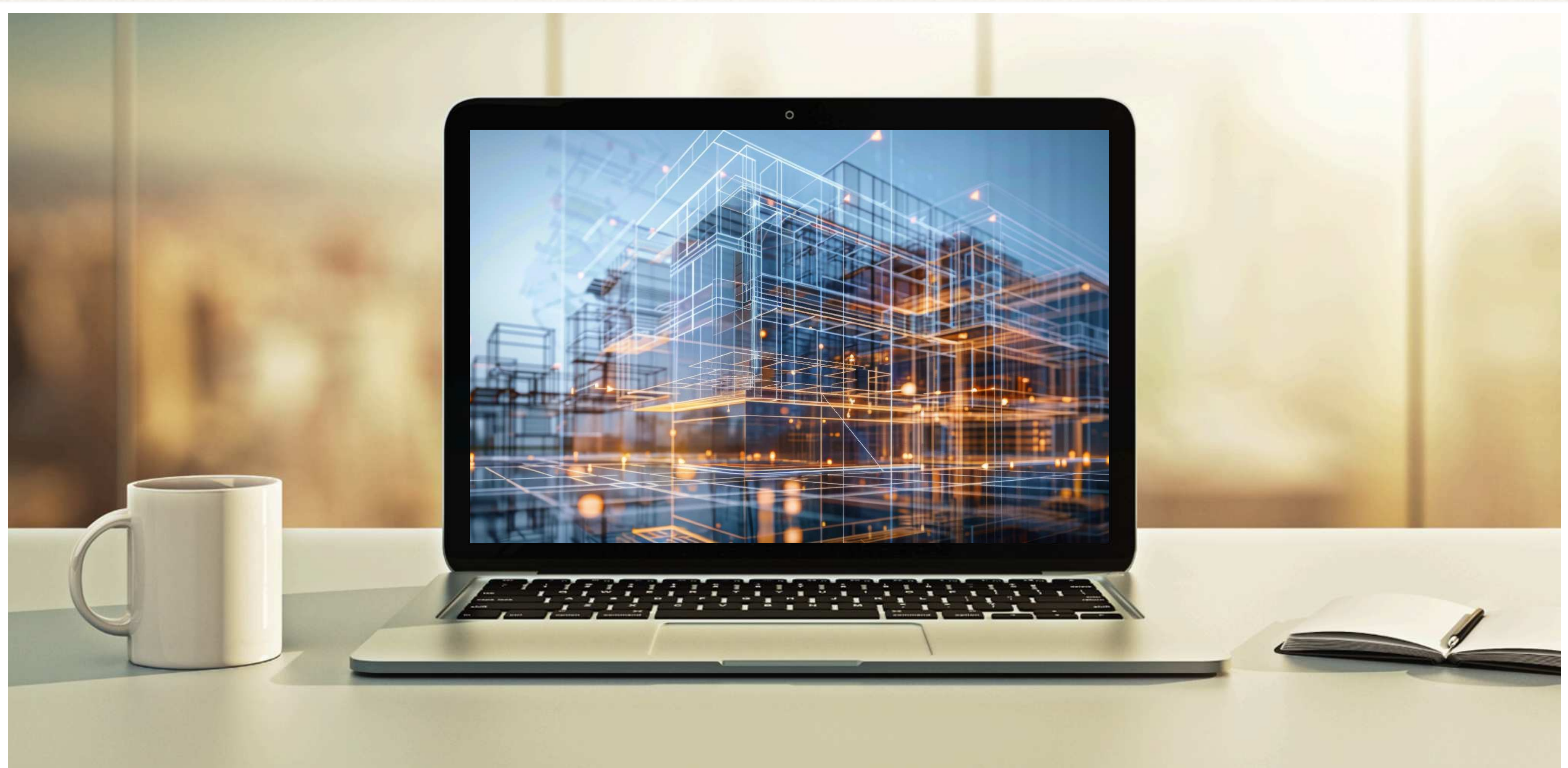
Development Process

Step-By-Step

1. Create value objects: `TripRequest`, `TripConfirmation`
2. Create Subsystems / services:
 - `FlightBookingService`, `HotelBookingService`, `PaymentBookingService`
3. Create Facade: `TripPlannerFacade`
4. Integrate into main app



Behavioral Design Patterns



Java Design Patterns

Creational

Factory
Singleton
Prototype

...



Structural

Adapter
Bridge
Composite

...



Behavioral

Chain
Iterator
Observer

...

Behavioral Patterns

Behavioral

Chain
Iterator
Observer

...

- Focus on object interaction and communication
- Helps assigns responsibilities and manage control flow
- Reduces tight coupling between objects that need to interact
- Makes it easier to manage communication and logic between objects

Behavioral Patterns - Analogies

Behavioral

Chain
Iterator
Observer

...

- Message Filtering Pipeline
 - Messages flow through a sequence of filters (authentication, logging, compression)
- GPS Navigation Apps
 - Different route / strategy based on traffic conditions
- Customer Service Call Center
 - Based on customer's request, route to appropriate department

Behavioral patterns makes it easier to manage communication and logic between objects

Chain of Responsibility Design Pattern



Chain of Responsibility Design Pattern

Intent

Problem

Solution

- **Intent**

- Pass a request along a chain of filters
- Each filter can handle the request in its own way
 - It may modify the request
 - It can pass the request to the next filter or stop the chain
- Decouples the sender of a request from its receiver(s)

- **Problem**

- A request may need to be handled by different objects under different conditions
- Hard-coding all possible handling logic in a single class creates complex, rigid code
- Tight coupling occurs between the sender and specific receivers
- Changing or reordering filters requires modifying existing code

Chain of Responsibility Design Pattern

Intent

Problem

Solution

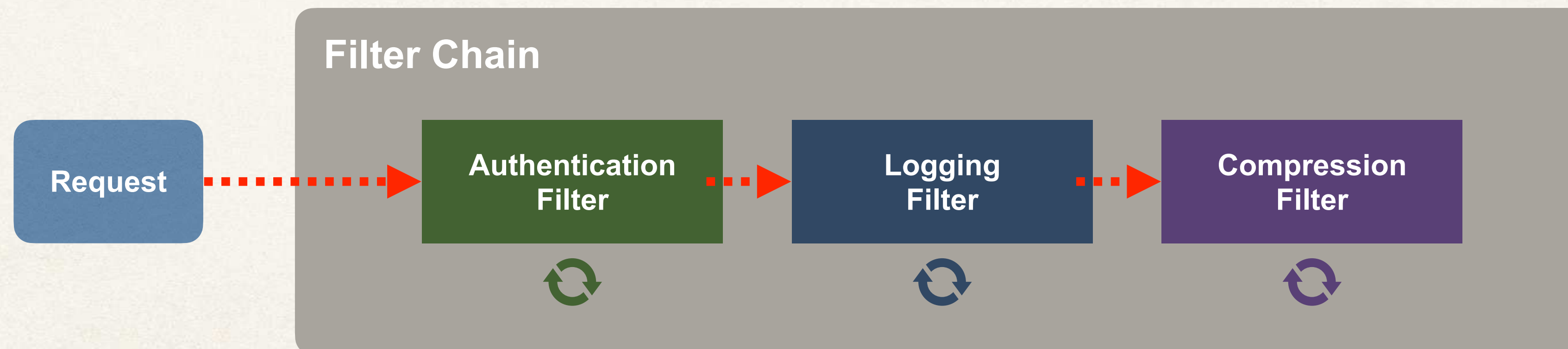
- **Solution**

- Organize filters into a chain
- Each filter:
 - Can process or modify the request
 - May or may not delegate to the next filter in the chain (based on custom logic)
 - The sender only knows the first filter in the chain, not who will eventually handle it

Benefits

Decoupled: Sender and filters are independent
Flexible: Filters can be added, removed or reordered easily
Extensible: Added new filters without changing existing ones

**Filter
Handler
Processor**



Examples

- Medical Triage
- Hiring Approval Process
- Message Filtering Pipeline

Medical Triage

Benefits

Decoupled: Sender and filters are independent
Flexible: Filters can be added, removed or reordered easily
Extensible: Added new filters without changing existing ones

- **Use Case**

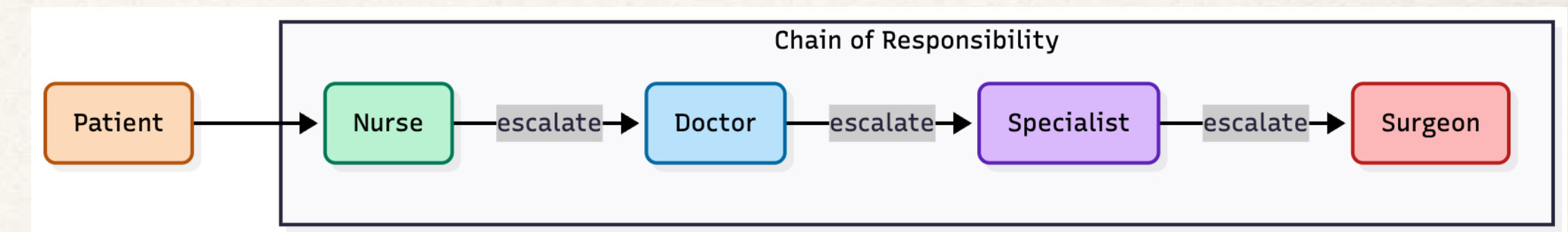
- Hospitals handle patients with different urgency levels
- Multiple medical roles assess and escalate cases based on severity

- **Solution**

- Build a chain: **Nurse -> Doctor -> Specialist -> Surgeon**
- Each role treats or escalates based on severity
- The patient intake triggers only the first step in the chain

- **Why Chain of Responsibility?**

- Decoupled: Intake logic doesn't depend on specific roles
- Flexible: Easily add, remove, or reorder roles



Hiring Approval Process

Benefits

Decoupled: Sender and filters are independent
Flexible: Filters can be added, removed or reordered easily
Extensible: Added new filters without changing existing ones

- **Use Case**

- Job applications move through multiple interview and approval stages
- Each stage evaluates different criteria like skills, fit, and final approval

- **Solution**

- Build a chain: **HR Recruiter -> Technical Interviewer -> Team Lead -> Hiring Manager**
- Each stage reviews and may approve or deny the candidate
- The process starts once the first reviewer receives the application



- **Why Chain of Responsibility?**

- Decoupled: The hiring system doesn't depend on specific reviewers
- Flexible: Easily adjust interview stages or approval levels
- Scalable: Add new reviewers or automated checks without changing existing logic

Message Filtering Pipeline

Benefits

Decoupled: Sender and filters are independent
Flexible: Filters can be added, removed or reordered easily
Extensible: Added new filters without changing existing ones

- **Use Case**

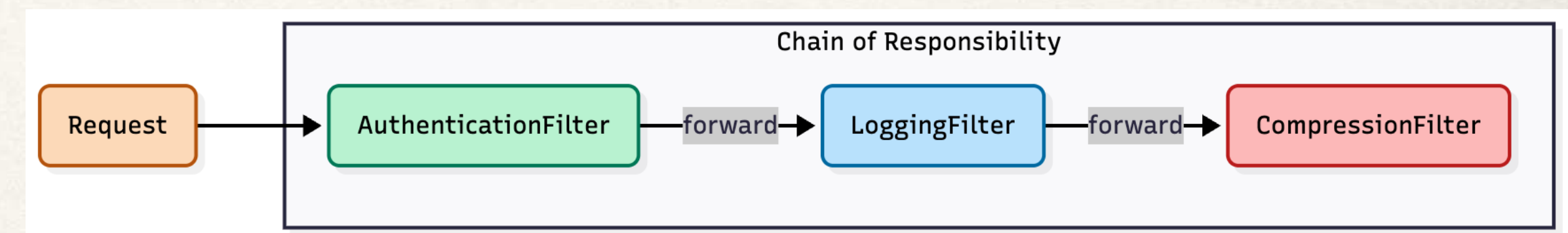
- Systems process incoming messages through multiple transformations
- Each step may validate, log, or compress before delivery

- **Solution**

- Build a chain: **AuthenticationFilter -> LoggingFilter -> CompressionFilter**
- Each filter processes or modifies the message, then may delegate to the next
- The sender triggers only the first filter in the sequence

- **Why Chain of Responsibility?**

- Decoupled: Each filter focuses on one responsibility
- Flexible: Easily reorder or extend filters
- Extensible: New filters can be added without changing existing ones



Chain of Responsibility

Message Filtering Pipeline Example

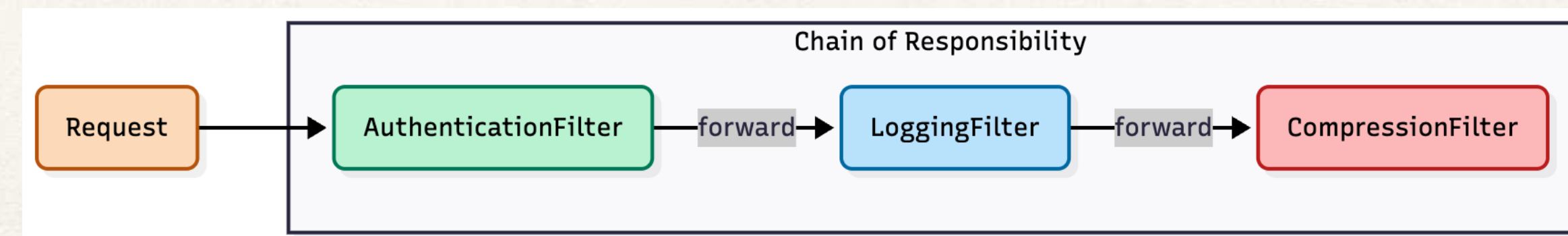


Message Filtering Pipeline

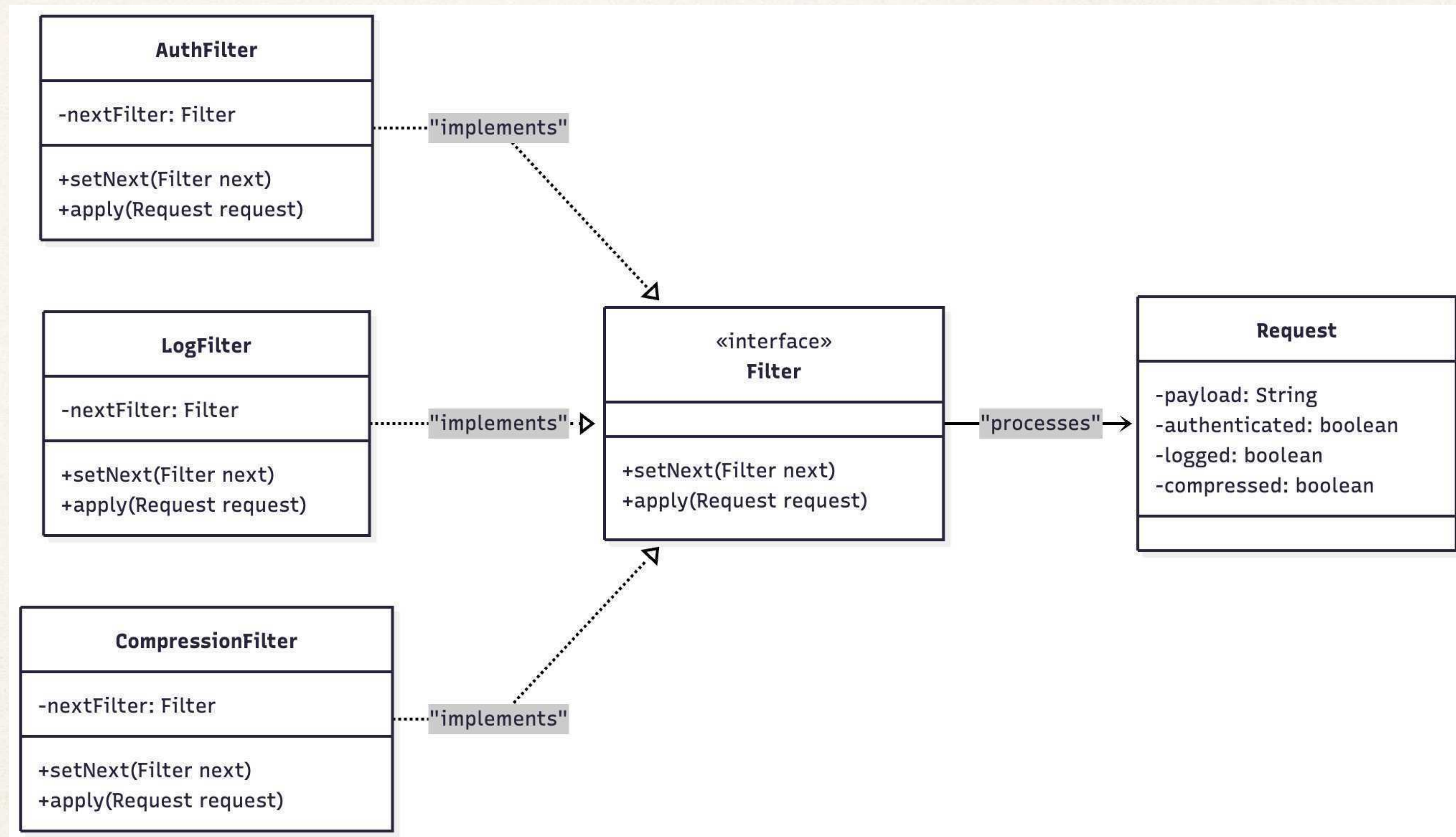
Benefits

Decoupled: Sender and filters are independent
Flexible: Filters can be added, removed or reordered easily
Extensible: Added new filters without changing existing ones

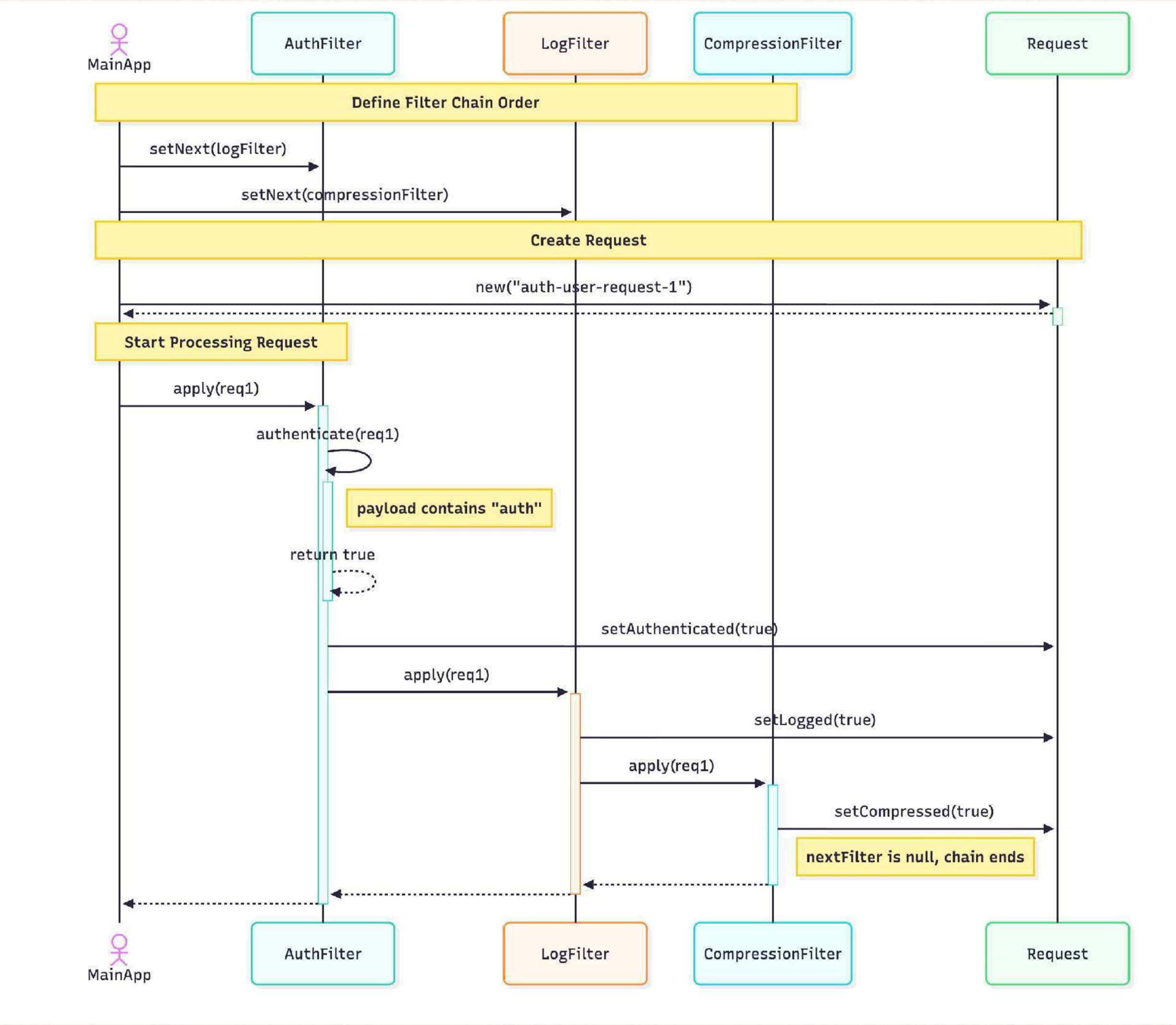
- **Use Case**
 - Systems process incoming messages through multiple transformations
 - Each step may validate, log, or compress before delivery
- **Solution**
 - Build a chain: **AuthenticationFilter -> LoggingFilter -> CompressionFilter**
 - Each filter processes or modifies the message, then may delegate to the next
 - The sender triggers only the first filter in the sequence



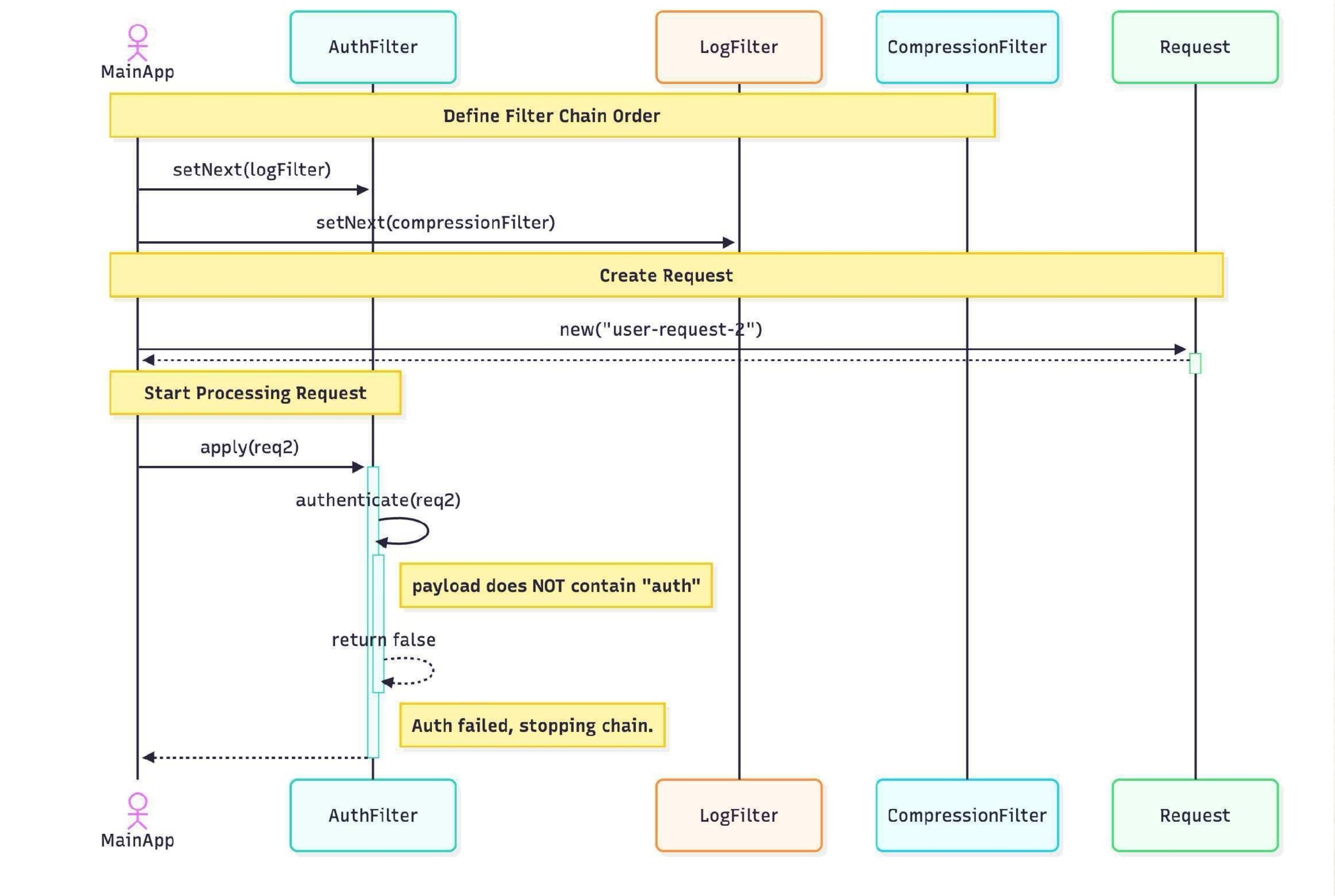
UML Diagram



Sequence Diagram - Request 1 (Auth Success)



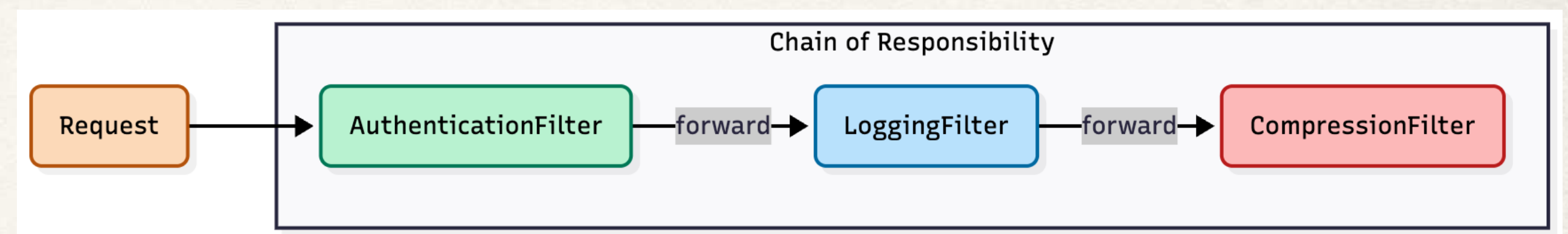
Sequence Diagram - Request 2 (Auth Failure)



Development Process

Step-By-Step

1. Create the Request class
2. Define interface: Filter
3. Develop Filter implementations
 - AuthFilter, LoggingFilter, CompressionFilter
4. Create MainApp
 - Define filter chain order
 - Start processing the request



Iterator Design Pattern



Iterator Design Pattern

Intent

Problem

Solution

- **Intent**
 - Traverse a collection without exposing its internal structure
 - Provide a standard, consistent way to access elements
 - Support multiple traversal strategies if needed
- **Problem**
 - Collections store data in different internal forms (list, tree, graph)
 - Client code becomes tightly coupled to how data is stored
 - Adding new traversal logic requires modifying collection code

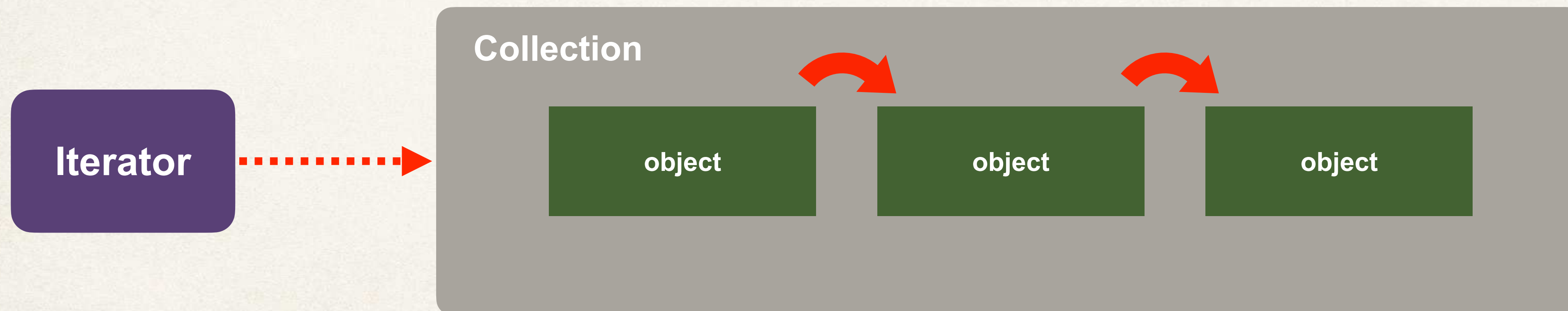
Iterator Design Pattern

Intent

Problem

Solution

- **Solution**
 - Extract traversal logic into a separate iterator object
 - The iterator tracks traversal state (current index, completion)
 - Client uses a simple interface: `hasNext ( )` and `next ( )`



Iterator Design Pattern

Intent

Problem

Solution

Benefits

Decoupled: Client doesn't see internal structure

Flexible: Add new traversal strategies easily

Customizable: Iterators can provide different traversal orders

Examples

- Train Schedule
- Song Playlist
- Course Catalog

Train Schedule

Benefits

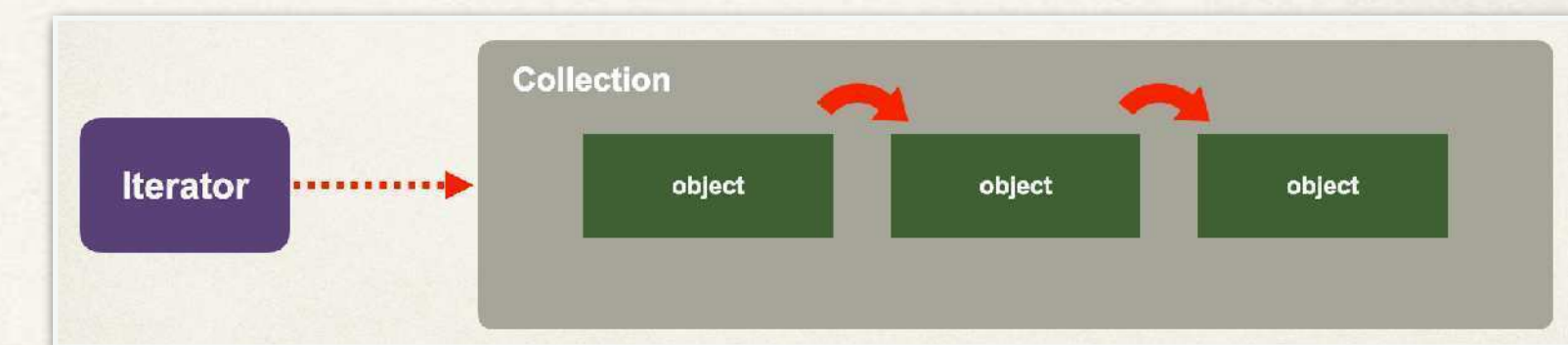
Decoupled: Client doesn't see internal structure
Flexible: Add new traversal strategies easily
Customizable: Iterators can provide different traversal orders

- **Use Case**

- Train systems maintain schedules for many routes and departure times
- Passengers or apps need to list trains without knowing how schedules are stored

- **Solution**

- Provide a `TrainScheduleIterator` to traverse schedule entries in order
- Iterator tracks the current position and returns the next train time
- Clients use simple calls: `hasNext()` and `next()` ... no knowledge of internal structure



- **Why Iterator?**

- Decoupled: Clients don't depend on how schedules are organized
- Flexible: Can support different traversal (earliest first, route-based, express-only)

Song Playlist

Benefits

Decoupled: Client doesn't see internal structure
Flexible: Add new traversal strategies easily
Customizable: Iterators can provide different traversal orders

- **Use Case**

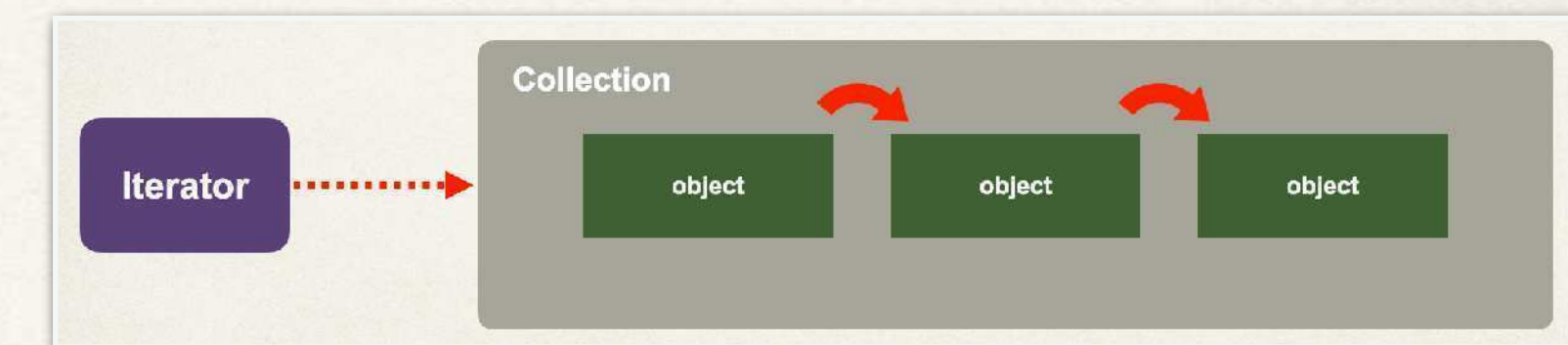
- Users need to play songs in order without knowing how the playlist is stored
- The playlist can change (add, reorder, remove songs)

- **Solution**

- Provide a SongIterator to step through songs one at a time
- Iterator keeps track of the current song position
- Client uses `hasNext ( )` and `next ( )` to play songs in order

- **Why Iterator?**

- Decoupled: Playback code is independent of playlist structure
- Flexible: Different play orders (shuffle, repeat, album order) can be swapped in easily



Course Catalog

Benefits

Decoupled: Client doesn't see internal structure
Flexible: Add new traversal strategies easily
Customizable: Iterators can provide different traversal orders

- **Use Case**

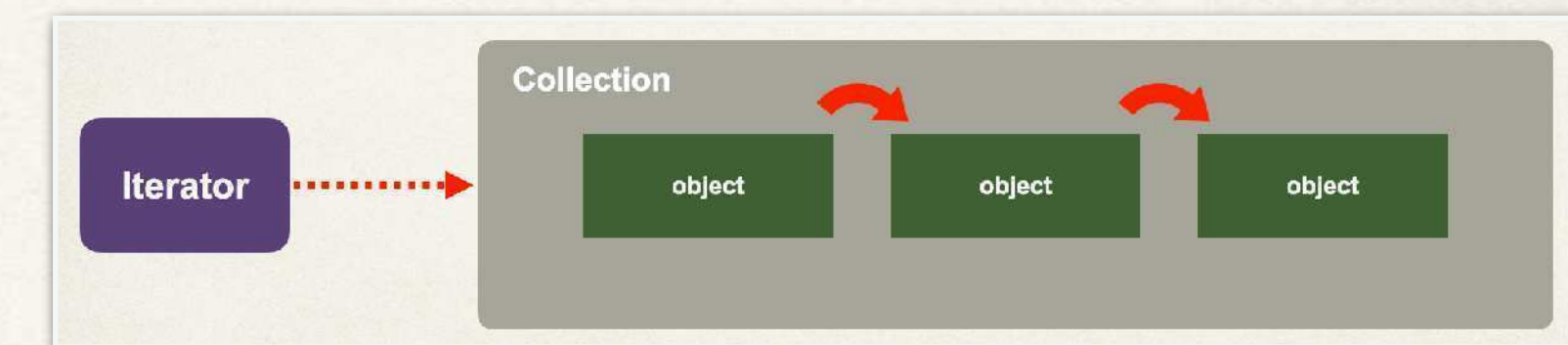
- The application stores many courses for display, search, or reporting
- Clients should access courses without knowing how they're stored internally

- **Solution**

- Introduce a CourseIterator to step through courses sequentially
- Iterator maintains its own position and retrieval logic
- Client interacts only through `hasNext()` and `next()`

- **Why Iterator?**

- Decoupled: Client code is independent of the catalog's internal structure
- Flexible: Catalog storage or ordering can change without affecting clients



Iterator Design Pattern

Course Catalog Example



Course Catalog

Benefits

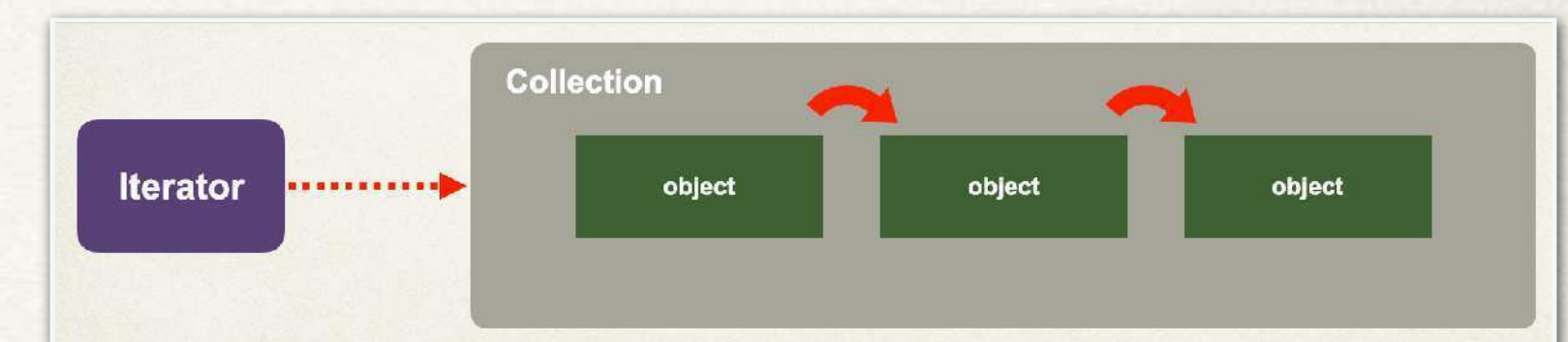
Decoupled: Client doesn't see internal structure
Flexible: Add new traversal strategies easily
Customizable: Iterators can provide different traversal orders

- **Use Case**

- The application stores many courses for display, search, or reporting
- Clients should access courses without knowing how they're stored internally

- **Solution**

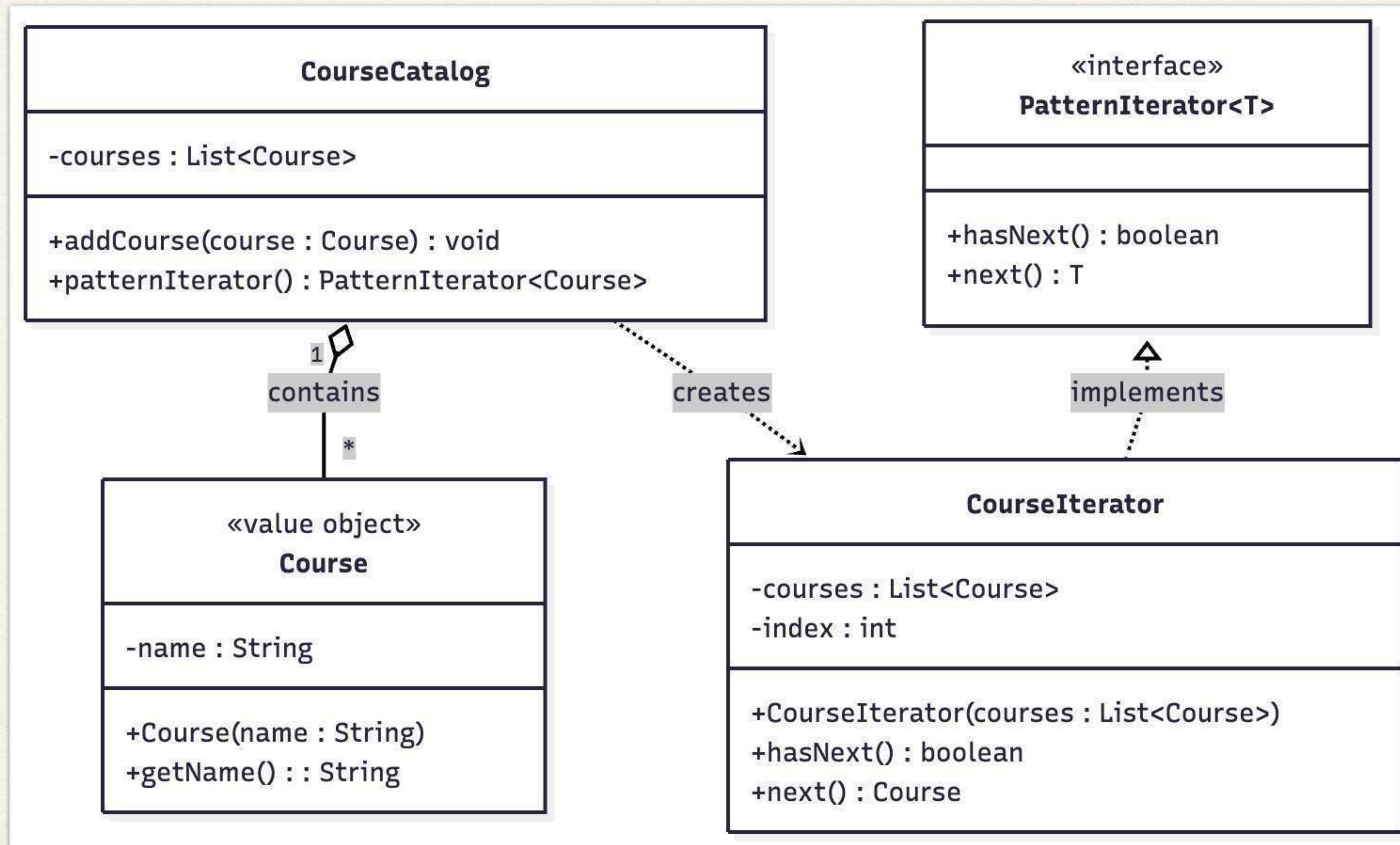
- Introduce a CourseIterator to step through courses sequentially
- Iterator maintains its own position and retrieval logic
- Client interacts only through `hasNext()` and `next()`



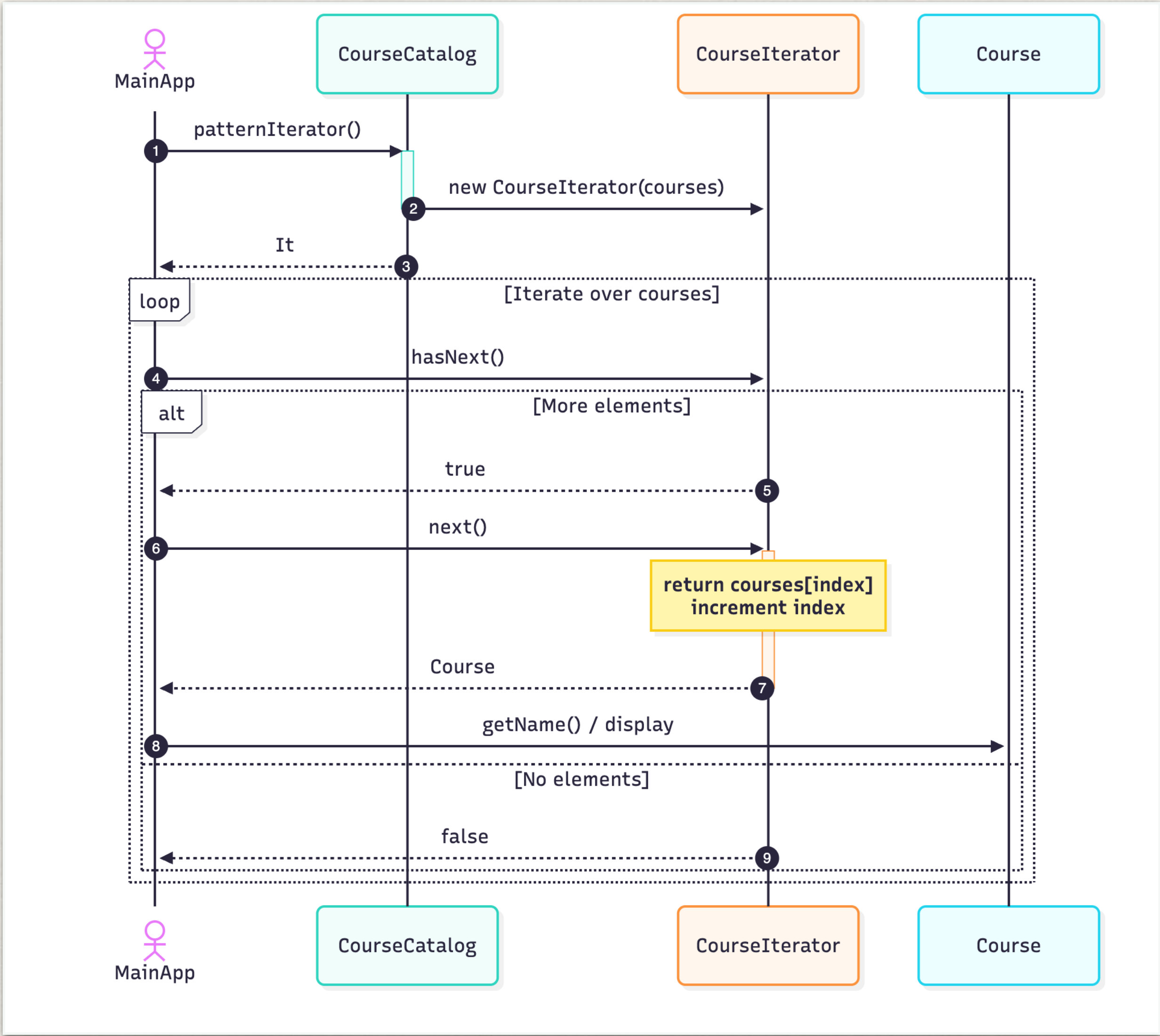
The Components

Component	Role	Description
PatternIterator (interface)	Iterator Interface	Defines the standard operations for traversing a collection: hasNext (), next ()
CourseIterator	Concrete Iterator	Implements traversal logic and maintains the current position within the course list.
CourseCatalog	Aggregate (Collection)	Stores the collection of Course objects and provides a method to create an iterator.
Course	Element	Represents a single item in the collection (a course entry).

UML Diagram



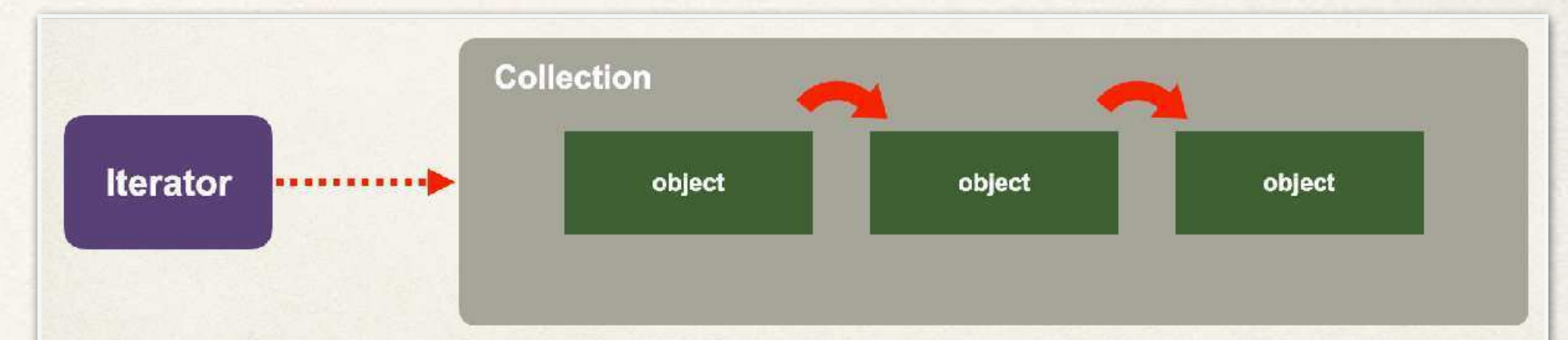
Sequence Diagram



Development Process

Step-By-Step

1. Create the Course class
2. Define interface: PatternIterator
3. Develop Iterator implementation: CourseIterator
4. Create CourseCatalog (aggregate collection)
5. Develop MainApp



Observer Design Pattern



Observer Design Pattern

Intent

Problem

Solution

- **Intent**
 - Set up a one-to-many notification system
 - Update dependent objects automatically
 - Let observers react without the subject knowing how
- **Problem**
 - Many components need updates when shared data changes
 - Polling or direct checks create tight coupling
 - Adding new update logic requires modifying existing code

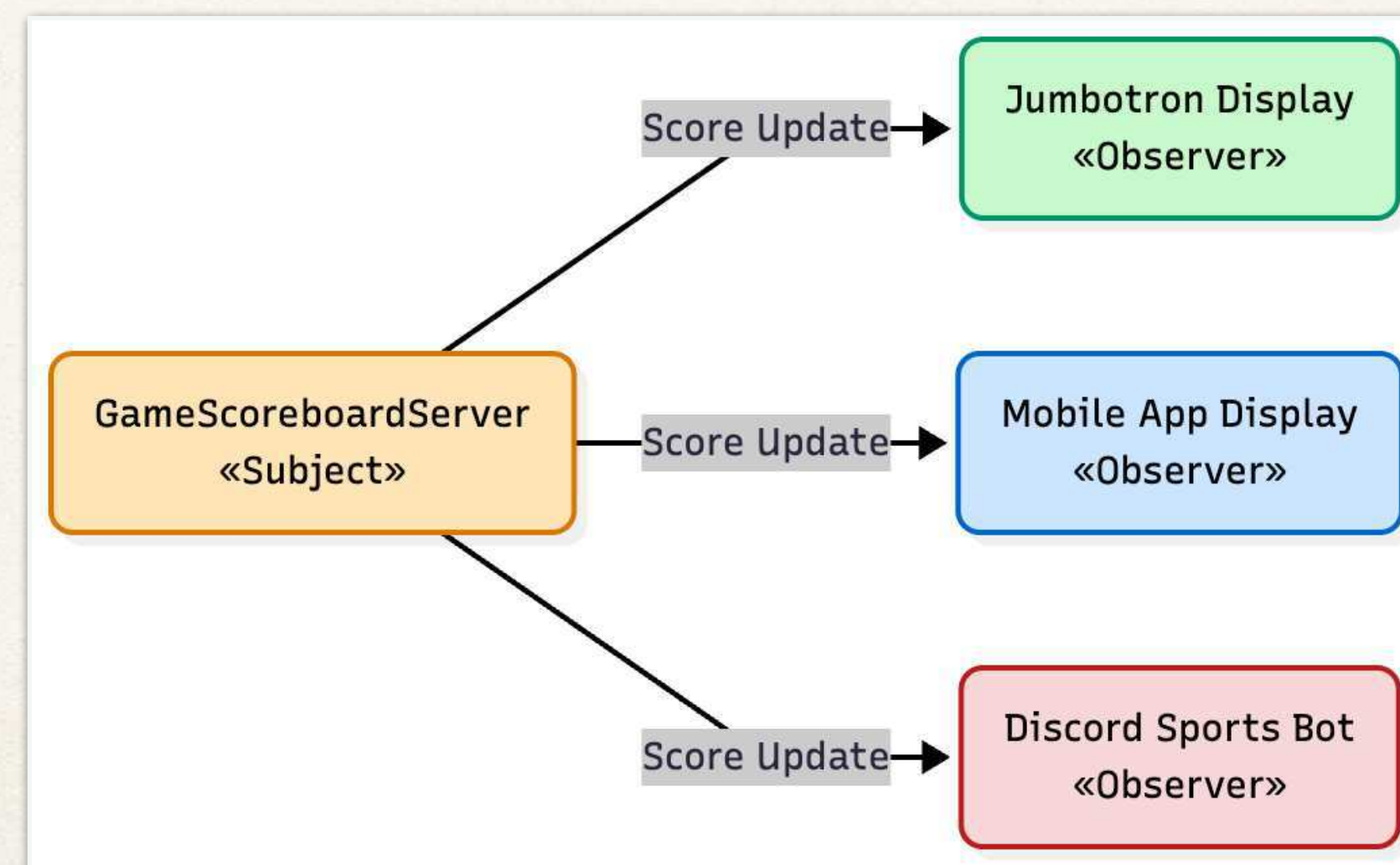
Observer Design Pattern

Intent

Problem

Solution

- **Solution**
 - Have the Subject maintain a list of observers
 - When the Subject's state changes, it notifies all observers
 - Each Observer implements a standard update method to react to changes independently



Observer Design Pattern

Intent

Problem

Solution

Benefits

Decoupled: Subject and observers are loosely connected.
Extensible: Add new observers without changing the subject.
Dynamic: Observers can subscribe or unsubscribe at runtime.

Examples

- Shipment Status Notifications
- Stock Market Ticker
- Sports Scoreboard

Shipment Status Notifications

Benefits

Decoupled: Subject and observers are loosely connected
Extensible: Add new observers without changing the subject
Dynamic: Observers can subscribe or unsubscribe at runtime

- **Use Case**

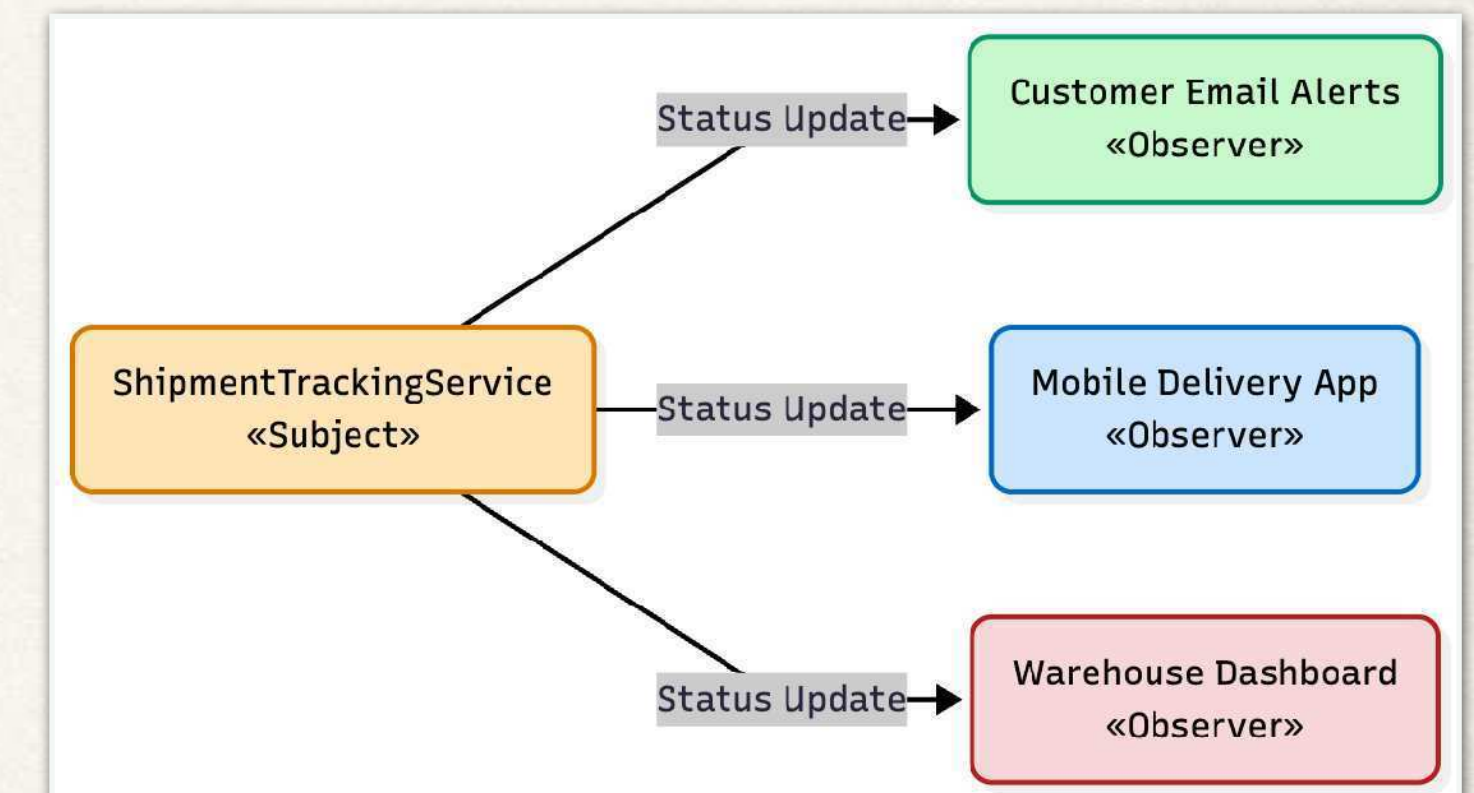
- Shipment status changes (Packed → Shipped → Delivered)
- Must notify customers and internal systems

- **Solution**

- The ShipmentTrackingService (Subject) broadcasts status updates
- Each subscriber is observer and reacts independently

- **Why Observer?**

- Decoupled: Systems receive updates without depending on internal shipment data.
- Flexible: New notification channels can be added without changing the source.



Stock Market Ticker

Benefits

Decoupled: Subject and observers are loosely connected
Extensible: Add new observers without changing the subject
Dynamic: Observers can subscribe or unsubscribe at runtime

- **Use Case**

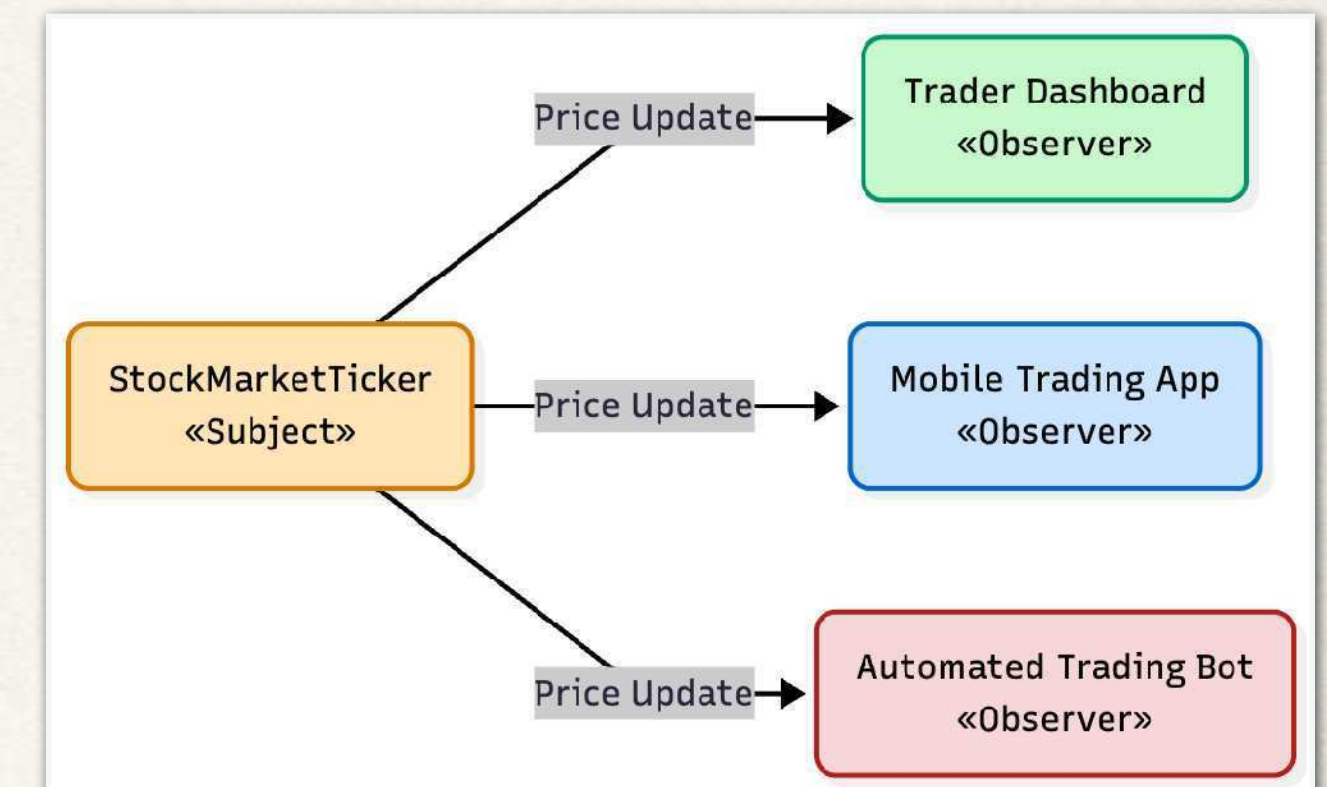
- Stock prices change frequently throughout the trading day
- Multiple systems must stay updated in real time (dashboards, apps, trading bots)

- **Solution**

- The StockMarketTicker (Subject) broadcasts price updates.
- Each subscriber is an observer and reacts independently.

- **Why Observer?**

- Decoupled: Observers don't depend on how price data is stored
- Flexible: New observers can be added without modifying the ticker



Sports Scoreboard

Benefits

Decoupled: Subject and observers are loosely connected
Extensible: Add new observers without changing the subject
Dynamic: Observers can subscribe or unsubscribe at runtime

- **Use Case**

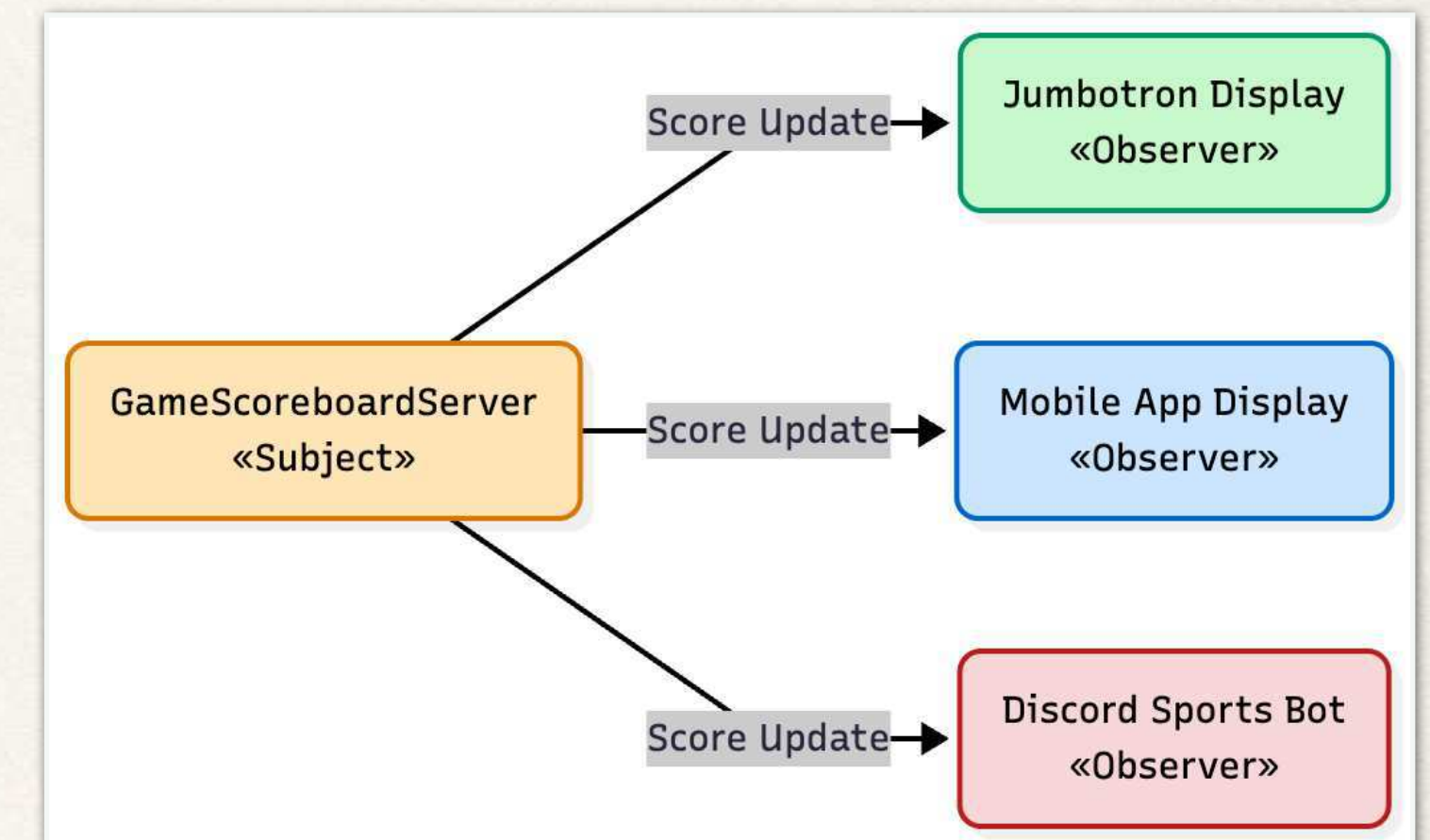
- Game scores change during play
- Multiple displays and apps need to update instantly

- **Solution**

- The GameScoreboardServer (Subject) broadcasts score changes.
- Each subscriber is an observer and updates its own display.

- **Why Observer?**

- Decoupled: Displays don't depend on how the score is stored
- Flexible: New displays can be added without changing the scoreboard



Observer Design Pattern

Sports Scoreboard Example



Sports Scoreboard

Benefits

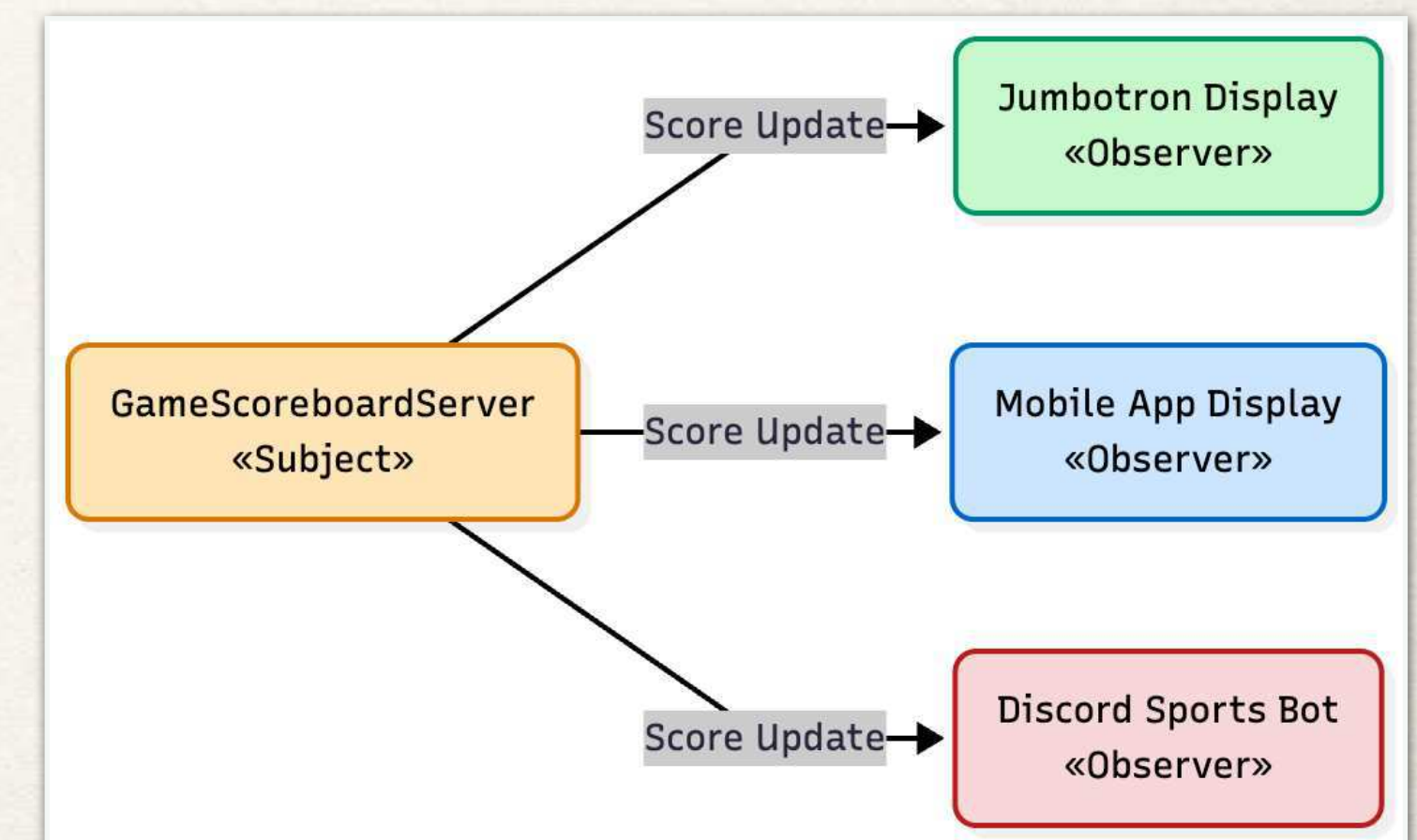
Decoupled: Subject and observers are loosely connected
Extensible: Add new observers without changing the subject
Dynamic: Observers can subscribe or unsubscribe at runtime

- **Use Case**

- Game scores change during play
- Multiple displays and apps need to update instantly

- **Solution**

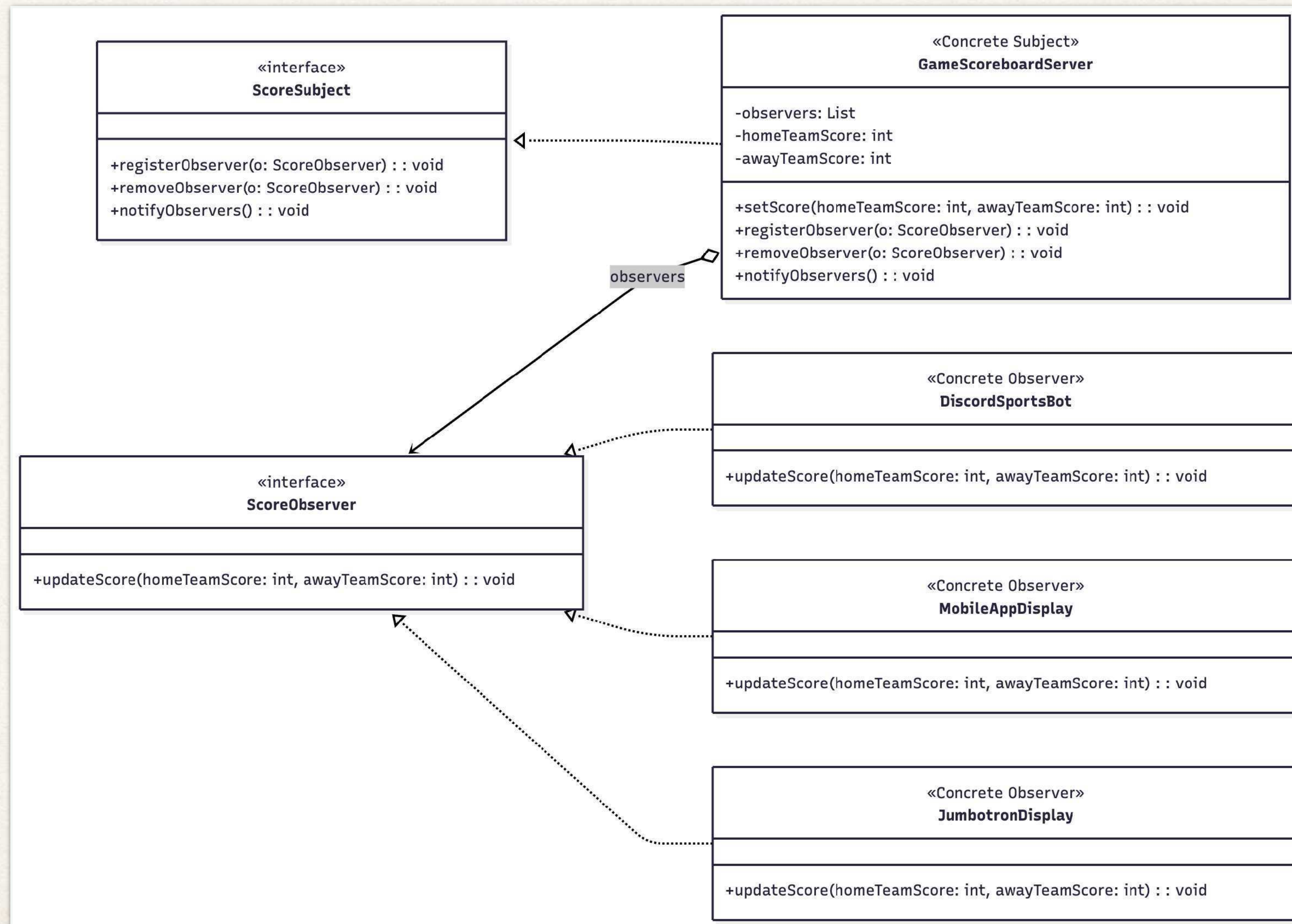
- The GameScoreboardServer (Subject) broadcasts score changes.
- Each subscriber is an observer and updates its own display



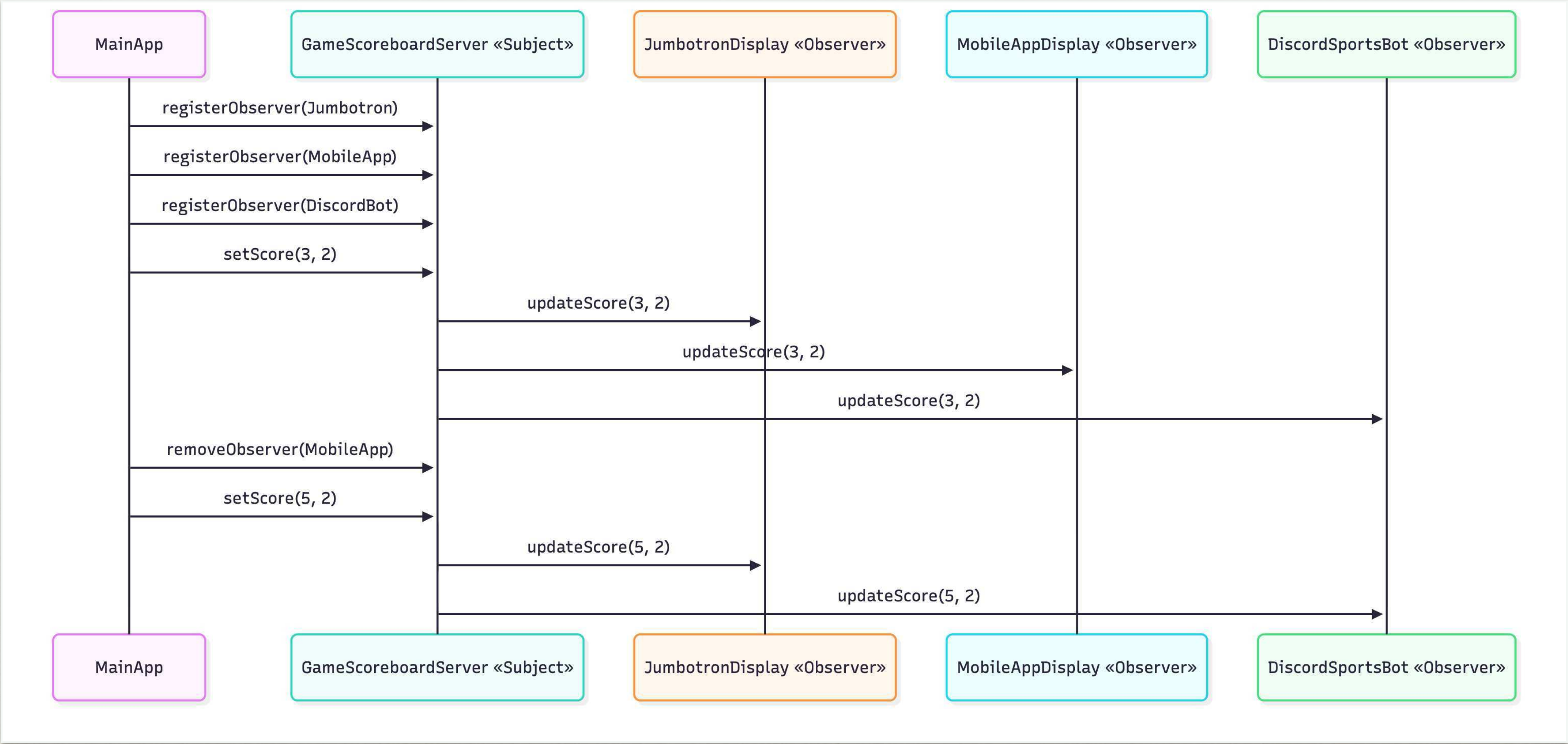
The Components

Component	Role	Description
ScoreSubject	Subject Interface	Defines methods to register, remove, and notify observers.
ScoreObserver	Observer Interface	Defines the update method observers must implement.
GameboardScoreServer	Concrete Subject	Stores the current score and notifies observers when it changes.
JumbotronDisplay	Concrete Observer	Updates the large in-stadium scoreboard when score changes.
MobileAppDisplay	Concrete Observer	Updates score shown to users in a mobile app.
DiscordSportsBot	Concrete Observer	Posts score updates automatically to a Discord channel.

UML Diagram



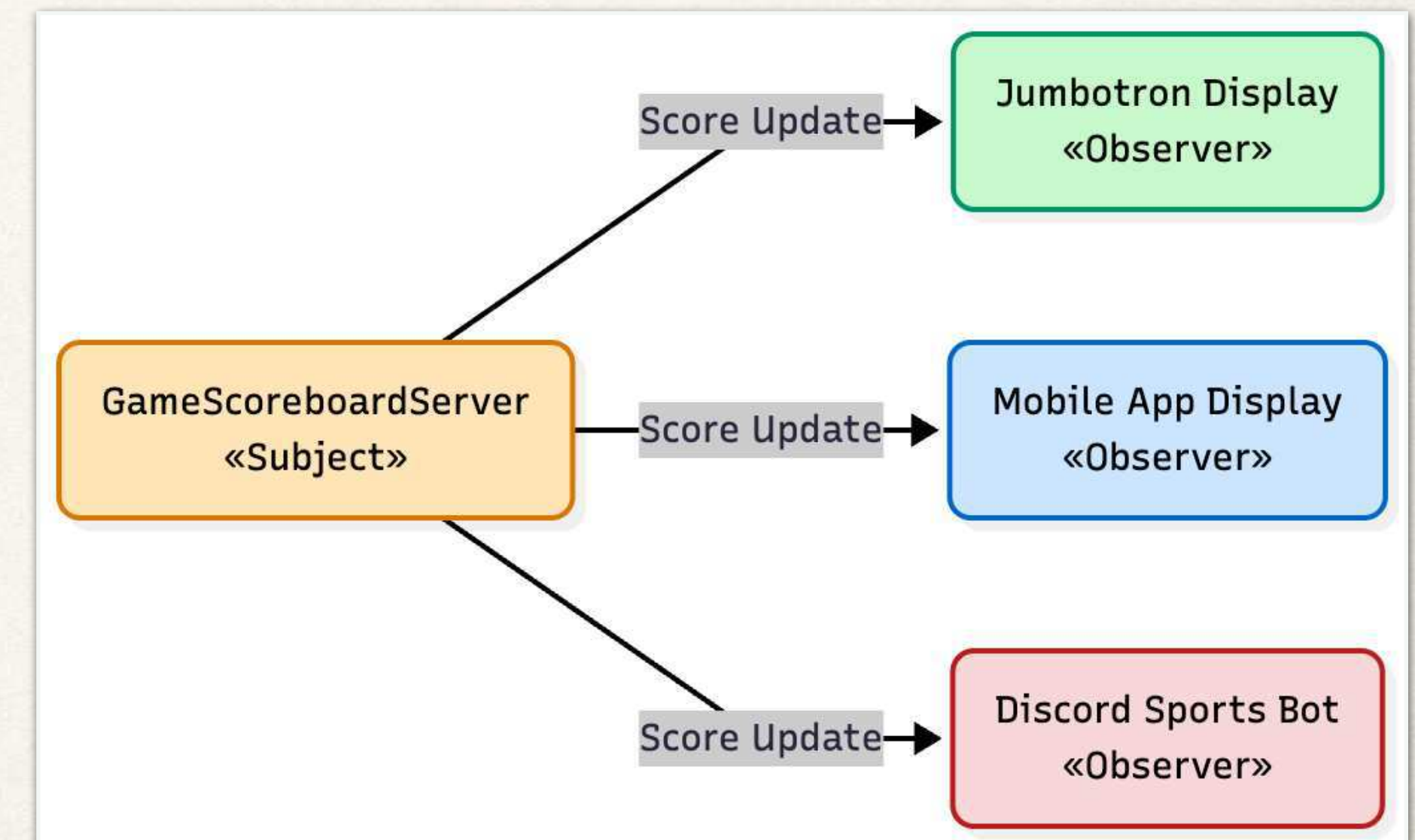
Sequence Diagram



Development Process

Step-By-Step

1. Define the ScoreObserver interface
2. Define the ScoreSubject interface
3. Develop the GameScoreboardServer (Concrete Subject)
4. Create Observer Implementations
 - JumbotronDisplay, MobileAppDisplay
 - DiscordSportsBot
5. Develop MainApp



Strategy Design Pattern



Strategy Design Pattern

Intent

Problem

Solution

- **Intent**
 - Define a family of algorithms and make them interchangeable
 - Vary behavior at runtime without changing the client code
 - Isolate how something is done from what needs to be done
- **Problem**
 - Algorithms are often hard-coded inside classes
 - Changing behavior requires modifying existing code
 - Multiple “if / else / switch” blocks cause rigidity and duplication

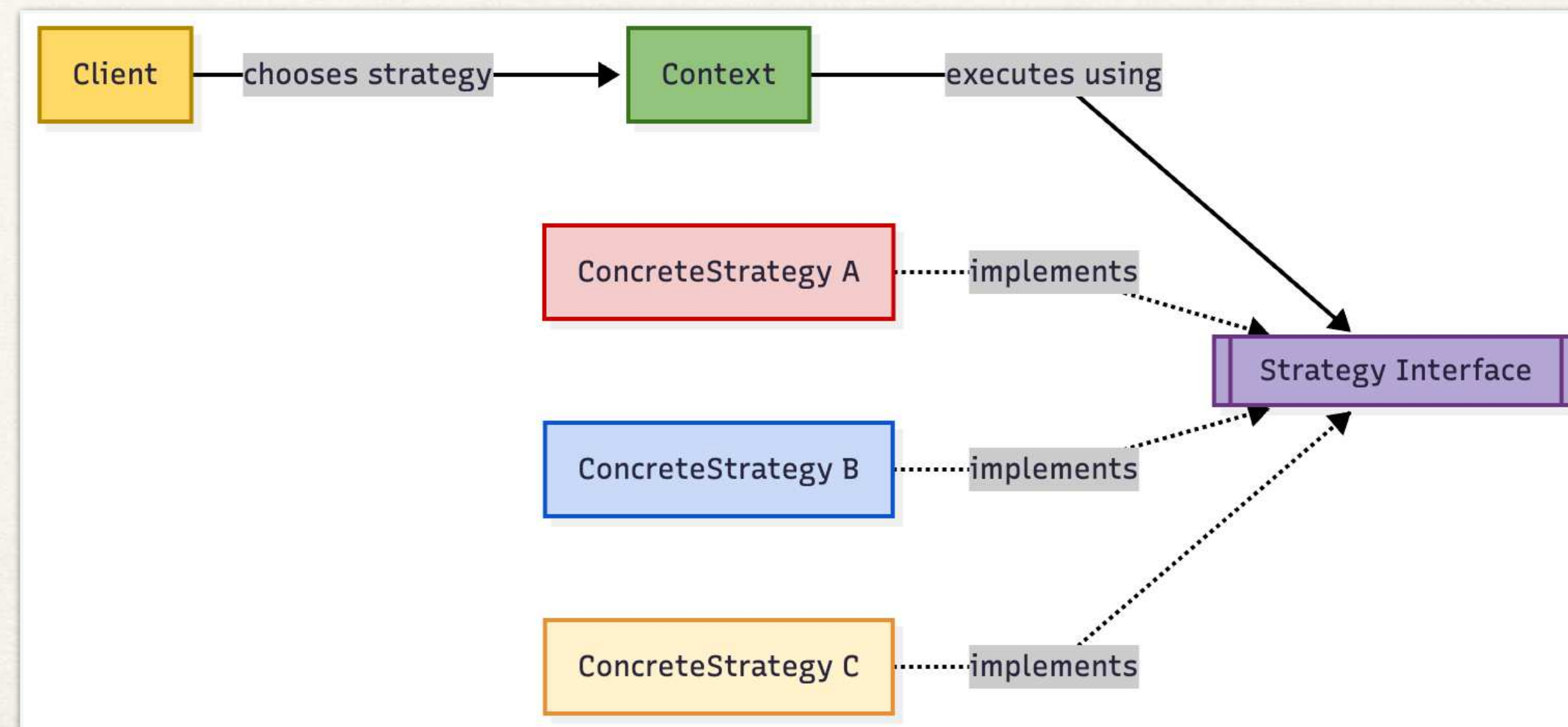
Strategy Design Pattern

Intent

Problem

Solution

- **Solution**
 - Extract the varying behavior into a **Strategy** interface
 - Implement different behaviors as **Concrete Strategies**
 - The **Context** holds a reference to a **Strategy** and delegates to it



Strategy Design Pattern

Intent

Problem

Solution

Benefits

Flexible: behavior can be swapped at runtime
Cleaner code: removes if/else chains and duplicated logic
Extensible: add new behavior without modifying existing code

Examples

- Shipping Cost Calculation
- Route Planning
- Course Catalog Sorter

Shipping Cost Calculation

Benefits

Flexible: behavior can be swapped at runtime
Cleaner code: removes if/else chains and duplicated logic
Extensible: add new behavior without modifying existing code

- **Use Case**

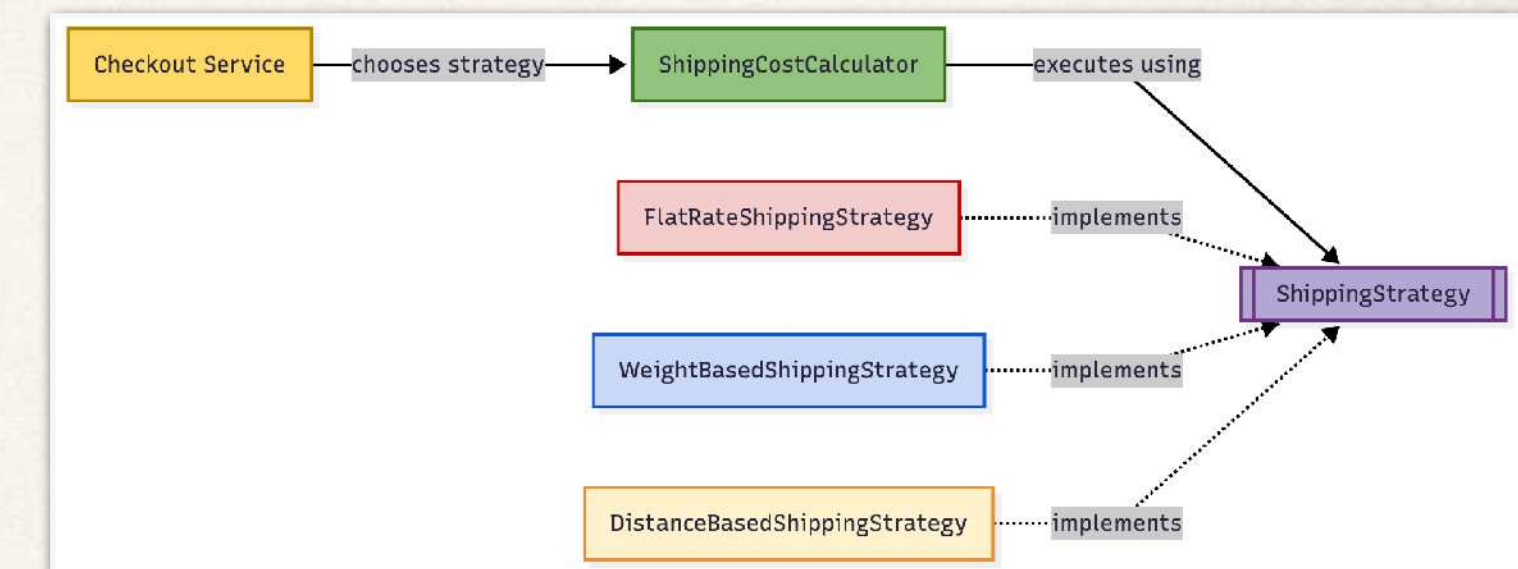
- Shipping costs differ by pricing model (flat, weight, distance)
- Must support new models without changing checkout code

- **Solution**

- ShippingCostCalculator delegates cost calculation to a ShippingStrategy
- The calculator forwards cost computation to the selected strategy

- **Why Strategy?**

- Decoupled: Checkout logic doesn't depend on how shipping is calculated
- Flexible: New pricing models can be added or swapped without changing existing code



Route Planning

Benefits

Flexible: behavior can be swapped at runtime
Cleaner code: removes if/else chains and duplicated logic
Extensible: add new behavior without modifying existing code

- **Use Case**

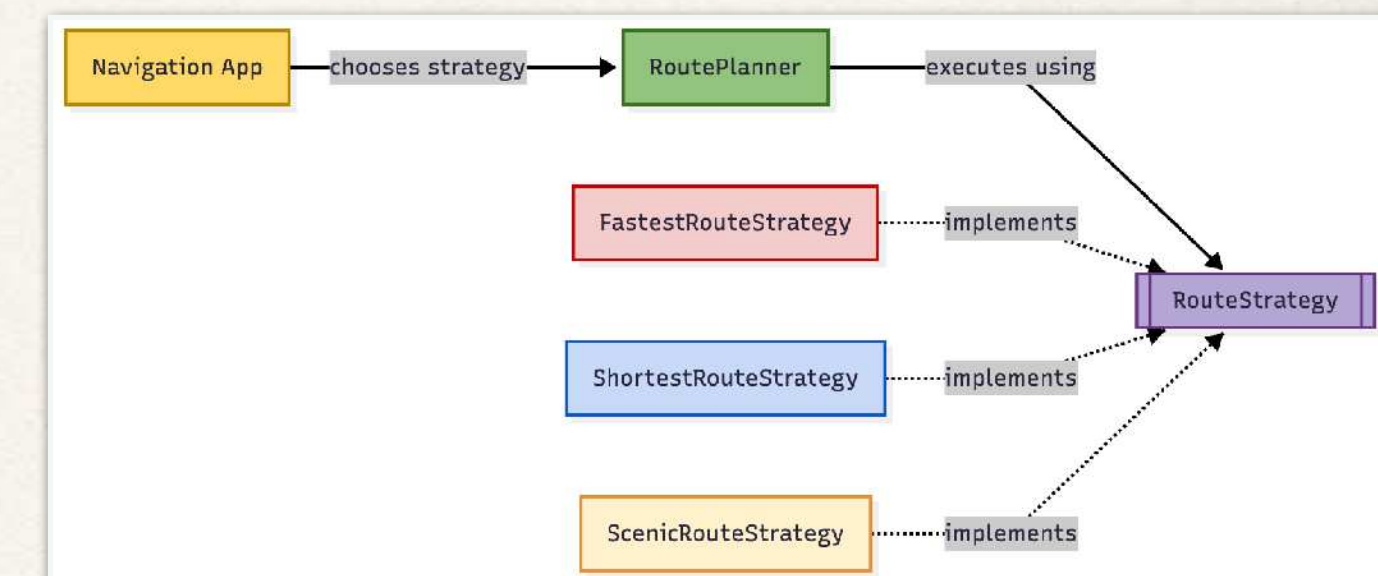
- Compute routes under different criteria (fastest, shortest, scenic, avoid tolls)
- Switch logic per user preference, traffic, or restrictions

- **Solution**

- RoutePlanner delegates route calculation to a RouteStrategy
- The planner uses the selected strategy to compute the route at runtime

- **Why Strategy?**

- Decoupled: Planner doesn't depend on a specific routing algorithm
- Flexible: Add / swap algorithms or choose at runtime without changing client code



Course Catalog Sorter

Benefits

Flexible: behavior can be swapped at runtime
Cleaner code: removes if/else chains and duplicated logic
Extensible: add new behavior without modifying existing code

- **Use Case**

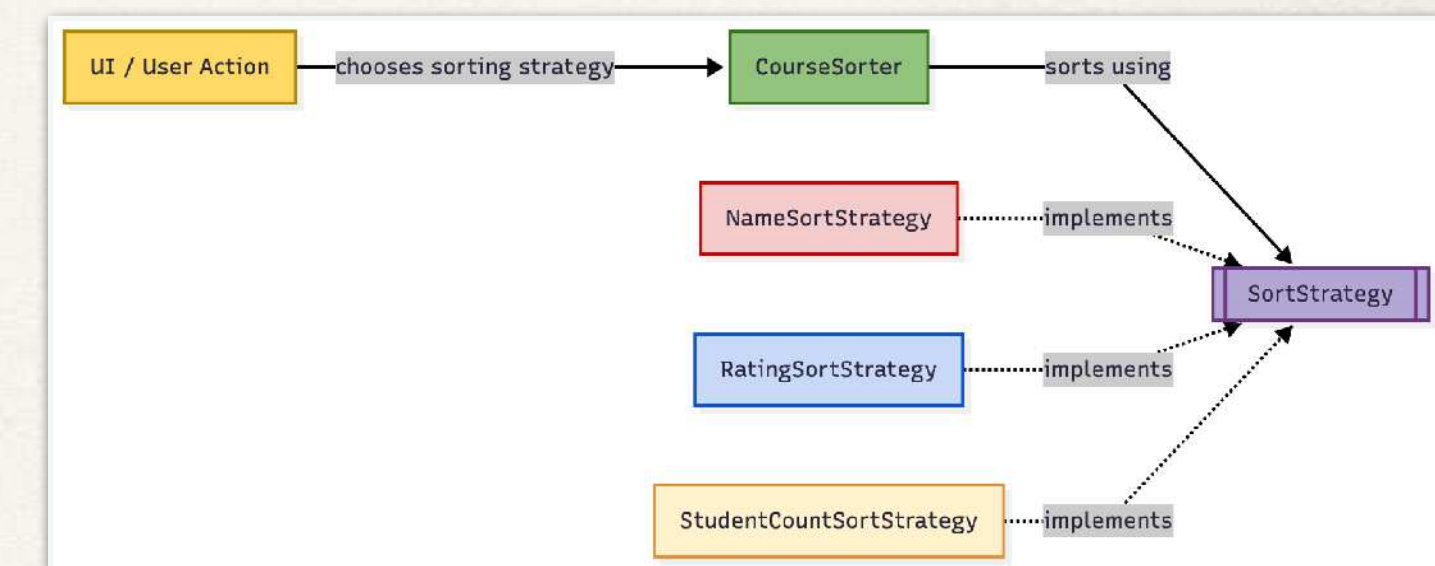
- Sort courses by different criteria (name, rating, student count)
- Allow users or UI filters to change sorting behavior dynamically

- **Solution**

- CourseSorter delegates sorting to a SortStrategy
- The sorter uses the selected strategy to reorder the course list at runtime

- **Why Strategy?**

- Decoupled: Sort logic is separated from the catalog display or UI
- Flexible: New sorting rules can be added or switched without modifying existing code



Strategy Design Pattern

Course Catalog Sorter Example



Course Catalog Sorter

Benefits

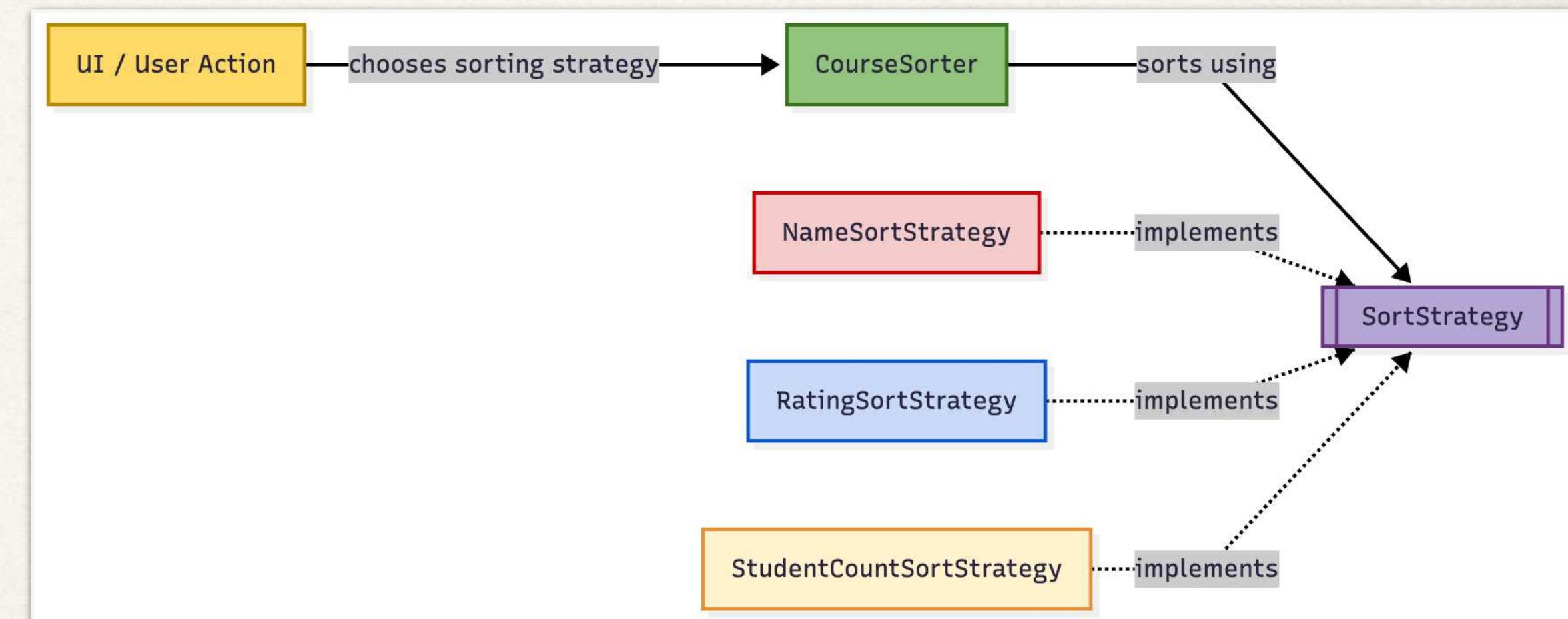
Flexible: behavior can be swapped at runtime
Cleaner code: removes if/else chains and duplicated logic
Extensible: add new behavior without modifying existing code

- **Use Case**

- Sort courses by different criteria (name, rating, student count)
- Allow users or UI filters to change sorting behavior dynamically

- **Solution**

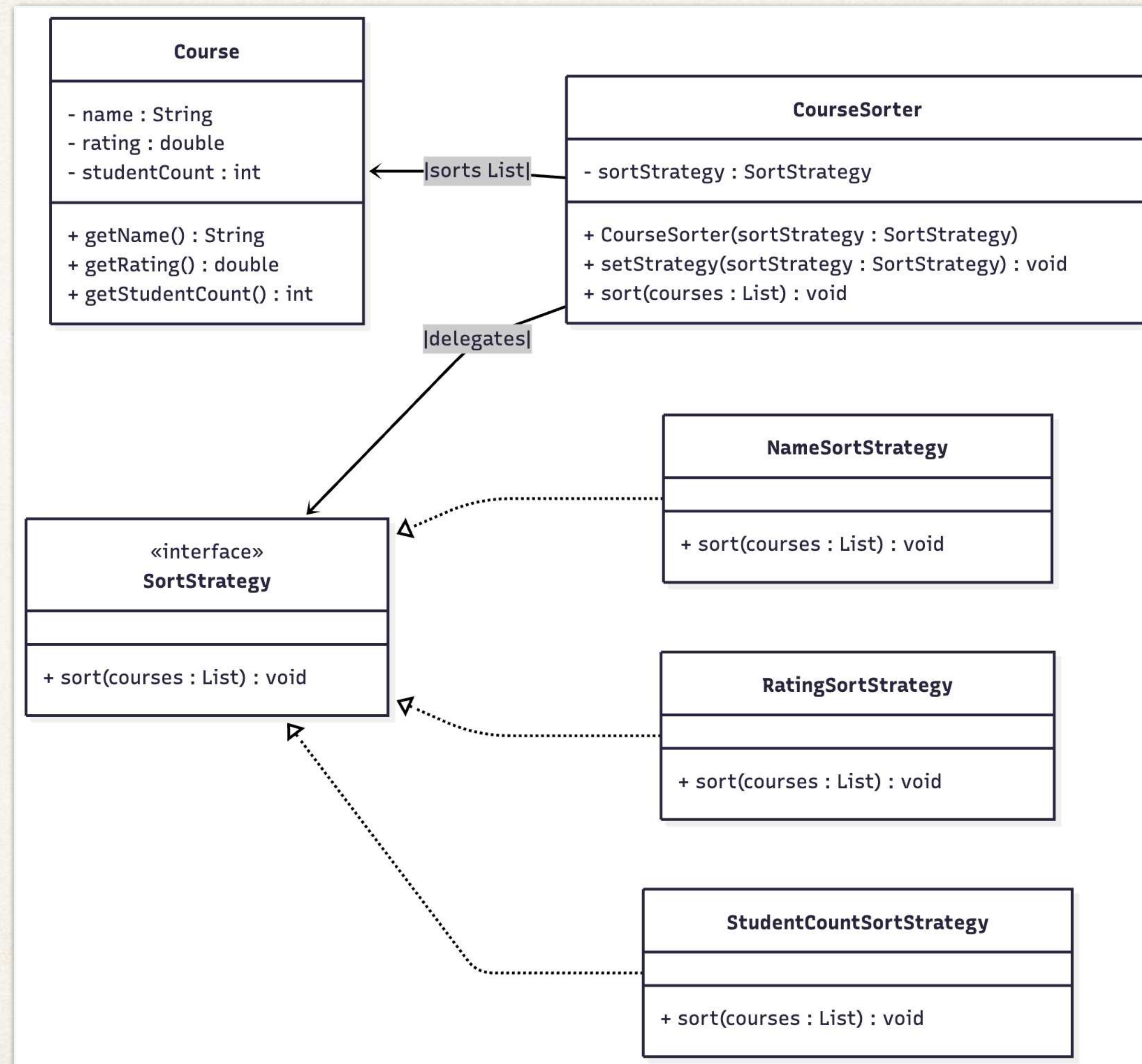
- CourseSorter delegates sorting to a SortStrategy
- The sorter uses the selected strategy to reorder the course list at runtime



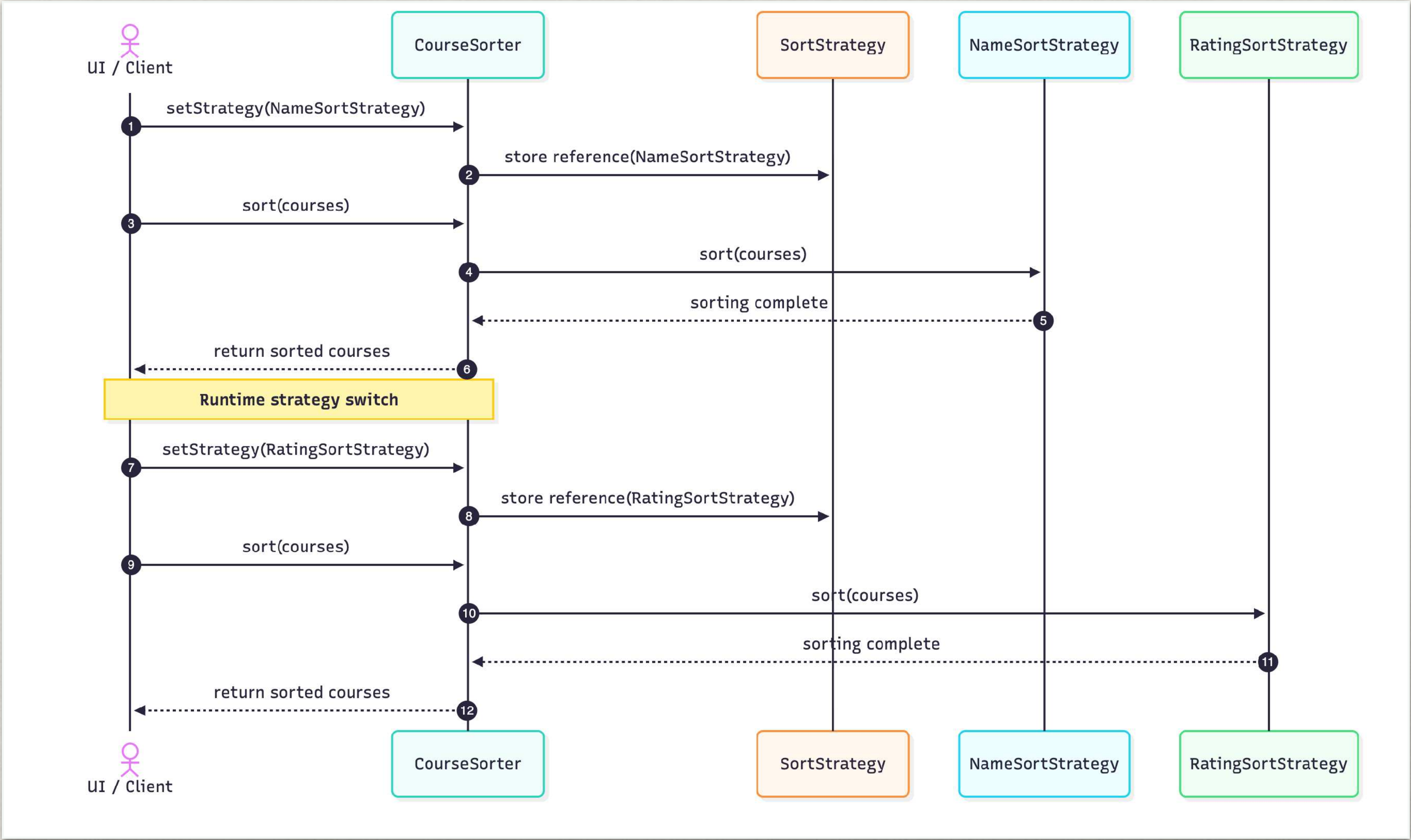
The Components

Component	Role	Description
SortStrategy	Strategy Interface	Defines the common method for sorting courses.
NameSortStrategy	Concrete Strategy	Sorts courses alphabetically by name.
RatingSortStrategy	Concrete Strategy	Sorts courses by rating.
StudentCountSortStrategy	Concrete Strategy	Sorts courses by number of enrolled students.
CourseSorter	Context	Holds a reference to a SortStrategy and delegates sorting to it.

UML Diagram



Sequence Diagram



Development Process

1. Define the SortStrategy interface

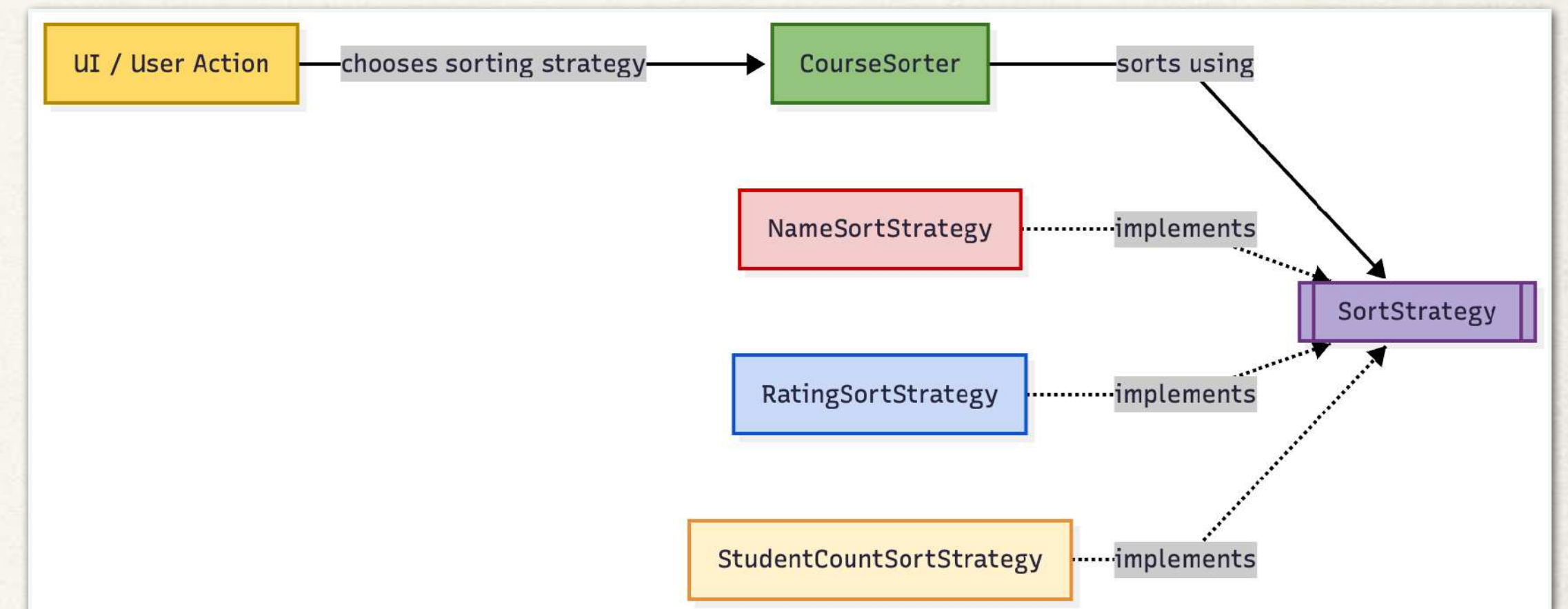
2. Implement concrete strategies

- NameSortStrategy
- RatingSortStrategy
- StudentCountSortStrategy

3. Implement the CourseSorter to delegate sorting

4. Develop MainApp

Step-By-Step



Course Summary



Java Design Patterns

**Mission
Accomplished**

Creational

Factory
Singleton
Prototype

...



Structural

Adapter
Bridge
Composite

...



Behavioral

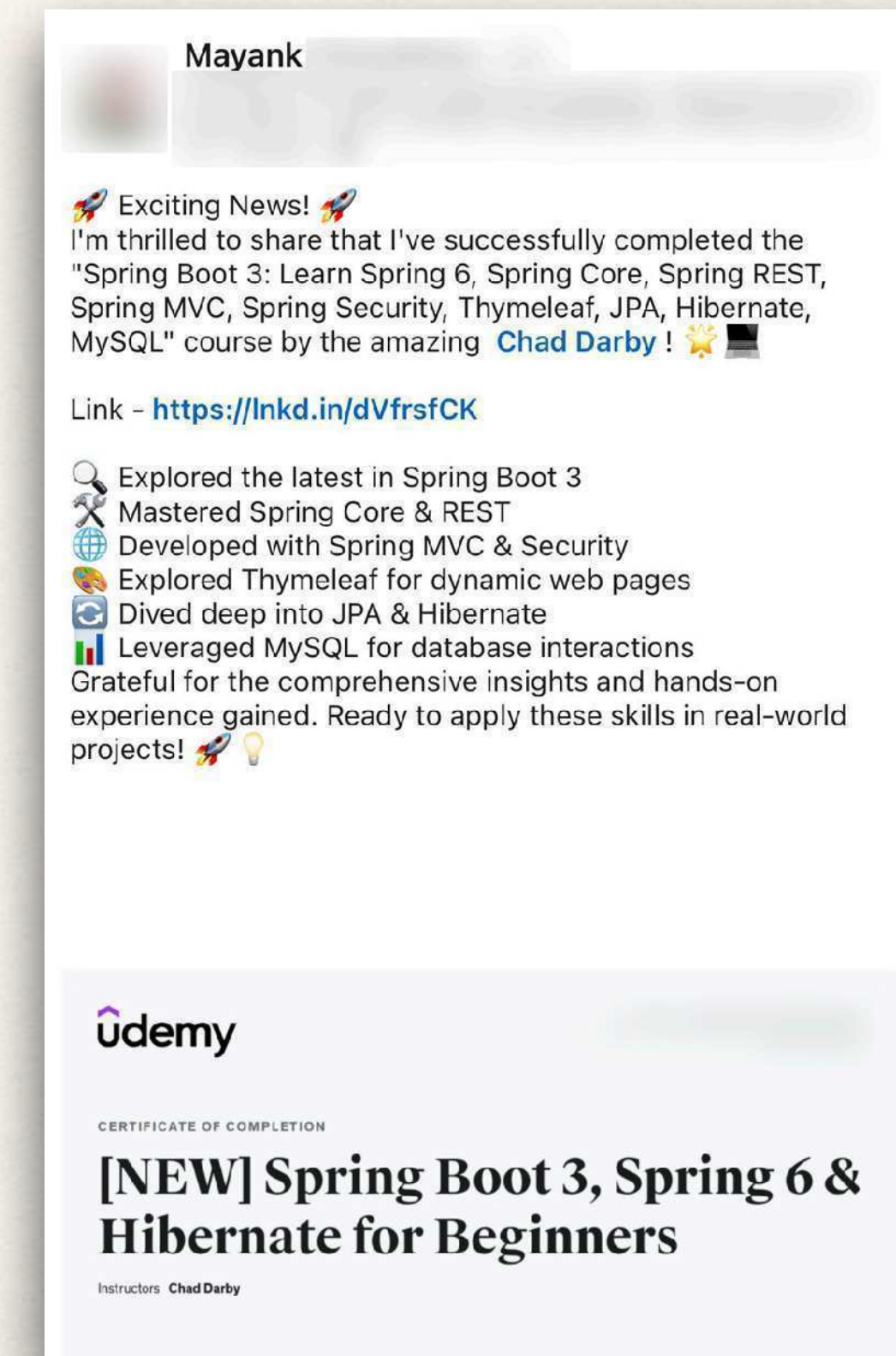
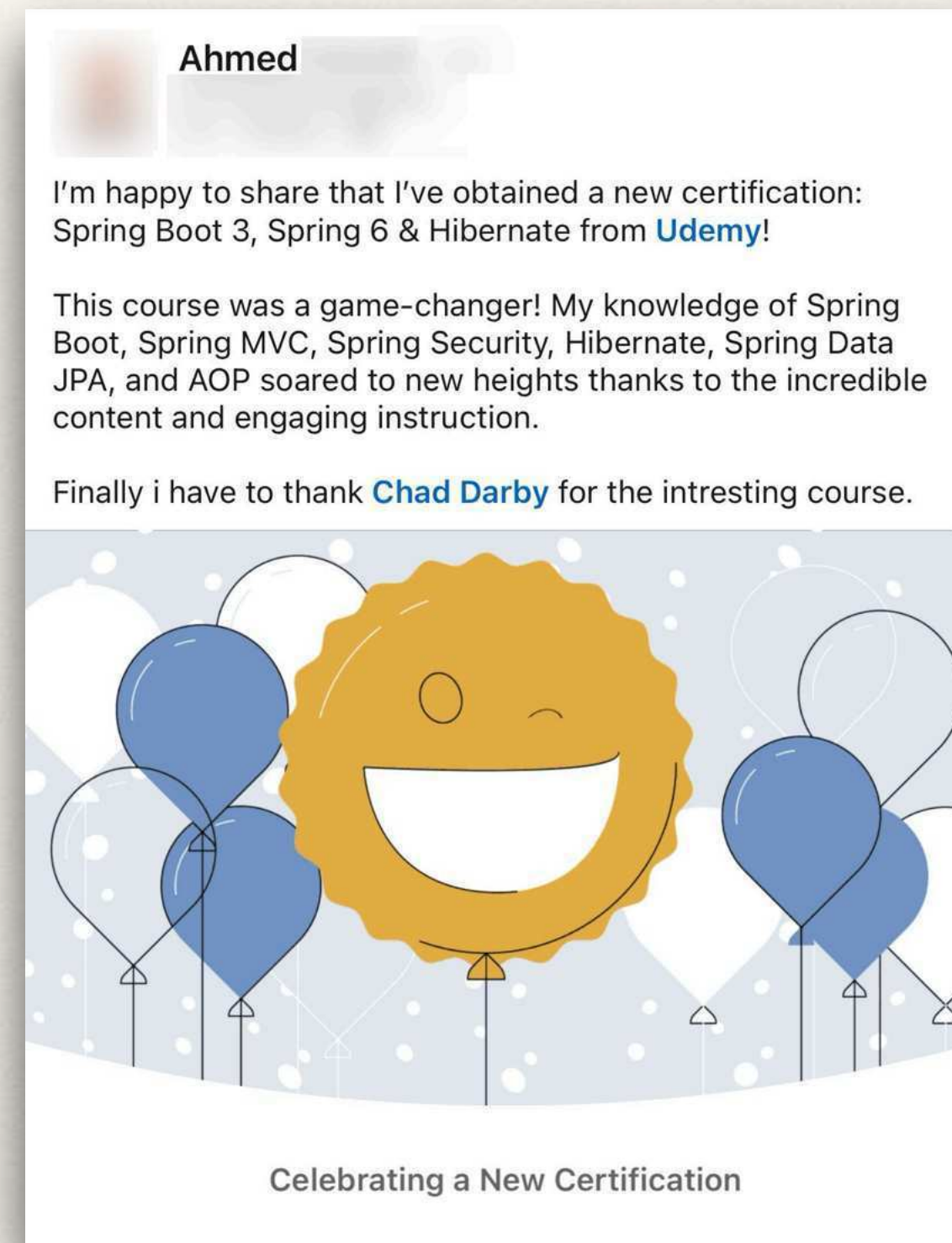
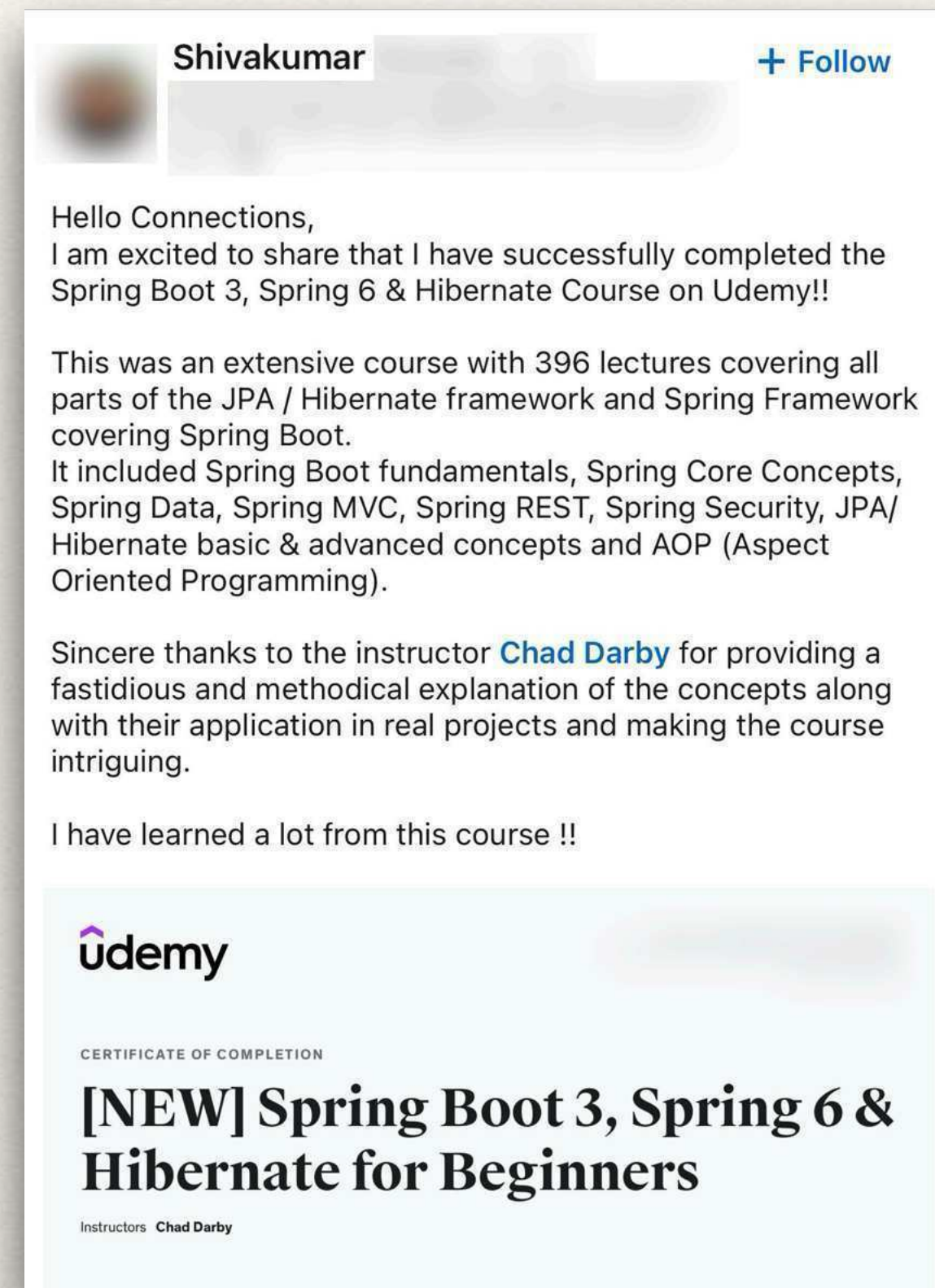
Iterator
Observer
Strategy

...



Share Your Accomplishment

- Download your course certificate and share it on social media!



Please Rate the Course

 Leave a rating

Your progress 





Contact Me

darby@luv2code.com

www.luv2code.com